

分类号: TP391

密 级:

学 号: 19308207034



西安科技大学
XI'AN UNIVERSITY OF SCIENCE AND TECHNOLOGY

硕士学位论文

Thesis for Master's Degree

基于 TVM 的卷积优化和计算图划分并行 调度方法研究与实现

申请人姓名: 王朝闻

指导教师: 蒋林

类 别: 全日制专业学位硕士

工程领域: 计算机技术

研究方向: 机器学习

2022 年 6 月

论文题目：基于 TVM 的卷积优化和计算图划分并行调度方法研究与实现

学科名称：计算机技术

硕 士 生：王朝闻 (签名)

指导老师：蒋林 (签名)

摘 要

随着人工智能技术的发展，诸如卷积神经网络（Convolution Neural Network, CNN）等各类人工智能算法及各类硬件平台的出现，增加了算法在不同平台上部署和开发的难度。张量虚拟机（Tensor Virtual Machine, TVM）作为通用的神经网络编译器，可以对不同类型的神经网络进行优化处理，在硬件平台生成高度优化的底层代码，已成为人工智能领域的主要优化部署平台之一。但是，不同设备因负载和通信开销等因素引起性能瓶颈，出现难以充分利用硬件资源的问题。因此，本文基于 CNN 算法的行为分析，提出一种基于 TVM 的卷积优化和计算图划分并行调度方法。

为了挖掘 CNN 算法中的分支信息和卷积特征信息，深入分析基于 TVM 的 CNN 算法行为模式。首先，采用后序遍历方式对计算图进行分支信息提取，获取分支开始、结束以及分支内部信息，使用分支信息构建特征计算图。其次，借助 TVM 中间表示，提取卷积的访存特征和计算特征。最后，根据得出的分支信息，对计算图进行手动划分实验结果表明，依据分支特征的划分方法与 TVM 传统方法对比，平均获得了 18% 的速度提升。基于卷积特征的卷积优化方法与传统方法对比，平均获得了 20% 的速度提升，能够有效挖掘特征信息。

针对内存优化卷积算法（Memory Efficient Convolution, MEC）在传统设备下因访问数据地址不连续导致的访存时间长等问题，提出一种适用于 MEC 算法访存行为的优化方法。该方法分为中间矩阵转换和矩阵运算两部分，首先，对于中间矩阵转换部分，采用修改数据读取顺序的方式对其进行优化，使数据读取方式符合算法的访存行为。其次，对于矩阵运算部分，采用更加适合矩阵运算的内存数据布局对卷积核矩阵修改，并利用 TVM 平台封装的计算函数，重新设计中间矩阵同卷积核矩阵的计算方式。最后，使用平台自带并行库对运算过程进行加速。实验结果表明，与传统 MEC 算法相比，在单个卷积层上平均获得了 50% 的速度提升，在多层神经网络中平均获得了 57% 的速度提升。

针对 TVM 中计算图划分方法依赖专家经验、且划分策略处理单一问题，提出一种基于分支特征子图划分方法。首先，基于计算图分支特征信息，对计算图进行前向遍

历，查找分支开始和结束节点，并切分和存储到数组中。其次，提取数组中各节点进行子图构建，统计各个子图间输入输出依赖配置，并存储到数组中。最后，基于子图间依赖信息对子图进行输入输出依赖配置，并进行参数和设备信息的选择与配置。实验结果表明，在分别采用 48 和 96 个 CPU 核心数时，与传统 TVM 运行机制对比，本文方法中 CNN 算法的推理速度提升了 20%和 15%，有效实现计算图的划分。

针对 TVM 中单个子图无法并行问题，设计并实现一种分支并行方法。首先，设计有向无环图和后序支配树，记录节点的键值、顺序和依赖关系等信息。其次，基于上述信息对分支进行搜索，并将分支节点打包成函数，标记为并行节点。第三，完成并行图标记后，在并行运行阶段对计算图进行处理，涉及并行线程池设计，数据交互及运行操作。实验结果表明，相比于 TVM 传统串行方法，分支并行方法在 CPU 和 GPU 上推理速度分别提升了 10%和 20%，相比于 Greedy 算法，本文算法推理速度平均提升了 5%，能够有效利用硬件设备资源。

关键词：张量虚拟机；内存优化算法；卷积神经网络；行为模式；计算图划分；并行化；

研究类型：应用研究

Subject : Research and implementation of convolution optimization and parallel scheduling based on TVM

Subject Name : Computer Technology

Name : Wang Zhaowen (Signature)

Instructor : Jiang Lin (Signature)

ABSTRACT

With the development of artificial intelligence technology, the emergence of various artificial intelligence algorithms such as Convolution Neural Network (CNN) increases the difficulty of algorithms deployment and development on different platforms. The Tensor Virtual Machine (TVM), as a universal neural network compiler, can optimize different types of neural networks and generate highly optimized underlying code on hardware platforms. It has become one of the major optimization deployment platforms in the field of artificial intelligence. However, different devices are difficult to make full use of hardware resources due to performance bottlenecks caused by load and communication overhead. Therefore, this thesis based on the behavior analysis of CNN algorithm proposes a parallel scheduling method of convolution optimization and computational graph partition based on TVM.

In order to mine the branch information and convolution feature information in CNN algorithm, the behavior pattern of CNN algorithm based on TVM is deeply analyzed. Firstly, extract the branch information from the computational graph by post-order traversal method, obtain the start, end and internal information of the branch, and construct the feature computational graph by the branch information. Secondly, with the help of the TVM, the features of access and computation of convolution are extracted. Finally, divide the computational graph manually according to the obtained branch information. Experimental results show that compared with the traditional TVM method, the method based on branch characteristics achieves an average speed improvement of 18%. Compared with traditional methods, the convolution optimization method based on convolution features achieves an average speed increase of 20%, and feature information is mined effectively.

Aiming at the problem of long access time caused by discontinuous data address of Memory Efficient Convolution (MEC) algorithm on traditional devices, an optimization method applying to MEC algorithm access behavior is proposed. The method is divided into

two parts: the intermediate matrix transformation and the matrix operation. First, the intermediate matrix transformation is optimized by modifying the data reading order to make the data reading mode conform to the access behavior of the algorithm. Next, for the matrix operation part, the convolution kernel matrix is modified by using the memory data layout more suitable for matrix operation, and the calculation function encapsulated by TVM platform is used to redesign the calculation method of the middle matrix and the convolution kernel matrix. Finally, the platform's own parallel library is used to speed up the computing process. Experimental results show that compared with the MEC, the average speed is improved by 50% on a single convolutional layer and more than 57% on a multi-layer neural network.

In view of the problems that the method of computing graph partition in TVM relies on expert experience and the partition strategy is single, a subgraph partition method based on branch features is proposed. At first, traverse the computational graph forward based on the characteristic information of the branches of the computational graph, search for the start and end nodes of the branches, slice and store them in an array. Then, each node in the array is extracted to construct the subgraph. The dependency configurations of input and output between each subgraph are counted and stored in an array. Finally, using dependency information between subgraphs is to configure input and output dependencies for subgraphs, and select and configure parameters and device information. Experimental results show that when 48 and 96 CPU cores are used, the speed of CNN algorithm is improved by 20% and 15% compared with traditional TVM operation mechanism, which effectively achieves the division of computational graph.

To solve the problem that single subgraph cannot be parallel in TVM, a branch parallel method is designed and implemented. Firstly, a directed acyclic graph and a back-order dominated tree are designed to record the key value, order and dependency of nodes. Secondly, the information is used to search for branches, and package the branch nodes into functions and label as parallel nodes. Thirdly, after the parallel graph marking is completed, the parallel runtime is used to process the computational graph, which involves inter-parallel thread pool design, data interaction and operation. Experimental results show that, compared with the traditional serial method of TVM, the branch parallel method can improve the inference speed by 10% and 20% on CPU and GPU, and the algorithm can improve the inference speed by 5% on average compared with the Greedy algorithm, which can make

efficient use of hardware device resources.

Keywords: Tensor virtual machine; Memory efficient Convolution; Convolutional neural network; Behavior analysis; Computer graph partition; Parallelization

Thesis : Application Research

目 录

1 绪论.....	1
1.1 课题来源、研究背景及意义	1
1.1.1 课题来源.....	1
1.1.2 研究背景及意义.....	1
1.2 国内外研究现状与发展趋势	4
1.2.1 面向 TVM 的计算图优化技术研究现状与发展趋势	4
1.2.2 CNN 访存优化的研究现状与发展趋势	6
1.2.3 计算图划分方法的研究现状与发展趋势	8
1.2.4 推理计算图并行调度的研究现状与发展趋势	10
1.3 论文的主要工作.....	13
1.4 论文的组织结构.....	14
2 CNN 算法的程序行为模式分析与特征提取.....	16
2.1 程序模式分析与特征提取方案	16
2.1.1 计算图分支特征提取设计.....	17
2.1.2 卷积特征提取设计.....	18
2.2 基于 TVM 的 CNN 计算图分支特征提取	19
2.2.1 TVM 中计算图结构信息分析与转换.....	19
2.2.2 计算图分支信息提取与表示.....	20
2.2.3 CNN 算法特征计算图构建	24
2.3 CNN 算法卷积特征提取	25
2.3.1 计算特征的提取与表示.....	27
2.3.2 访存特征的提取与表示.....	28
2.4 CNN 算法特征提取结果分析及有效性验证	29
2.4.1 计算图分支特征分析及有效性验证.....	29
2.4.2 CNN 卷积算法特征分析及有效性验证	32
2.5 本章小结.....	34
3 基于访存模式的 MEC 卷积算法优化方法研究.....	35
3.1 MEC 卷积算法访存模式优化研究.....	35
3.1.1 中间矩阵转换访存特征优化研究.....	36
3.1.2 矩阵相乘计算特征优化研究.....	38
3.2 MEC 卷积算法中间矩阵转换访存优化设计与实现.....	39
3.2.1 中间矩阵访存特征优化方案设计.....	39
3.2.2 单通道下矩阵转换算法实现.....	41
3.2.3 多通道下中间矩阵转换算法实现.....	42

3.3 MEC 卷积算法的矩阵运算优化设计与实现.....	44
3.3.1 矩阵计算特征优化方案设计.....	44
3.3.2 单通道下矩阵运算方法实现.....	45
3.3.3 多通道下矩阵运算方法实现.....	47
3.4 MEC 优化算法实验结果与分析.....	48
3.4.1 实验环境与实验方案.....	48
3.4.2 单层卷积实验结果与分析.....	50
3.4.3 多层卷积网络实验结果与分析.....	54
3.5 本章小结.....	55
4 基于分支特征的推理计算图划分方法研究	57
4.1 推理计算图划分方案设计	57
4.1.1 计算图分支自动划分方案.....	57
4.1.2 子图信息自动配置方案.....	59
4.2 面向推理计算图的划分方法设计	60
4.2.1 计算图分支自动划分方法设计.....	60
4.2.2 子图构建方法设计	64
4.3 子图划分后信息配置方法设计	66
4.3.1 子图间连接策略设计.....	66
4.3.2 子图设备及参数分配策略设计.....	67
4.4 推理计算图划分方法的实验结果与分析	68
4.4.1 实验环境与实验方案.....	68
4.4.2 推理计算图划分结果与性能比较分析	69
4.5 本章小结.....	73
5 推理计算图划分的并行调度设计实现	74
5.1 计算图并行化设计方案.....	74
5.1.1 并行节点标记方案设计.....	74
5.1.2 运行时并行调度方案设计.....	76
5.2 计算图并行节点标记方法研究	78
5.2.1 针对并行节点的后序支配树构建.....	78
5.2.2 带有并行节点标记的计算图生成.....	79
5.3 运行时并行调度算法设计	80
5.3.1 并行线程池设计.....	80
5.3.2 并行节点数据交互设计.....	82
5.3.3 节点并行化实现.....	83
5.4 并行调度实验结果与性能分析	84
5.4.1 实验环境与实验方案.....	84
5.4.2 计算图并行调度性能比较及分析.....	84
5.5 本章小结.....	88

6 总结与展望.....	89
6.1 总结.....	89
6.2 展望.....	90
参考文献.....	91
附录二 流水线算法和 Greedy 算法流程	97

1 绪论

1.1 课题来源、研究背景及意义

1.1.1 课题来源

本研究课题来源于科技创新 2030-新一代人工智能重大项目“数据中心高性能机器学习芯片及开发工具链”（项目编号：2020AAA0104603）。为适应多种不同新型神经网络专用加速器后端或异构资源底层架构，克服因负载和通信开销等因素引起的性能瓶颈，开展面向多架构神经网络计算图的多约束组合优化研究。通过构建异构资源性能评价模型、计算图优化的研究，充分挖掘和利用异构硬件计算潜力，设计高性能方案，实现高效神经网络推理流程。项目最终验收指标为面向多架构的情况下，基于原语图级别的优化，性能提升 10%以上。

课题针对项目中推理计算图优化方法开展研究，负责承担单一架构的计算图任务划分及优化策略的研究工作，解决计算图在运行时无法充分利用设备资源的问题。主要使用 TVM 对计算图粗粒度（图级别）和细粒度（算子级别）两方面进行加速优化，实现对算子的优化以及计算图的有效划分调度。以典型的 CNN 算法为例，提取有效的算法特征，在 TVM 中设计有效的优化策略，发挥 TVM 的优势，提升 CNN 算法的处理性能。首先，本课题通过挖掘 CNN 算法中分支的信息及卷积的计算依赖和访存依赖信息，构建特征计算图。其次，使用基于访存模式的优化方法，进行卷积算法的访存优化，实现 TVM 下卷积算法的优化。然后，通过分支划分方法，进行分支划分，以降低多次推理的总时间开销。最后，使用分支并行方法，降低 TVM 下分支网络的运行时间开销，提升模型运行速度。

1.1.2 研究背景及意义

随着摩尔定律所导致的计算性能的指数性提高，通用计算设备并行加速技术的发展，以及互联网与物联网的蓬勃发展所导致的大数据的出现，基于统计机器学习的人工智能技术得到了迅速发展。深度学习（Deep Learning, DL）是一类机器学习算法，深度学习模型现在可以识别图像，处理自然语言并在具有挑战性的策略游戏中击败人类。从云服务器到自动驾驶汽车和嵌入式设备，在广泛的设备上部署智能应用的需求越来越大^{[1][2]}。由于硬件特征的多样性，将深度学习工作负载映射到这些设备是复杂的，这些硬件特征包括嵌入式 CPU、GPU、FPGA 和 TPU^[3]。面向这些硬件目标各有不同的内存

组织和计算功能单元，为了适应硬件的多样性，将计算有效地映射到 DL 硬件是很重要的，陈天奇等人设计出了深度学习编译器——TVM^[4]。TVM 将深度学习框架（如 PyTorch 和 TensorFlow）中描述的模型定义作为输入，将针对各种硬件特性生成的高效代码作为输出，并且对模型规范、模型定义和特定代码实现之间的转换和硬件体系结构进行了高度优化^[5]。此外，现有的深度学习编译器还借鉴了来自通用编译器（如 LLVM^[6]）的成熟工具链，这些工具链在不同的硬件架构之间提供了更好的可移植性。因此，在 TVM 上进行各种优化成为工业界和学术界的研究热点。

（1）CNN 算法呈现出访存密集型和计算密集型特征

近年来，深度学习的发展对各个科学领域产生了深远的影响。它不仅在自然语言处理（natural language processing, NLP）^[7]、计算机视觉（computer vision, CV）^[8]等人工智能领域表现出了非凡的价值，而且在电子商务^[9]、智慧城市^[10]、药物发现^[11]等更广泛的应用领域也取得了巨大的成功。随着深度残差网络 ResNet^[12]、GoogLeNet^[13]和 Inception 网络^[14]等通用卷积神经网络模型的在各个领域的应用，这些应用越来越呈现出计算密集性和访存密集性等计算特性^[15]。其模型算法中包含大量密集性矩阵运算，包括矩阵乘法，向量内积等^[16]。因而 CNN 算法为代表的越来越呈现出计算密集性、访存密集性特征。

（2）TVM 成为优化 CNN 中计算和访存密集型算子的有效工具

深度学习编译器作为针对神经网络优化和部署的编译器，在神经网络优化方面有很多贡献。深度学习编译器将神经网络结构抽象化成计算图，针对计算图作出了一些关于图领域的优化和编译领域的专用优化。在优化前，开发人员通常会编写分析代码，交由编译器对神经网络算法进行分析，得出神经网络的某些特征，如访存特征、计算特征等信息。同时对这些信息进行总结，开发人员通过这些信息，对神经网络进行优化。作为深度学习编译器，TVM 中包含了针对计算图的分析工具，能够提取神经网络中层内和层间的依赖关系，得到需要的信息，这些信息具有针对性，可以指导下一步的研究。

针对 CNN 等计算和访存密集性应用，目前的 DL 框架，如 TensorFlow^[17]、MXNet、Caffe^[18]和 PyTorch，依赖于计算图这种中间表示形式来实现编译优化，如自动分区和动态内存管理^[19]等技术。这些框架大多专注于某些服务器类的 GPU 设备，并将特定目标的优化交给高度工程化和特定操作优化的算子库。操作符级别的算子库需要大量手动调优，过于专门化和不透明，很难跨硬件设备移植。TVM 使用通用的图形级和操作符级的算子优化，解决了深度学习特有的优化难题，如高级操作符融合^[20]、数据布局转换和卷积并行化，通过这些方法可以很好的解决 CNN 中计算和访存密集型的算子优化问题。此外，TVM 还通过使用一种新颖的、基于深度学习的开销模型来快速搜寻代码优化方

案，从而实现后端代码的自动优化和生成。因此，TVM 成为分析及优化计算和访存密集型算子的有效工具。

（3）通用平台下的计算图划分及并行调度为提升资源利用率提供有效途径

在基于符号编程的深度学习框架中，包括 Theano^[21]、Caffe 和 TensorFlow，都采用了计算图来描述神经网络结构。计算图（Computational Graph）类似通用编译器中的数据流图，采用有向图来定义一个神经网络的计算流程。计算图可以根据时间先后顺序分为三个阶段：定义阶段、构造阶段和执行阶段。定义阶段是指定义各个层和操作符的名字，所需参数，输入输出类型和形状等。构造阶段是根据定义的脚本实例化对象，并分配计算内存。最后一步为执行阶段，按照计算图定义的算子运算顺序开始执行计算^{[22],[23]}。计算图可以作为编译器的中间表示（Intermediate Representation, IR）来分离深度学习模型的前端优化和后端硬件部署，这种分离机制大大降低了神经网络部署的难度。

随着神经网络在不同硬件上配置的需求越来越多，传统的计算架构平台已经无法满足，近几年跨设备计算成为了研究热点。TVM 和 Gluon 都有其各自的跨设备计算图划分方法，TVM 中采用拓扑排序算法对计算图进行节点的遍历重写，Gluon 采用广度优先算法对节点进行遍历重写。两者划分计算图的目的相同，都是基于单任务情况下的推理计算图划分，图划分只考虑设备和神经网络节点的适配性，将最适合该设备的算子划分到设备对应的子图中。当我们需要多任务并行处理时，两者划分的子图无法满足多任务并行的情况。因此，需要提出一种新的计算图划分方法，在 TVM 原有计算图划分的基础上，进一步研究针对多任务并行处理的计算图划分方案，为下一步多任务调度方案提供基础。

在进行多任务推理时，只能在第一个任务处理完成后进行第二张图片的推理，即串行执行任务，这样会使设备出现计算资源闲置的问题，减慢了整体的推理速度。针对多任务推理时设备计算资源闲置的问题，需要提出一种多任务并行调度方案。该方案对多任务进行合理调度划分，使计算资源得到较好的利用，从而加快多任务推理的整体速度^{[24],[25]}。

综上所述，TVM 作为人工智能领域中主流的神经网络编译器之一，能将神经网络算法统一转换为计算图，并且在编译角度对图进行优化，部署到各类神经网络平台，同时对所部署的平台进行硬件原语级别的优化。随着各种神经网络算法的出现，算法中包含的大量计算特征和访存特征，TVM 内部提供了相应的特征提取的接口，同时开放了源代码。可以利用这两种方式对计算密集型和访存密集型特征进行提取。针对计算和访存密集性的算法，主要体现在卷积中的矩阵运算，TVM 使用内置的运算函数，可以基

于访存特征对矩阵运算进行自定义优化。同时含有编译优化领域常用的块化和并行等方法，两者结合对算法进行优化加速，成为优化卷积运算的主要工具之一。对于多分支网络，TVM 提供 BYOC 和流水线机制两种方法来进行多分支网络的划分和加速，并提供相应接口对机制进行修改，可以修改内部算法对网络进行并行加速。

1.2 国内外研究现状与发展趋势

深度学习编译技术指专门针对深度学习的编译优化技术，是发掘软硬件等复杂条件约束下深度学习编译器执行性能的有效手段。深度学习编译器与传统编译器有一定的差别，传统编译器注重优化寄存器使用和指令集匹配，其优化往往偏于局部。而深度学习编译器的优化涉及到全局的优化，包括内存预分配，算子融合等全局优化手段。此外，深度学习编译器又借鉴传统编译器的优化方法，使得神经网络能够得到局部和整体两方面的优化，同时具备一定的可移植性，使神经网络可以便捷的部署到多个平台。但是，目前深度学习编译器仍然存在诸多不足，国内外学术界和工业界在相关领域开展了广泛研究。

1.2.1 面向 TVM 的计算图优化技术研究现状与发展趋势

随着深度学习模型越来越多地不同硬件上部署，促进了各个深度学习社区对深度学习编译器的研究和开发。工业界和学术界都提出了一些深度学习编译器，如 TVM、Tensorflow XLA、nGraph。神经网络编译器类似于传统的编译器，以文献[4]提出的 TVM 架构为例，如图 1.1 所示，其通用设计体系结构主要包括两部分：编译器前端和编译器后端。中间表示（IR）分布在前端和后端，IR 是程序的抽象，用于程序优化。前端将输入模型转换为计算图表示，称为 Relay IR。

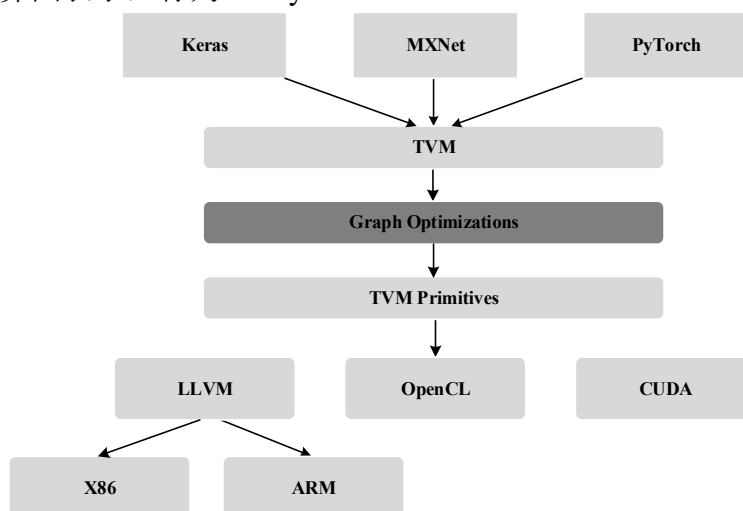


图1.1 TVM 整体架构

在前端优化完成后，将优化后的计算图传递给后端。后端将高级 IR 转换为低级 IR，并对低级 IR 执行针对硬件的特定优化。一方面，它可以直接将低级 IR 转换为第三方编译器低级语言，如 LLVM IR，利用现有的优化库进行通用优化和代码生成。另一方面，它可以利用神经网络模型和硬件特征的先验知识，通过定制的编译过程优化低级 IR 并生成高效地硬件代码。通常，应用于特定硬件的优化包括硬件内部映射、内存分配、内存延迟隐藏、并行化以及循环优化等手段，并生成针对不同硬件目标的代码^[26]。

XLA 和 TVM 两者之间也存在着差异，如图 1.2 所示，在前端图优化方面，TVM 专注于计算密集型算子（如卷积操作），把一部分图优化工作转移到 TVM 开发者身上，有比较好的灵活性。文献[5]中提到，XLA 则专注于非计算密集型算子，XLA 会根据用户提供的计算图直接生成可执行代码，强调自动化，开发者无法针对计算图做一些自定义优化。在后端针对硬件优化方面，TVM 使用机器学习结合专家知识的方法，在目标硬件上不断学习并更新相关运算的参数，找到最佳参数配置，实现高效运算^{[27],[28]}。

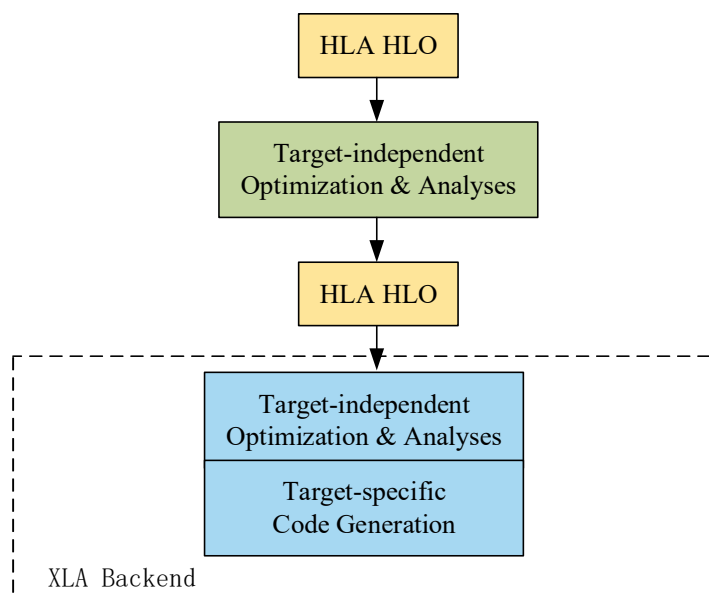


图1.2 XLA 整体架构

如图 1.3 所示，nGraph 由 Intel 公司提出，采取三层架构前端通过 bridge 对各类神经网络框架进行转换。输入到 nGraph 中间件对神经网络进行优化，优化完成后交由转换器生成各类硬件原语，使用 cuDNN、MKL-DNN 等后端专用的加速库对神经网络进行特定的硬件优化。中间件使用 nGraph IR 对神经网络进行表示和优化，nGraph IR 表示的是包含无操作状态节点的有向无权图，称为静态图。节点在神经网络中表示为多维数组，每个节点定义了输入的 shape 和属性，输出的 shape 和节点的类型。在 TVM 中对节点的轴阶数和元素布局之间存在固定关系，但是，中间件没有将二者绑定，使得用户可以直接对节点进行操作。转换成 IR 后，对 IR 进行针对后端的专有优化，使用模式匹配、

内存管理和张量布局转换等与后端优化库相结合的方法对 IR 进行优化^{[29],[30]}。

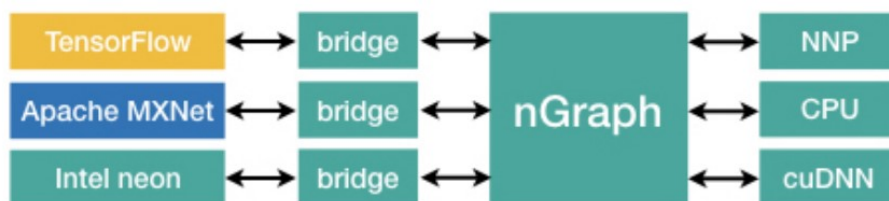


图1.3 nGraph 整体架构图

神经网络编译器对比如表 1.1 所示，TVM 使用 Relay 和 Halide 两种 IR 表示方式，可以对神经网络进行高级别的图优化和低级别的硬件原语优化，其他两种采用单一 IR 表示方式，无法提供多层面的优化，并且 TVM 有针对特定后端的优化库。此外，TVM 还采用机器学习的方式对神经网络进行自动剪枝，通过 BYOC 接口使得开发人员可以对多个后端进行神经网络的部署。

表1.1 各类神经网络编译器对比

	TVM	XLA	nGraph
量化	√	√	√
IR	Relay/Halide	HLO/HLO	nGraph/None
动态图	√	√	√
自动剪枝	√	√	×
支持后端	CPU/GPU/ARM/FPGA	CPU/GPU/TPU	CPU/Intel GPU/NNP GPU

1.2.2 CNN 访存优化的研究现状与发展趋势

卷积层作为卷积神经网络模型中计算和访存最密集的网络层之一，其性能影响着整个神经网络的性能。随着深度学习网络在各种资源受限型设备中部署，内存使用率低的卷积算法逐渐走入人们的视野，当前新兴的卷积方式有空间组合优化算法(Spatial pack)、间接卷积算法和 MEC (Memory Efficiency Convolution) 算法。

文献[31]提出了空间组合算法，它是一种基于分治法的方法。如图 1.4 所示，算法利用空间特性将卷积计算划分为若干份分别处理，在空间上将输入划分若干份。划分后，一个大的卷积计算被拆分成若干个小的卷积运算，除了将划分的小块中原始数据拷贝外，还需要将相邻小块的边界数据拷贝。虽然，在划分过程中计算总量不变，但是，计算小矩阵时访存局部性更好，可以加速数据的读写速度，获得性能提升。由于实际应用中卷积可以拆分成很多份，随着拆分出的份数越多，其局部性也越好，负面作用是消耗

的额外内存也越多。

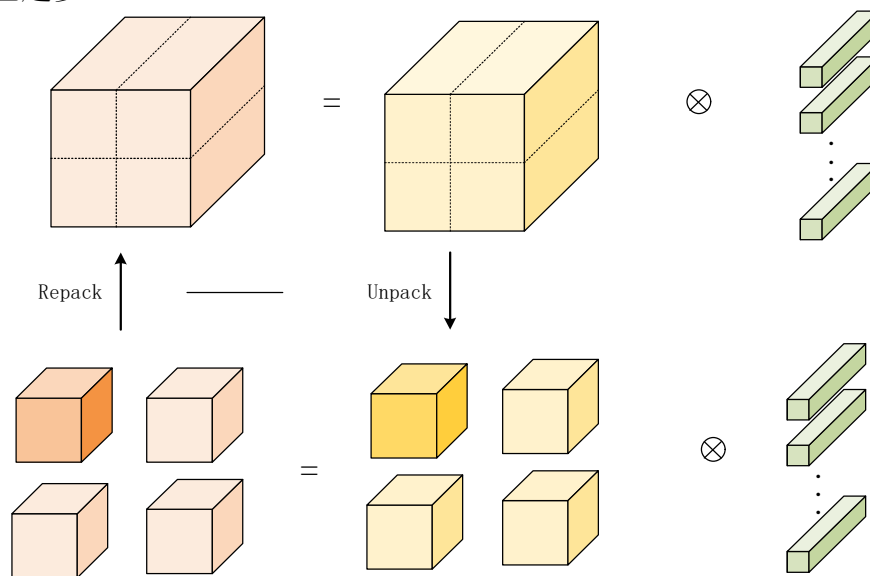


图1.4 空间卷积优化过程

间接卷积优化算法是 Marat Dukhan 在文献[32]中提出的卷积算法，是一种较快的卷积优化算法。图 1.5 所示为间接卷积算法和 Im2col 卷积算法的比较，传统的 Im2col 算法将输入和卷积核计算转换成矩阵相乘，转换过程中需要对输入进行额外的数据拷贝，占用大量额外内存。为解决这个问题，间接卷积算法会预分配一个间接缓冲区，将输入特征图中每个值对应的内存地址保存在间接缓冲区内。在计算时，随着计算位置的移动，选用不同的间接缓冲区即可得到对应输入地址，无需再根据数组坐标计算输入位置。这种方法虽然极大的节省了内存占用，但是，仅限于某些特定数据布局，不具备普适性[33]。

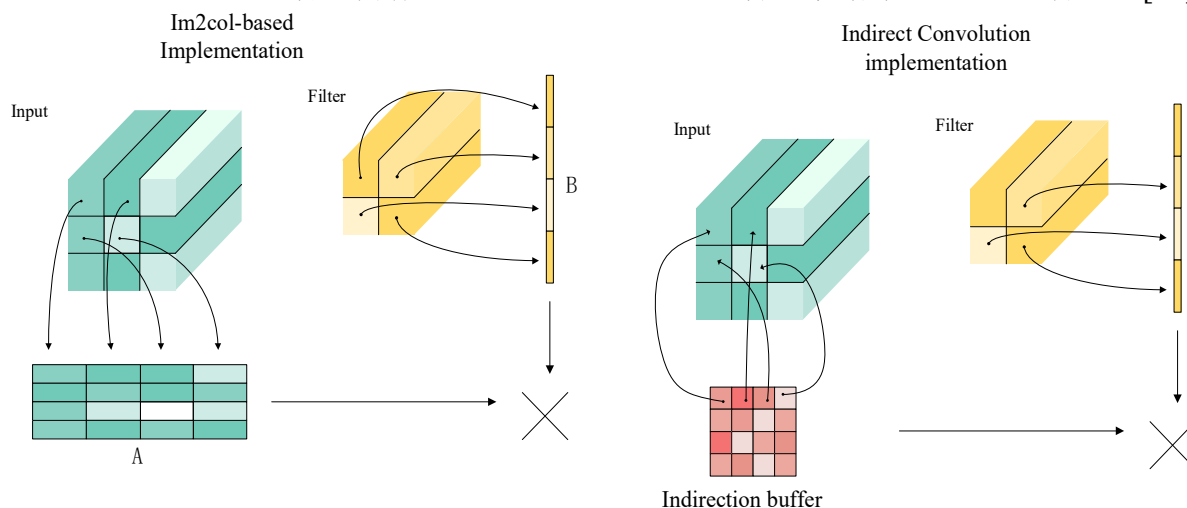


图1.5 间接卷积过程示意

为了平衡计算速度和读取速度，学者们提出了新的卷积算法。文献[26]基于 Im2col[34]算法提出了卷积算法 MEC。该算法将输入矩阵转换成的多个列向量按行高这

一维度进行展开后，再重组为中间矩阵，然后将中间矩阵划分成小矩阵同卷积核进行矩阵乘法运算。如表 1.2 所示，同 Im2col 相比，MEC 算法大幅降低了内存使用。但是，MEC 算法因为段矩阵地址不对齐，访问的数据地址不连续，导致其矩阵存储不符合神经网络使用的以行主序进行存储的编程语言存储方式，致使 MEC 算法存在 cache 命中率低，内存访问延时长的问题。因此，以 MEC 算法为例，结合算法分析，对算法进行优化。

表1.2 各种卷积算法比较

	Im2col	Indirect	Spatial	MEC
内存占用	大	无	较小	中等
读取速度	较低	较低	较高	较高
计算速度	较低	较高	较高	较低

1.2.3 计算图划分方法的研究现状与发展趋势

随着众多编译后端（如 LLVM、CUDA）的出现和异构平台的普及，推理计算图划分（也称子图划分）成为深度学习编译领域中一个重要的发展方向。支持子图划分的神经网络编译器可以将计算图划分为多个子图，子图可以用不同的方式进行处理。为编译器提供了更多的研究机会。首先，它为集成图库进行优化提供了可能。以 nGraph 和 DNNL 为例，DNNL 是一个优化库，它的图优化利用了大量高度优化的内核。DNNL 与 nGraph 的集成使 DNNL 能够加速 nGraph 生成的子图的执行。其次，它使异构和并行执行成为可能。一旦将计算图划分为子图，不同子图的执行就可以同时分配给异构硬件目标。以边缘设备为例，其计算单元可能包括 ARM CPU、Mail GPU 和 DSP。可以利用编译器生成子图充分使用所有计算单元，显著提高神经网络的推理速度。在众多深度学习编译器中，Gluon 和 TVM 为计算图划分提供了良好的支持和维护。

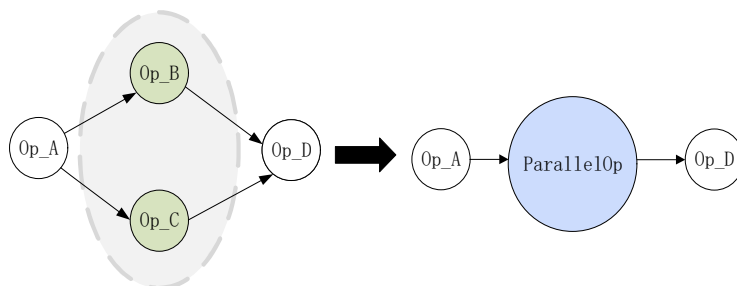


图1.6 Gluon 算子融合步骤

文献[35]提出了 Gluon，其子图划分方法主要分为两个步骤，首先，Gluon 会提供一个节点选择器，它使用广度优先搜索算法遍历图中的每个节点。将符合默认规则的图节

点纳入子图，在这过程中用户可以自定义节点选取规则，当选择器搜索完成后，会将纳入子图的节点卸载并接入到子图中。其次，在所有子图划分完成后，把分配到同一后端或设备的几组子图重新合成一个新的计算图，完成后利用 Gluon 内置的优化方法对新的计算图进行优化^[35]，具体过程如图 1.6 所示。

TVM 作为深度学习编译器中模块化设计最好的编译器之一，zhiics 等人在文献[37]中提出了关于计算图划分的相关设计，现已成为 TVM 中最具竞争力的一个功能，如图 1.7 所示。与 Gluon 中子图划分的思想不同，TVM 采用划分更加高效的分区算法。TVM 采用拓扑排序算法对计算图的每个节点进行遍历，同时对计算图的节点进行着色，以区分不同子图。待计算图着色完成后，对计算图进行划分，根据不同后端生成高度优化的代码。

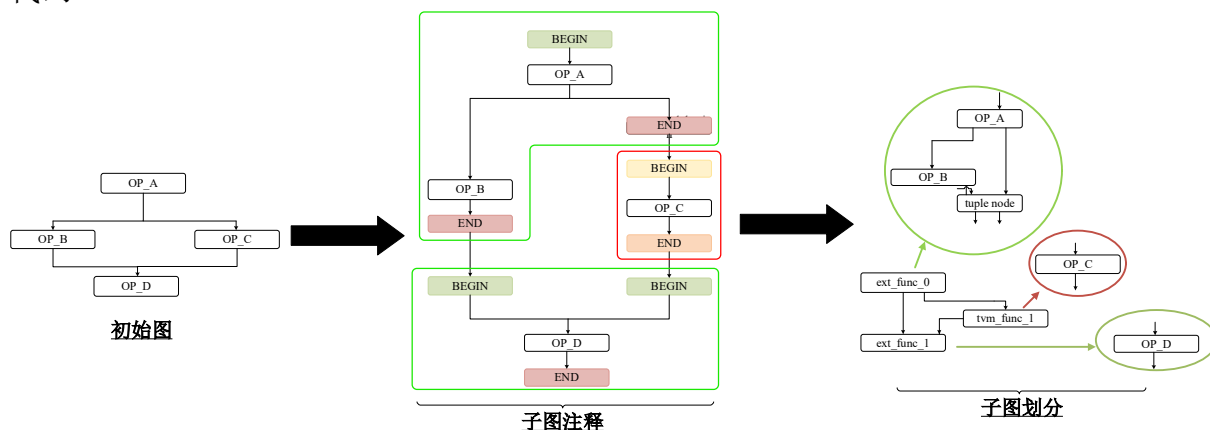


图1.7 TVM 子图注释划分

文献[36]提出的计算图划分基于线程级别，是为了防止数据竞争导致的 CPU 失速。nGraph 根据目标节点将整个计算图划分为不同子图，如图 1.8 所示，图中被划分为 0 和 1 两个分区。分区 0 分配顶点为{0,1,2}的六条边（如 V0-V1,V0-V2），分区 1 分配顶点为{3,4,5}的六条边（如 V0-V4,V1-V5）。每个线程依次处理一个分区中的节点，仅根据目标节点在同一分区的边更新相邻顶点的属性和数据，该方案避免了多个线程同时更新一个节点的属性和数据，避免了由于数据争用导致的潜在数据更新。

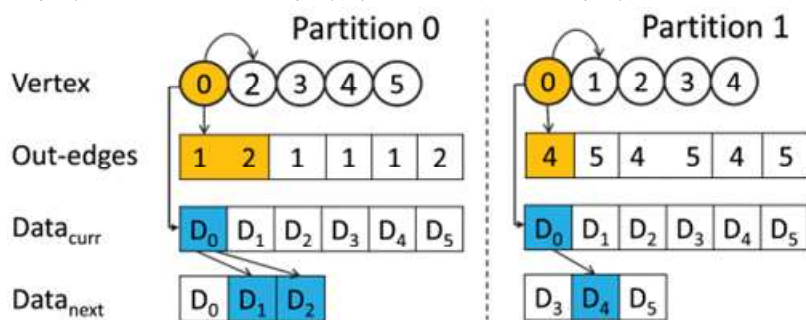


图1.8 针对目标设备的 nGraph 图分区

从表 1.3 中可以看出，TVM 和 Gluon 两种计算图划分方法都是基于专家经验对计算

图进行手动划分，尚未对计算图进行自动划分，增加了分支划分的难度。nGraph 采用自动划分方法，但是应用场景为多核 CPU，不适用于跨设备划分。TVM 由于支持多设备、异构以及多设备下的并行，具备更多的扩展性。因此，本文根据分支特征信息，基于 TVM 中流水线机制计算图进行分支划分。

表1.3 各种划分算法比较

编译器	应用场景	解决策略
Gluon	多设备或异构后端	设计节点选择器，对符合条件的计算图进行划分，划分后对同一后端的算子和子图进行融合
TVM	多设备、异构和并行	使用自定义分区算法和手动子图划分方法对计算图进行划分
nGraph	多核 CPU	设计自动划分策略，防止数据争用

1.2.4 推理计算图并行调度的研究现状与发展趋势

随着深度学习应用场景越发复杂，需要深度学习模型一次性推理多个任务的情况逐渐增多。在推理计算图划分方面，在文献[38]中，TVM 将重点放在单一计算图的划分上，即只有在当前推理任务完成后才能进行下个推理任务。此种方法忽略了多个计算图（即多个推理任务）在不同设备间的协同运行。

如图 1.9 所示，调度程序将计算图划分到 CPU-1+CPU-2 的多设备上，当 CPU-1 完成子图 A 的计算任务后，交由 CPU-2 计算后续子图，这会使硬件资源产生不必要的浪费，同时，减慢了整体的推理时间。此外，当进行多分支网络推理时，若子图 A 和子图 B 为两个包含共同输入且无其他依赖的分支，能充分发挥设备性能的方式为子图 A 与子图 B 并行，但是 TVM 中计算图执行方式为串行。因此，需要一种调度方法使推理任务在多个设备上同时进行并行推理。

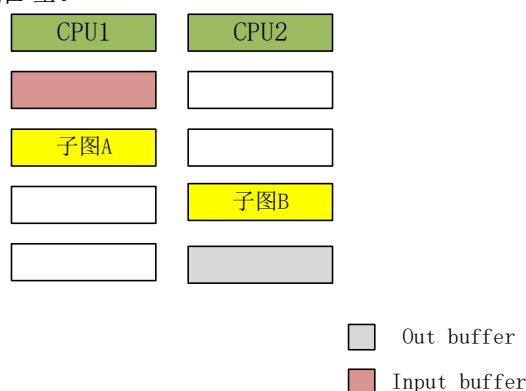


图1.9 不同设备间计算图任务划分

DART 调度结构由文献[39]提出,如图 1.10 所示,分为 RT 和 BE 两个任务,RT 任务分为两个阶段,分别为 Node1 和 Node3,Node2 未被使用,BE 任务的三个阶段为 Node1-3 顺序执行。DART 将每个任务的推理计算图划分为多个阶段,然后将这些阶段分配到不同的节点中,并由节点的工作组以流水线的方式执行。

在划分时对计算图进行自适应划分,这是因为计算图的各个部分对异构的计算资源的敏感程度不同。例如,在 CNN 模型中,一些层可以通过使用更多的 CPU 核数来获得较大的性能提升,而其它层没有明显提升。如果将这类 CNN 模型单一的分配到一类计算资源上,无法使网络模型的各个层都得到较好的加速。因此,文献[39]提出的算法可以灵活有效的将推理任务的各个层按照特征分配给不同的硬件资源,以获得更好的加速效果。

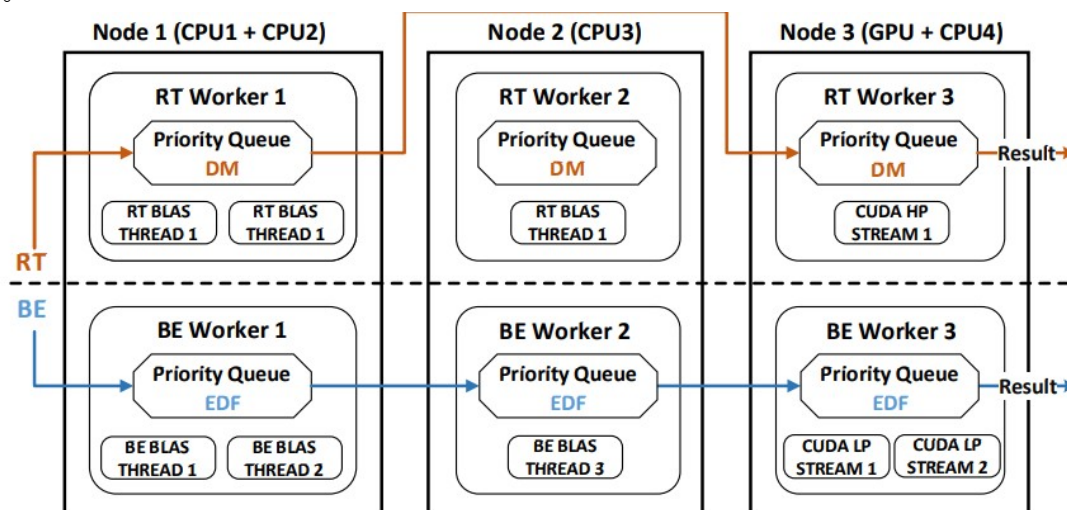


图1.10 DART 任务执行流程

为了加快神经网络的运行速度,许多学者和开发人员进行了神经网络并行的相关研究。文献[40]提出了关于神经网络节点运行的几种策略,如图 1.11 所示,主要有顺序调度和直接并行调度两种。原始计算图为如直接并行调度所示,采用含有三个分支结构的算子作为示例计算图。采用顺序调度执行时,编译器优先运行左侧分支,之后从左到右依次执行分支结构,最后完成分支的合并操作。

顺序调度的优点是无需考虑硬件资源的限制,对所有分支结构进行串行操作,缺点是无法充分利用资源。直接并行调度的思想是将所有可并行操作在第一时间进行并行,如图 1.11 所示,对于 Conv[a]、Conv[c]和 Conv[d]三个算子,在同一时刻运行,运行完成后,进行 Conv[b]的计算。直接并行调度在运行时需要考虑硬件设备资源的问题,例如,当有过多的算子在某个线程中或者设备上执行时,超过了线程或者设备的资源使用情况,会导致性能降低。

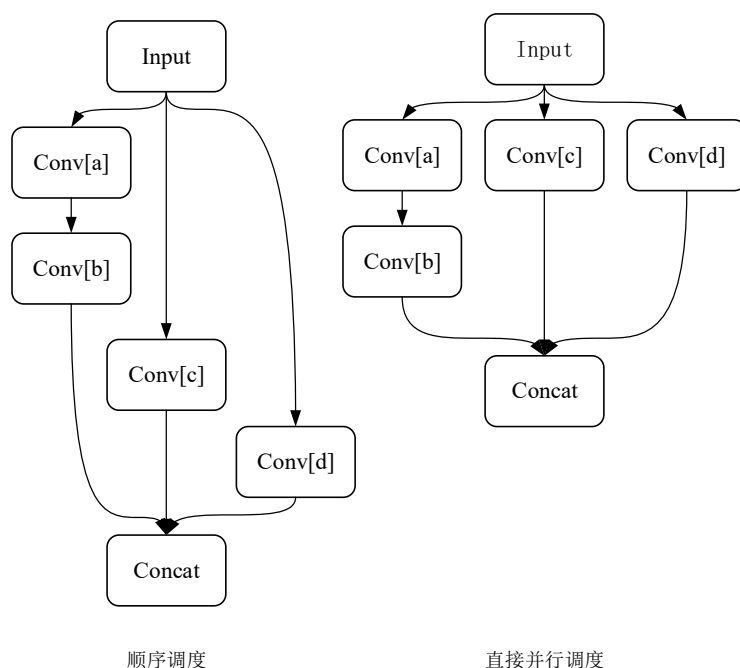


图1.11 算子调度策略

从表 1.4 中可以看出，并行调度方法中直接并行调度容易出现资源抢占问题，DART 方法中，无法对算子内部提供高度优化。流水线调度虽然可以充分利用 TVM 中现有的优化加速方法，对多个神经网络进行加速推理，但是划分需要依赖专家经验。因此，需要一种合适的并行调度方法，对多个分支进行合理的并行调度，以实现神经网络的性能提升。

表1.4 并行调度方法比较

调度名称	调度方式	优点	缺点
顺序调度	采用串行方式运行神经网络	可以充分使用算子融合等优化手段	没有进行图级别的并行加速
直接并行调度	对分支进行直接并行，不考虑资源问题	调度方式简单，不用考虑其他限制因素	容易出现直接抢占资源的情况
DART	采用串行+并行结合的异构并行方式	将节点按照特征分配到适合的设备上，做到较好加速，解决了资源抢占问题	没有做到算子内部的充分优化
流水线调度	基于 TVM，将计算图划分成各个子图在不同设备上运行	充分利用 TVM 中高度优化的算子库	子图划分需要依赖专家经验手动划分

1.3 论文的主要工作

本文的研究目标是实现基于 TVM 的卷积优化和计算图划分并行调度方法，首先，分析 CNN 算法的行为模式，提取计算图分支特征及卷积访存特征，用于挖掘神经网络算法的优化途径。其次，合理设计卷积算法中的访存方式，实现卷积算法的优化加速。然后，设计分支划分策略，实现 TVM 下的计算图自动划分。最后，研究推理计算图的并行化策略，实现 TVM 下 CNN 算法的并行加速。

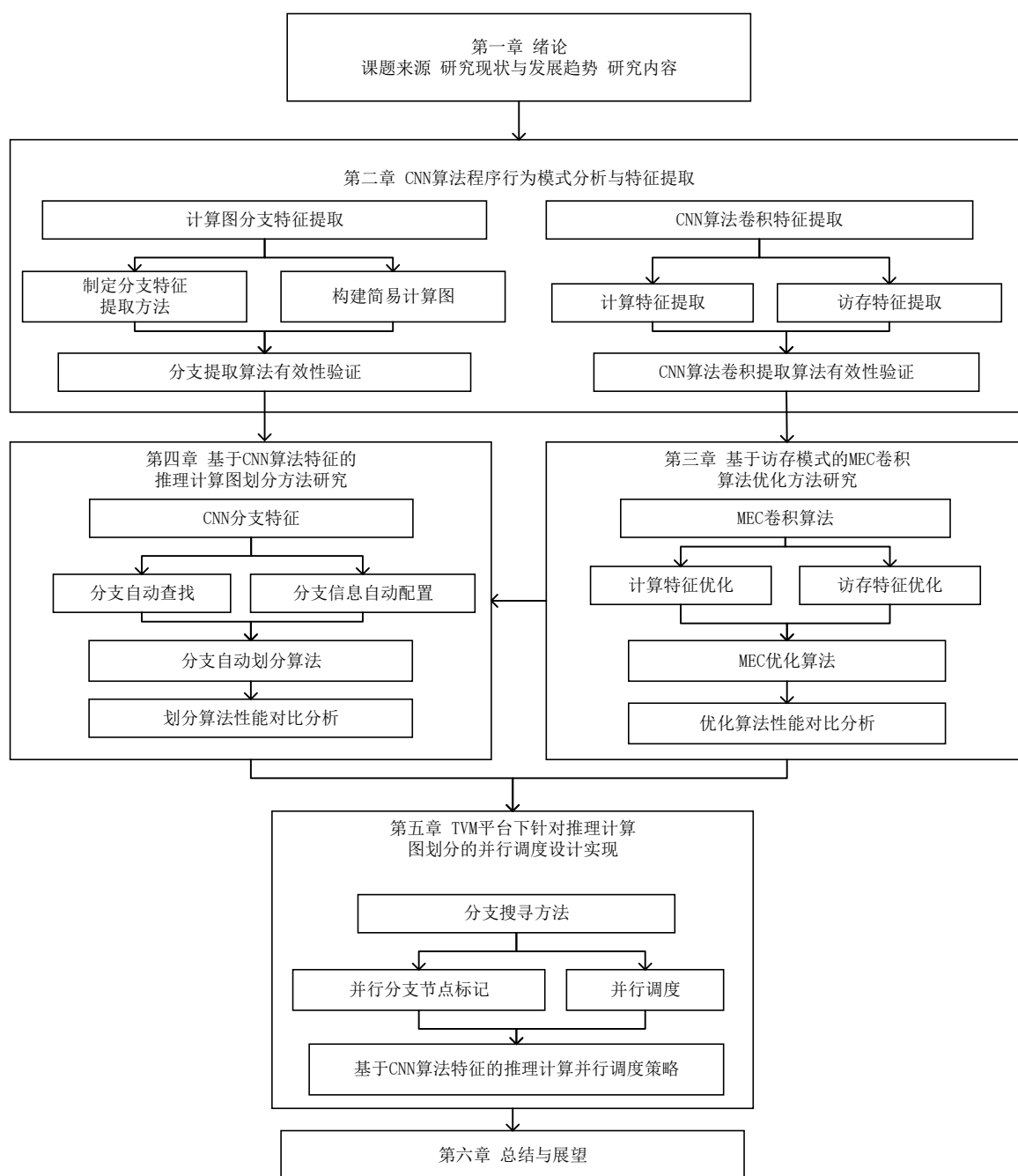


图1.12 论文研究内容及逻辑关系

论文主体由四部分构成，分别对应第 2、3、4、5 章，如图 1.12 所示，分别为 CNN 程序行为模式分析与特征提取、基于访存模式的 MEC 卷积算法优化方法研究、基于 CNN 算法特征的推理计算图划分方法研究和 TVM 平台下针对推理计算图划分的并行调度设计实现。本文的主要研究内容如下：

(1) 为深入挖掘神经网络算法的优化途径，本文研究 TVM 平台下 CNN 程序行为模式。首先，基于 TVM 提供的计算图和数据流图，研究不同层之间的数据依赖和控制依赖等计算依赖关系。其次，基于程序访存模式的时间特征和空间特征，研究不同层间的访存依赖关系，为推理计算图划分和访存优化提供支撑。

(2) 传统 CNN 算法的主体是卷积层，为了挖掘卷积层的性能，减少数据移动，从而最小化访存开销，提升执行性能。首先，根据访存模式中的空间特征信息，针对 MEC 卷积算法研究符合特定硬件访存模式的数据读取优化策略。其次，研究针对 MEC 卷积算法的数据布局优化策略。

(3) 为了发挥设备性能，拟开展针对推理计算图的划分方法。首先，在分区之前，研究在不影响子图划分的情形下，针对当前推理计算图进行特定优化策略，如算子融合，简化推理等操作。其次，以不同硬件设备的计算特征和访存特征为指导，研究将计算图进行划分为多个子图，对不同设备上的子图按照其硬件结构进行特定优化的策略。最后，针对不同的后端硬件，研究不同子图的配置策略。

(4) 当前已有的解决方案仅支持推理任务以串行化的方式运行在平台上，为了进一步提升性能，拟提出一个并行设计方案。首先，针对单个推理任务，根据当前的硬件环境，结合 CNN 的程序特征，研究子图中无数据依赖算子的并行标记策略。其次，为充分利用硬件结构，研究在单个推理任务分支并行策略。

1.4 论文的组织结构

本文针对 TVM 编译器中计算图优化尚未解决的问题，分别从行为模式分析、卷积优化、计算图分支划分及分支并行这四部分展开深入研究。本文组织结构安排如下：

第 1 章：主要介绍课题来源、背景及意义，分析总结 TVM 中访存优化、分支划分和计算图并行的研究现状和发展趋势。对现有研究成果分析总结，提出本文的研究内容。

第 2 章：详细讨论 CNN 算法的行为模式和特征提取，通过使用 TVM 提出 CNN 算法分支特征及卷积特征提取算法，并进行提取。利用分支信息构建简单计算图，在 TVM 下分析过程并进行验证，为第三章卷积优化和第四章分支划作支撑。

第 3 章：详细讨论 CNN 算法中卷积优化策略，针对 MEC 算法，使用第二章提出的卷积特征提取算法，提取卷积特征，基于特征提出一种基于访存优化的优化算法。该方

法根据卷积行为模式提取的访存和计算依赖信息，重新设计 MEC 算法卷积计算方式，实现对 MEC 的加速优化，并将优化方法进行实验验证与分析。

第 4 章：详细讨论 TVM 下的 CNN 算法划分策略，使用第二章获取的分支特征，提出一种基于分支信息的计算图划分方法。该方法根据分支特征提供的节点信息，使用自定义的计算图划分功能对计算图进行划分，实现 TVM 下 CNN 算法的分支自动划分，并将划分结果进行实验验证与分析，并将分支划分算法作为第五章分支并行的支撑。

第 5 章：针对推理计算图的并行化，基于第四章分支划分，设计并实现 TVM 下 CNN 算法的并行化方法。该方法通过识别标记可并行分支，设计分支并行算法，实现 CNN 算法的并行化加速，并对实验结果进行验证与分析。

第 6 章：对本文所提问题及其解决方案进行总结，指出本文方法尚存在的不足和后续工作的研究重点。

2 CNN 算法的程序行为模式分析与特征提取

在 TVM 中运行 CNN 算法时，存在大量结构信息和计算、访存信息，这些信息对于挖掘 CNN 算法的性能具有重要意义。因此，本章通过行为模式分析，设计分支特征和卷积特征提取算法，提取 CNN 算法中的计算图特征及计算、访存特征，描述 CNN 算法的计算和访存特性，用以指导计算图划分及卷积优化。

2.1 程序模式分析与特征提取方案

在 TVM 中，CNN 算法的表示方式为计算图，研究人员从粗粒度和细粒度两个方面对计算图进行优化。因此，本章主要分析 CNN 算法中的分支结构特征及卷积算法的访存和计算特征。对于算法的分支特征，需要提取算法的分支信息来构建特征计算图。由于 TVM 采用数据流图的形式定义计算图，可以表达计算图的执行顺序，及各个节点之间的依赖关系。此外，近年来众多学者将研究方向转为分支并行的性能提升上，取得较好的效果。通过提取 CNN 算法的结构信息构建特征计算图，用来概括 CNN 算法的分支特征。

对于卷积算法的计算特征和访存特征而言，需要借助 TVM 获取卷积算子的访存和计算信息。由于 TVM 下 CNN 算法的各个结构用算子来表示，可以根据 TVM 提供的接口编写算法获取卷积算子内部的数据执行顺序和依赖关系。并且，目前对于 TVM 下的卷积算法优化主要以设计调度方案为主，合理的调度方案可以取得较好的效果。通过算子特征和调度方案，以提取 CNN 卷积算子的访存模式和算法信息。具体研究内容及其逻辑关系如图 2.1 所示。从图中可以看出，CNN 算法行为模式分析的研究内容包括 CNN 算法的分支特征提取和 CNN 算法的卷积特征提取。研究内容逻辑关系为使用 CNN 分支特征提取方法提取分支特征并进行计算图构建，使用卷积特征提取方法提取卷积特征。

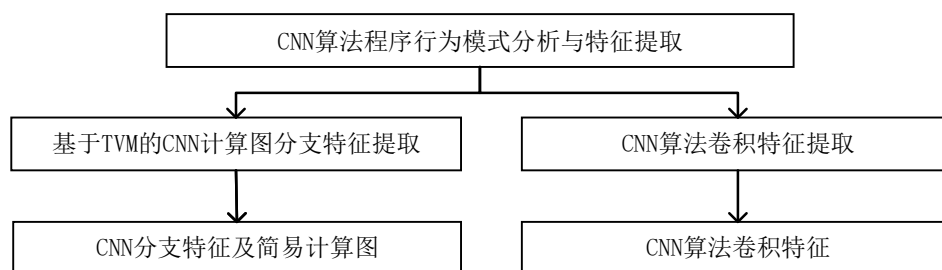


图2.1 程序模式分析研究内容及其逻辑关系

2.1.1 计算图分支特征提取设计

通过分析计算图的分支结构，提取能够描述计算图整体分支结构的特征信息，并根据特征信息构建特征计算图，为 CNN 算法在计算图分支划分和分支并行提供支撑。为更好的实现分支特征提取，本节制定了计算图分支特征提取原则、提取过程及详细的提取方案，提取原则如图 2.2 所示。CNN 计算图分支特征提取原则如下：1) 该特征能够有效表示分支信息，如分支开始节点、分支合并节点以及分支个数等。2) 该特征能够详细描述各个分支内部的主要节点及依赖关系。3) 该特征能够表述算法的执行顺序，如不同分支的执行次序。

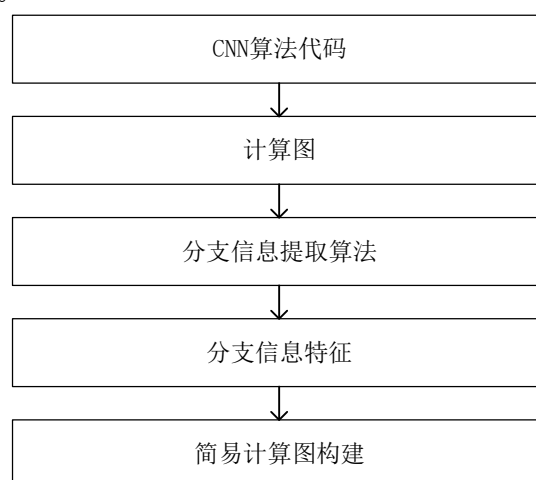


图2.2 分支提取原则

从图 2.2 中可知，首先，通过输入 CNN 算法得到对应计算图，对计算图做出处理后，交给编写的分支信息提取算法获取分支执行的顺序，节点信息。最后，将得到的信息进行处理，使用 TVM 平台下的计算图搭建功能，完成能够表示分支信息的特征计算图构建。分支信息提取流程如图 2.3 所示，包含三部分：1) 使用后序遍历方式查找分支汇合节点，获取分支信息。2) 借助分支信息提取各个分支的详细信息特征。3) 查找并记录分支开始节点，构建特征计算图。

通过后序遍历计算图识别分支并提取特征信息流程如所示。首先，通过设计后序遍历的识别算法，识别分支结束节点，获取节点名称、分支数目及各节点入口。然后，通过后序遍历对每个分支进行信息提取，获取各个分支的开始节点信息和结束节点信息。最后，识别分支开始节点，获取节点名称，继续重复上述步骤向上遍历计算图，直至遍历完成。通过获取的计算图分支信息，得到节点间的依赖关系和执行顺序。通过分析计算图的节点间依赖关系和节点类型，提取 CNN 算法特征，构建基于特征信息的特征计算图。该计算图能够概括的表述 CNN 算法的分支信息和数据流向，用于后续分支划分与并行的测试用例。

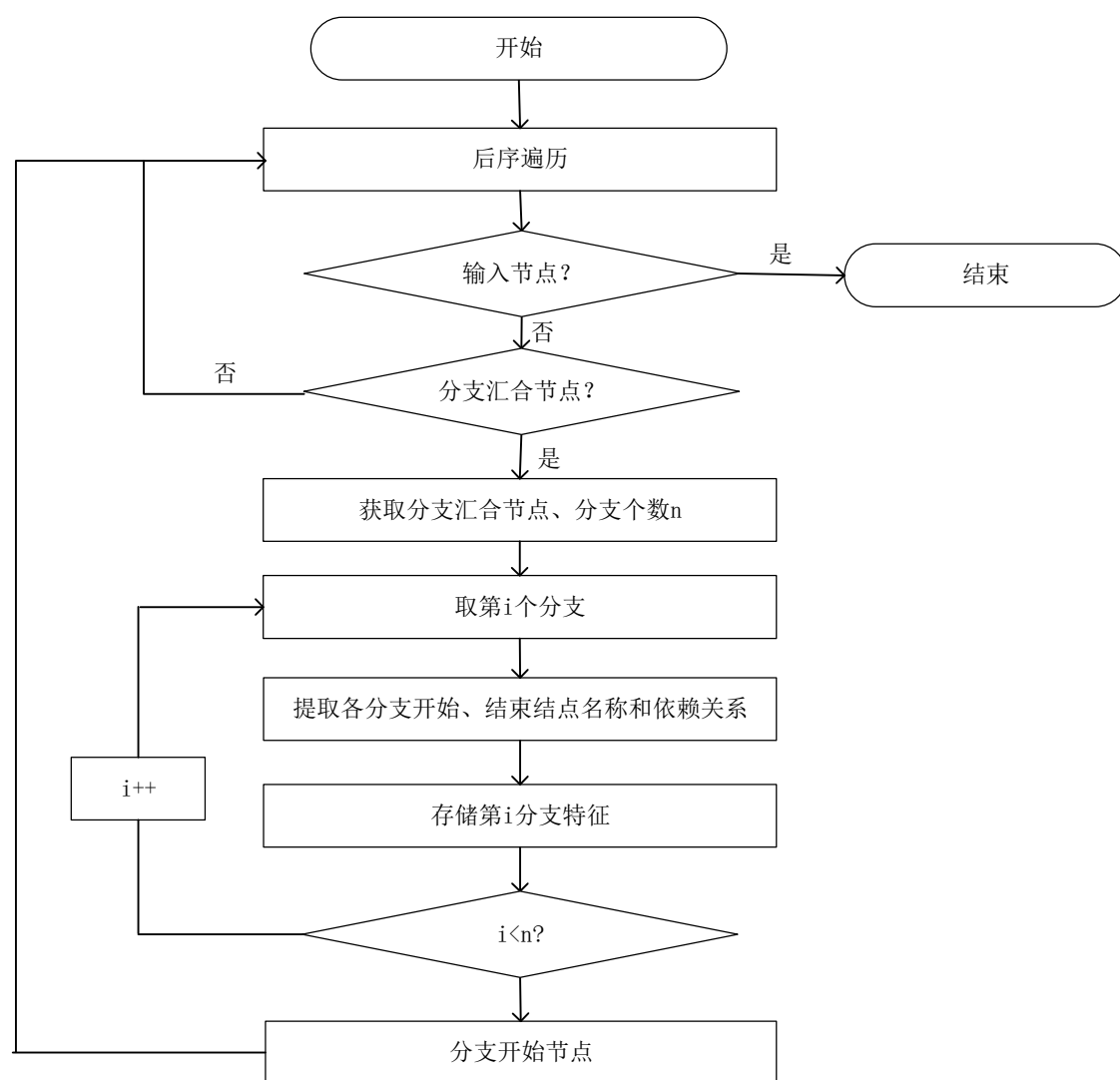


图2.3 分支信息提取流程

2.1.2 卷积特征提取设计

卷积层作为神经网络中最重要的算子，承担着提取输入矩阵特征的作用，通常需要大量的计算来完成图像的处理和提取^{[41],[42]}。通常步骤为：将输入图像转换为矩阵参数，利用内置的多套卷积核同输入矩阵进行矩阵乘法操作，从而提取出各类特征^[43]。如上所述，卷积操作多为矩阵运算，即输入矩阵同卷积核矩阵进行矩阵乘法操作，在代码层面，矩阵相乘转化为循环累加和累乘。

因此，卷积特征提取实质上是对循环进行特征提取，包括计算特征提取和访存特征提取两部分。借助 TVM 中更接近底层的 IR——TIR (Tensor IR)，来进行卷积特征的提取，卷积特征提取设计流程如图 2.4 所示。首先，提取循环表层数据，即循环初始量 i ，循环结束条件以及初始量 i 的计算方式。其次，提取循环的计算特征，包括依赖距离和

依赖数目。最后，提取循环的访存特征，如数据执行次序和数据访问次数。提取完成后，进行特征的输出表示。

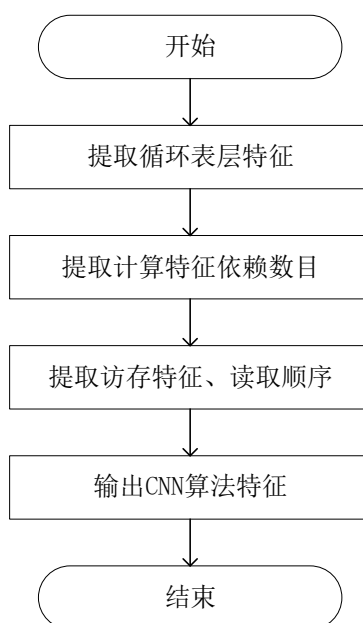


图2.4 卷积特征提取基本流程

2.2 基于 TVM 的 CNN 计算图分支特征提取

TVM 编译器能够将 CNN 算法转换为计算图，计算图包含 CNN 算法的所有特征信息，如数据流向，节点类型等。同时，由于将 CNN 算图用图来表示，可以利用遍历算法对计算图进行遍历，方便的提取各类特征信息，用于构建能够概括表示 CNN 结构的特征计算图。

2.2.1 TVM 中计算图结构信息分析与转换

在 TVM 编译器中，处理 CNN 算法以中间表示（Relay Intermediate Representation, Relay IR）为主，也称为计算图。如表 2.1 所示，列举了一系列常用操作对应的 Relay IR。

表2.1 TVM 中 Relay IR 结构表

类型	声明
函数	def@函数名 (参数){函数体}
变量	%变量名:[Tensor(维度, 数据类型)]
常量	%常量名:数值
算子	%序号=操作符 (常量/变量/算子)

从表中可以看出，每个类型都有标注信息，包括每种类型的声明以及使用，为分支特征提取提供有效支撑。例如，由于算子以操作符和参数组成，每个算子含有不同的操作符和参数，具有特异性，因而，能用来进行特异性算子的检索。

通过中间表示可以将 CNN 算法转换为图形化结构，从而使用图结构方面的思想对 CNN 算法进行优化。为了更好的表示经 TVM 转换后的 CNN 算法，以一段 CNN 算法为例，其中间表示如表 2.2 所示。首先，创建一个函数体，名称默认为“@+main”。其次，将变量 a 和 b 作为图形函数的参数进行输入，变量注明本身的维度和字符类型。最后，对算子而言，“=”左侧用依次递增的序号标明，右边表示进行的运算，以及参与运算的参数，参数是算子、变量或常量其中之一。但是，当算子的输出作为 CNN 算法的输出时时，左侧无序号标注。

表2.2 TVM 转换前后代码结构对比

转换前	转换后
a = relay.var(name="a",shape=(1,1))	def @main(%a:[Tensor((1,1),float32)],
b = relay.var(name="b",shape=(1,1))	%b:[Tensor((1,1),float32)]){
c = relay.conv2d(a,b)	%1=conv2d(%a,%b)
d = relay.add(c,a)	%2=add(%1,%a)}

2.2.2 计算图分支信息提取与表示

2.2.2.1 基础量提取

为了有效表示计算图中的基础数据，提取基础量来表示计算图中使用的最小单元，包括不同算子、卷积核大小、常量等。基础量信息如表 2.3 所示。从表中可得，基础量包括算子类型、数据类型、输入常量、默认常量和数据布局五部分。算子类型表示参与运算的算子类型，包括卷积、池化等操作。数据类型表示参与运算的变量及常量的数据类型，一般有 float32 和 int8 两种。默认常量表示算子内默认参数配置，如卷积操作需要的填充（padding）和步长（strides）。数据布局表示参与运算的数据维度及各维度大小。基础量提取过程如图 2.5 所示，图中执行过程是通过设定搜索条件遍历计算图各个算子，判断是否搜索到符合条件的基础量。若检测到，则将算子对应信息写入基础量记录表，记录的信息包括算子名称、算子包含的参数、算子数值类型和数据布局。其中，参数包括参数、变量和常量三种。之后，依次搜寻计算图并进行基础量节点的判定和记录，直至到达最后节点。

表2.3 基础量表示

指标	表示	示例
算子类型	Tensor	Conv2d、Relu
数据类型	DataType	Float32、Int32
输入常量	Input_value	%Fc.weight
默认常量	Default_value	Padding、Strides
数据布局	Data_layout	[1,1,1,1]

以卷积算子搜索为例，提取方法为，使用 TVM 读取 CNN 算法，转换为计算图，编写算法查找符合条件的卷积算子。具体提取步骤如下：

步骤 1：读取计算图中每行数据，判断算子名称是否为卷积，若是，则将名称记录到 s_name 中，并读取算子内部数据。

步骤 2：读取卷积名称后跟随的参数，即内部数据，依次判断各个参数是否为算子，若是，转到步骤 3，否则转到步骤 4。

步骤 3：若参数是算子，则将算子列为下次需要查找的点，执行步骤 1。

步骤 4：若不是算子，判断是否为常量类型，若是，则记录参数名称到参数储存表。

步骤 5：读取剩余常量、数据类型和数据布局，分别存入常量、数据类型和数据布局表。

步骤 6：计算图读取完毕，保存所有信息。

通过基础量提取方法能够获取当前节点的各类信息，并将符合条件的节点存储起来，是分支信息提取中最基础的一步。

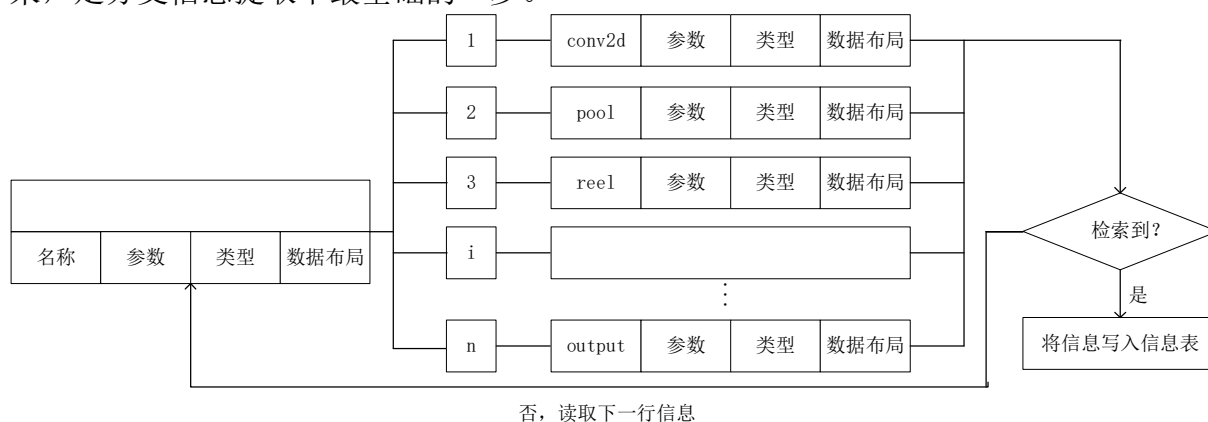


图2.5 基础量提取示意图

2.2.2.2 分支节点搜寻

计算图作为有向无环图（Directed Acyclic Graph, DAG）的一种，采用后序遍历的方式对计算图进行节点查找。由于本节分支特征搜寻方法基于后序遍历，用于搜寻分支开始节点、分支本身以及分支结束节点。因此，首先查询的是分支结束节点。节点信息如表 2.4 所示，分支结束节点的名称通常为累加（Add）和连接（Concat）两类算子，参数类型为算子（Tensor），且有多个算子作为参数。

表2.4 节点信息

算子名称	表示	解释
累加	Add(%1,%2)	
连接	Concat(%1,%2,%3)	
激活	Relu(%2)	“%”+数字 表示算子
池化	Max_pool2d(%3, pool_size=[3, 3], strides=[2, 2], padding=[1, 1, 1, 1])	“%”+名称 表示输入常量 名称+“=” 表示默认常量
卷积	Conv2d(%4, %layer1.0.conv1.weight, padding=[1, 1, 1, 1], channels=64, kernel_size=[3, 3])	

通过这些信息，可以定义分支搜索步骤如下（以 Concat 为例）：

步骤 1：后序遍历计算图，依次读取算子名称，若名称为 Add 或 Concat，则进行步骤 2，否则重复步骤 1。

步骤 2：依次读取算子的参数部分，使用 Isinstance(Tensor)函数判断参数是否全为算子（Tensor）类型，若是，进行步骤 3，否则执行步骤 1。

步骤 3：储存各个算子，对每个算子分别进行遍历。

步骤 4：依次遍历各个分支，并将各分支的节点信息进行记录。

通过分支结束节点的提取方法，可获得节点名称 Concat 和各个算子，便于后续分支节点遍历。

2.2.2.3 分支开始节点查找

计算图中各个分支经过遍历后，需要确定分支开始节点以结束分支信息的存储，这里我们根据后序遍历的特点，制定分支开始节点的查找方案。如图 2.6 所示为分支内嵌套分支的情况，本小节将以此作为案例进行开始节点的查找算法描述。

首先，查找算法从结束节点 A 开始查找，遇到结束节点 A 时，创建一维数组 S_tmp，遍历第一个分支 B-C。由于节点 D 向上无算子，所以算法搜索至节点 D 结束，将 B-C-D 存入数组 S_tmp。

其次，开始第二个分支的查找，遍历至 E 时，发现 E 为结束节点，则创建临时数组 s_tmp1 ，遍历 E 的第一个分支 F-G-D，并将其存入数组 S_tmp1 中。同时分别在 S_tmp 中查找 F、G、D，发现 D 存在 S_tmp 中，则证明 D 为分支 B-C 和当前分支的开始节点。

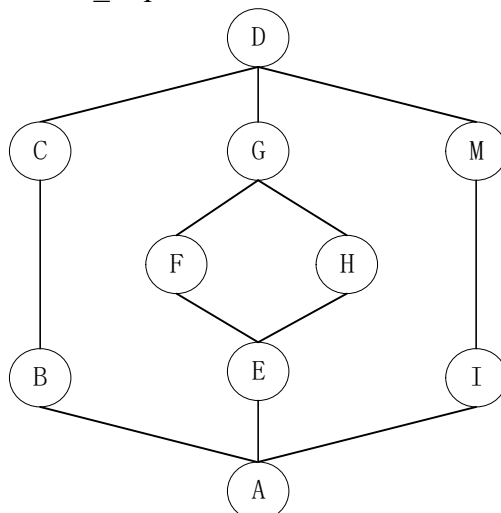


图2.6 分支嵌套结构

然后，结束以 E 为终止节点的第一个分支 F-G 的遍历后，开始以 E 为终止节点的第二个分支遍历。遍历时，将 H 的信息作为查找条件在 S_tmp1 中查找其是否存在，若不存在则遍历 H 的前驱节点 G，发现 G 存在于 S_tmp1 中。则表明 G-F-H-E 是以 G 为开始节点，E 为结束节点的分支，将分支信息记录在分支信息表中，结束 E 为终止节点的分支查找。

最后，进行以 A 节点为结束节点的第三个分支的遍历，依次在 S_tmp 中搜索 I、M、D，发现 D 存在于 S_tmp 中，则证明 I-M 为 D 的第三个分支，记录分支信息和对应开始节点，结束查找。

综上所述，对于计算图分支节点的识别，分别讨论了基础量提取、分支开始节点和分支结束节点的查找方法。此外，对于含有多个分支结构的计算图，包含特定的访问顺序，在存储信息时，采用如图 2.7 所示顺序存储信息。

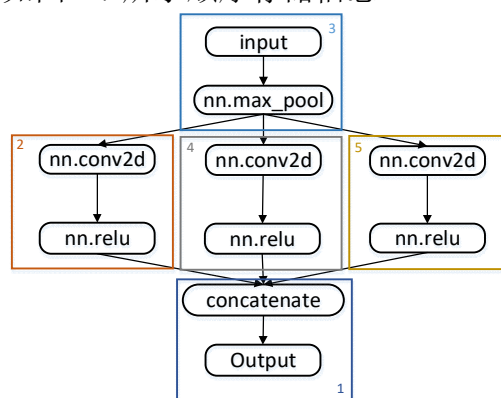


图2.7 多个分支结构计算图

2.2.3 CNN 算法特征计算图构建

在分支特征信息提取后，需要提取分支特征信息加以分析，构建特征计算图用于后续分支划分及分支并行的测试用例，提取的分支信息如下表所示。

如表 2.5 所示，提取的特征信息包括分支开始和结束节点，各个分支的开始及结束节点。从中可以得出，该分支以池化（pool）操作作为分支开始节点，分为三条分支。每条分支的起始节点都为卷积（conv2d）节点，每条分支的结束节点都为激活函数（relu）节点，结束节点以 concatenate 结束。通过这些信息，可以利用 TVM 自带的函数构造特征计算图，解决原有计算图结构重复以及分支包含节点过长的问题，简化特征，用于后续测试。

表2.5 分支信息表示

分支开始节点		
nn.max_pool2d(%34, pool_size=[3, 3], strides=[2, 2], padding=[0, 0, 0, 0])		
各分支起始节点		
nn.conv2d(%35, %Mixed_5b.branch1x1.conv.weight, padding=[0, 0, 0, 0], channels=64, kernel_size=[1, 1])	nn.conv2d(%35, %Mixed_5b.branch5x5_1.conv.weight, padding=[0, 0, 0, 0], channels=48, kernel_size=[1, 1])	nn.conv2d(%35, %Mixed_5b.branch3x3dbl_1.conv.weight, padding=[0, 0, 0, 0], channels=64, kernel_size=[1, 1])
各分支结束节点		
nn.relu(%38)	nn.relu(%45)	nn.relu(%56)
分支结束节点		
%65 = (%62, %63, %64); %66 = concatenate(%65, axis=1)		

构建计算图的代码如表 2.6 所示，首先，声明参与运算的变量参数。其次，声明表 2.5 中的八个节点，除 concatenate 节点具有特异性（起拼接算子的作用）外，均采用加法（add）、乘法（multiple）及除法（subtract）运算进行简化处理。最后，将计算图打包成函数，并给参数赋值，运行计算图。

表2.6 特征计算图构建

Simple_net_build():

```

1. x = relay.var("input0", shape=(2, 2))
2. z0 = relay.var("z0", shape=(2, 2))
3. .....
4. z6 = relay.var("z6", shape=(2, 2))
5. //参数声明
6. r0 = relay.add(x, z0)
7. a1 = relay.subtract(r0, z1)
8. a1_1 = relay.multiply(a1,z2)
9. .....
10. a3 = relay.subtract(r0, z5)
11. a3_1 = relay.multiply(a3,z6)
12. //节点建立
13. r1 = relay.concatenate((a1_1, a2_1, a3_1), axis=0)
14. var_list=[x,z0,z1,z2,z3,z4,z5,z6]
15. fun = relay.Function(var_list, r1)//计算图建立
16. params=get_params(var_list)
17. input=tvm.nd.array(np.full((2,2), 1).astype("float32"))
18. run_split(fun,params,input)//计算图运行

```

2.3 CNN 算法卷积特征提取

本节借助 TIR 对进行卷积运算的主要循环结构进行计算特征和访存特征的提取与表示，为卷积优化提供支撑，TIR 结构如表 2.7 所示。

表2.7 TIR 基本结构

名称	表示	解释
@main	prinmfn(A_1:handle)	主函数及其参数
attr	“global_symbol”:”main”	各类参数的默认设置
buffers	A: Buffer(A_2: Pointer(float32), [1024, 1024])	缓冲区变量，数据类型，数据布局
buffer_map	A_1: A	缓冲区匹配
function	for (m:int32,0,1024){A_2=0f32}	函数体，用于执行函数

在 TVM 中，中间表示（IR）分为两种，第一种是贴近上层的 Relay IR，可以使用图级别的优化方法对其进行优化。第二种是贴近底层语言的 Tensor IR，该语言更接近真实底层代码，可以对其进行操作，获取粒度更小的特征，如依赖距离，数据访问次序等特征。表 2.7 为 TIR 的基本结构及其解释，本节所有特征提取均基于以下结构进行。

通过 TIR 可以将 CNN 计算图转换为更靠近硬件的语言，以矩阵乘法为例，其中间表示如表 2.8 所示。表达式经过 TIR 分解后，一条语句被分解为三个循环语句和两个循环体。从表 2.8 可得，转换前算法高度精炼，功能明确。转换后将程序原有的计算过程逐步分解为多个操作，利于指令执行。例如，在分析转换后代码时，可以得出循环次数和访存变量的次序。

表2.8 TIR 转换前后代码结构对比

转换前	转换后
<pre> C = te.compute((M, N), lambda m, n: te.sum(A[m, k] * B[k, n], axis=k), name="C") </pre>	<pre> for (m: int32, 0, 1024) { for (n: int32, 0, 1024) { C_2[((m*1024) + n)] = 0f32 for (k: int32, 0, 1024) { C_2[((m*1024) + n)] = ((float32*)C_2[((m*1024) + n)] + ((float32*)A_2[((m*1024) + k)]*(float32*)B_2[((k*1024) + n)])) } } } </pre>

循环作为卷积神经网络中最常见的逻辑结构，结合循环的 TIR 提取其结构信息，包括循环的次数、循环中变量的数目、循环中常量数目等。循环结构的信息表 2.9 所示，其中循环条件、循环体和循环变量用于描述循环的起始和执行操作。

表2.9 循环结构信息

指标	表示	解释	作用
循环条件	For.cond	循环执行条件	检测是否继续循环
循环体	For.body	循环执行主体	判定数据依赖关系
循环变量	For.inc	循环自增量	判定循环是否结束

以表 2.8 所示循环为例，循环主体为 Te.sum 函数，函数内部变量包括 C_2、B_2 和

A_2, 循环数目为 3, 本节设计的循环结构提取步骤如下:

步骤 1: 判断当前代码是否为循环, 是则进入循环, 读取循环的初始量、起始范围及条件, 存储到 S_tmp 中, 并进入步骤 2, 否则进入下一行继续进行步骤 1 的判断。

步骤 2: 进入循环内部读取循环体, 若发现循环则执行步骤 1, 否则执行步骤 3。

步骤 3: 读取 C_2 的当前值, 存储到 S_val 中, 继续读取下一行数据, 若无数据则执行结束, 否则进入步骤 1。

通过循环结构的提取方法, 可得循环结构信息包括: 循环变量为 m, n, k , 循环条件为 $m/n/k \in [0, 1024]$, 循环体涉及到 C_2 数据更新和 B_2 、 A_2 的数据依赖, 循环自增值为 1。

2.3.1 计算特征的提取与表示

通过分析 TIR 结构中数据之间的计算关系, 提取计算特征 (数据之间的数据依赖数目和依赖距离) 用以描述数据间的关系。计算特征的具体含义如表 2.10 所示, 其中依赖数目、依赖距离分别代表操作参与运算的数据量, 变量相互调用的距离 (循环次数)。例如, $(2, 1)$ 表示的含义为, 当前数据有两个依赖的变量, 依赖距离为 1。

表 2.10 TIR 中卷积计算特征表示

特征	含义
依赖数目	参与运算的变量/常量数目
依赖距离	依赖常量/变量的最短距离

根据 TIR 中的循环结构信息读取与表示, 实现计算特征的提取。其具体步骤为:

1) 根据特定硬件结构, 使用 TVM 将卷积运算代码生成对应 TIR。然后, 对 TIR 的循环代码进行搜索。依次判断每行代码是否为有运算操作, 若有, 则将赋值变量和参与运算的常量或变量分别记录, 统计其数目, 同时记录参与运算的操作和变量/常量类型。

2) 在循环结构体中, 找到进行运算的代码, 统计参与运算的变量或常量被重复访问时所经历的最短循环次数, 循环次数的统计以循环体所在最内层循环为主。

以表 2.8 中的 $C_2[(m*1024) + n] = 0f32$ 一行代码为例, 统计依赖数目时, 即统计 C_2 所依赖的 m 和 n 两个变量, 数目为 2。其次, 记录运算类型为乘法, 加法和赋值操作。最后, 记录参与运算的数据类型为 $int32$ 和 $float32$ 。统计依赖距离时, 首先获取 C_2 所依赖的两个对象 m 和 n , 分别标记距离为 0。随着内层循环的进行, n 的依赖距离不断被更新, 最后得到 n 的依赖距离为 1。 m 的依赖距离待到下次进行内部循环时被访问, 所以 m 的依赖距离为 1024, 用符号表示 C_2 的计算特征为 $C_2(2, (1, 1024))$ 。

但是,特征提取针对 Winograd^[44]算法和 FFT^[45]算法这类转换类的卷积算法时,特征提取的效果不明显,或者提取后的特征无法显著用于提升卷积速度。Winograd 算法采用空间转换的方式,将占用计算时间较多的乘法操作转换为加法操作,同时,采用分块形式对矩阵进行操作。提取对应计算特征时,分块操作使得依赖距离降低,由于 Winograd 算法的特性,提取的特征无法有效提出优化方法。

2.3.2 访存特征的提取与表示

通过分析存储空间的数据读写方式,设计访存特征提取方法(不同数据间的执行顺序和同一个数据的访问次数)用以描述数据间的数据移动和读写频率。其含义如表 2.11 所示,从表中可以看出,TIR 中卷积访存特征包含执行顺序和数据访问次数两部分。例如,(1,22)表示该变量的执行顺序为 1,数据重复读取的总数为 22 次。

表2.11 TIR 中卷积访存特征表示

特征	含义
执行顺序	数据间的执行顺序
数据访问次数	数据重复读取总数

根据数据依赖关系,实现访存特征提取,具体步骤为:

- 1) 根据卷积计算特征,获取当前执行代码的操作类型、数据流向、参与运算的数目。
- 2) 对同一个数据,在循环运算过程中累计重复读写的数据次数,设为该数据的数据访问次数。
- 3) 对于不同数据之间,根据循环体中语句的依赖关系,提取数据间的依赖关系,按照依赖关系的前后顺序,标记数据的执行顺序。

以%1: C₂[((m*1024) + n)] = 0f32 和%2: C₂[((m*1024) + n)] = ((float32*)C₂[((m*1024) + n)] + ((float32*)A₂[((m*1024) + k)]*(float32*)B₂[((k*1024) + n)]))为例。根据访存特征提取步骤,首先获取%1 和%2 的操作类型为乘累加,数据流向为一维数组连续存储,参与运算的数目为 2 和 4。其次,对于同一个数据,在循环中统计读写次数,则 C₂、A₂ 和 B₂ 的读写次数均为 1024。最后对于不同的数据,循环中的依赖关系为, C₂ 依赖 A₂ 和 B₂,三者同时依赖 m 和 n。n 为最外层循环的初始条件,由依赖关系可得出数据的先后执行顺序为 n++, m++, %1, %2,得到的访存特征如表 2.12 所示

对于快速傅里叶变换卷积算法^[44],采用线性变换对矩阵进行运算,运算时会对卷积核进行膨胀操作,通常适用于卷积核大小为 9×9 以上的矩阵。在进行变换时,需要对

卷积核进行较多的填充,算法的特性为卷积核越大速度提升越明显,这样对算法进行访存特征提取时,由于算法的特性,使得无法将数据访问频率高的大部分数据放到较近距离。而且,当前主流神经网络算法的卷积核大小为 3×3 和 7×7 ,FFT 算法加速比不高且占用内存过大。

表2.12 矩阵乘法访存特征

变量	执行顺序	数据读取次数
m	1	3096
n	1	3096
k	2	1024
C_2	4	1
A_2	3	1024
B_2	3	1024

2.4 CNN 算法特征提取结果分析及有效性验证

2.4.1 计算图分支特征分析及有效性验证

2.4.1.1 实验环境与实验方案

实验选用 TVM0.8dev0 神经网络编译器作为软件测试平台,Ubuntu18.04 作为实验的操作环境,使用 Intel Xeon Gold 6248R @3GHz 作为实验的硬件平台。实验数据集如表 2.13 所示,为卷积神经网络中常用的网络模型,包括 ResNet^[46]网络、GoogleNet 等。

表2.13 实验数据集

名称	基本块	卷积层数	全连接数	池化数
ResNet18	8	17	1	4
ResNet152	50	151	1	4
Inception_V3	11	94	0	59
GoogleNet	10	57	0	14
Squeezenet	7	26	0	4
VGG16	6	13	3	5

计算图分支特征分析及有效性验证的实验方案从 CNN 算法的计算图分支特征提取

和特征计算图的执行时间两方面进行，以验证计算图分支信息提取的有效性。特征提取是将各个神经网络作为输入，经过分支提取算法，获取不同网络的分支信息，以构建特征计算图，在 2.4.1.2 小节中进行详细描述。然后，利用分支特征信息手动对不同网络的特征计算图进行划分，用实验结果证明分支提取有效性，在 2.4.1.3 小节中进行描述。

2.4.1.2 分支特征提取及分析

经过 2.2 章节计算图分支信息提取算法的遍历搜索，得到如表 2.14 所示的分支信息表。表中信息以 inception_v3 为例，根据表中数据可得，分支特征提取包括分支结构的各个起始和终止节点、分支结构数量和是否存在嵌套分支等信息。可以得出分支开始节点为 Concatenate 和 MaxPool 两种，其中 Concatenate 作为分支开始节点的情况占多数。

表2.14 Inception_v3 网络分支信息表

特征	表示
分支开始节点	MaxPool (1)、Concatenate (13)
各个分支起始节点	Conv2D (34)、AvgPool (1)、MaxPool (1)
各个分支终止节点	Relu
分支结束节点	Concatenate
分支结构数量	11
最大子分支数量	4
有无嵌套分支	有
嵌套分支最大子分支数量	2

各个分支的终止节点统一为 Relu，最后分支结束节点为 Concatenate。将这些信息进行汇总，能够利用其构建出一个特征计算图作为 Inception_v3 网络的特征图。其中，分支的开始节点为 Concatenate，分支个数为 4，各个分支的起始节点为 Conv2d，终止节点为 Relu，分支结束节点为 Concatenate。在第二个分支中创建一个包含两个分支的嵌套分，分支信息提取算法在构建过程已在 2.2.3 节中描述。

根据获取的分支信息，经过总结后，使用 TVM 搭建一个符合 Inception 分支特征的特征计算图。为突出特征，减少其他不必要的干扰，使用 Concatenate (Concat) 作为分支结构的开始和结束，分支内仅有分支开始 (Conv2d) 和结束节点 (Relu)，由于分支大部分以 Concatenate 结束，所以嵌套分支也以 Concatenate 作为结束节点。

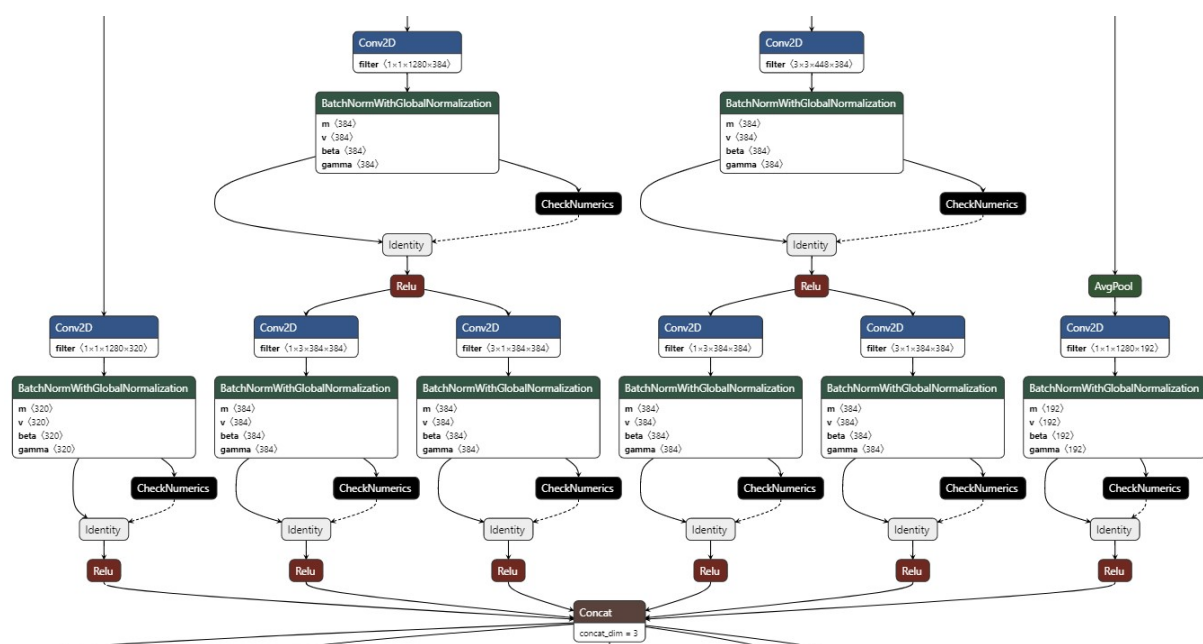


图2.8 Inception_V3 网络部分结构示意图

如图 2.8 所示，为表 2.14 中提取分支信息中最匹配的结构。该分支包含四个分支，其中两个分支为嵌套分支。根据图 2.8 和表 2.14 的信息建立的特征图如图 2.9 所示，该特征图包含两者的特征信息，有效提取了 inception 网络的分支特征。

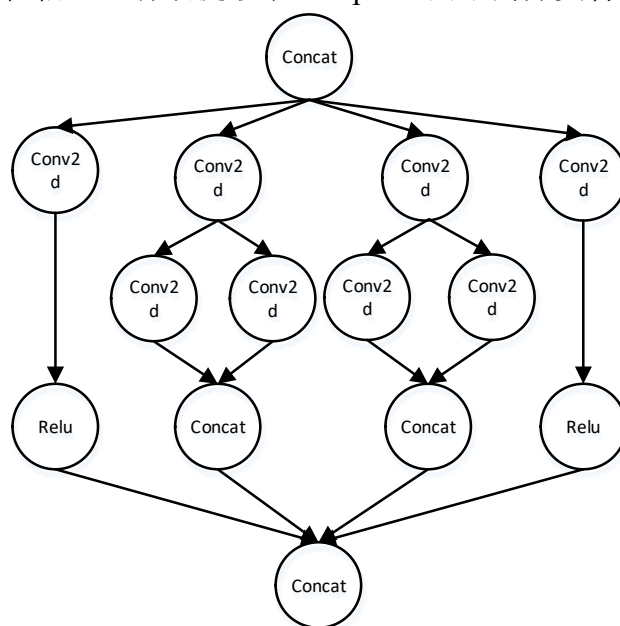


图2.9 Inception_v3 特征计算图

2.4.1.3 基于分支信息的计算图划分性能对比

针对表 2.13 所示的六种神经网络模型，使用基于分支特征的划分方法和未经划分的 CNN 算法进行对比，划分获取的性能比较如图 2.10 所示。其中，划分实现形式采用流水线中手动划分来切分计算图，与文献[4]运行函数作对比。输入参数为图片矩阵数据。

从图中可以看出,根据分支特征信息得出的划分方案,可以有效的使模型加速,平均加速在 18%左右,可以验证分支特征信息的有效性。

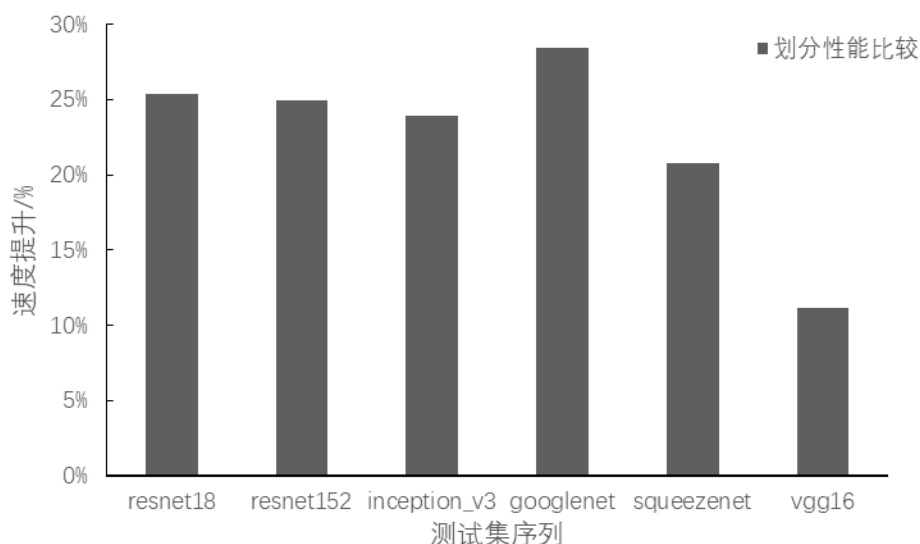


图2.10 划分性能比较

2.4.2 CNN 卷积算法特征分析及有效性验证

根据 TVM 中 TIR 提取的卷积特征如表 2.15 所示,其中,主要为矩阵运算的循环特征提取,包含计算特征和访存特征两部分。由此表可以得出,传统卷积运算方法随着卷积规模的增大,依赖距离也不断增加,因此需要将依赖距离进行缩短。同时,根据同一数据使用的频度(重复数),将频度较高的数据使用间隔缩短,修改原有矩阵的运算方式,可以加快运算速度。

表2.15 传统 CNN 卷积运算特征描述

卷积规模	计算特征		访存特征	
	依赖数目	依赖距离	重复数	访问顺序
128	256	64	64	A×B A 按行访问 B 按列访问
256	512	128	128	
512	1024	256	256	
1024	2048	512	512	

依据表 2.15 中的特征信息的分析,提出针对卷积运算的优化方案,以验证特征提取的有效性。优化操作主要分为两部分,第一,针对计算特征修改循环顺序以解决依赖距离长的问题,使较长循环放在内部,较短的循环放在外部。第二,基于访存特征修改矩

阵运算的读取顺序，将原先的矩阵分成各个小块进行运算，使重复访问的数据时间间隔缩短。

在优化方案的指导下，本节使用表 2.15 中的不同卷积规模作为测试用例，进行优化方案的测试，测试结果如图 2.11 和图 2.12 所示，分别为计算特征优化和访存特征优化两部分。由于本节进行的实验所得时间单位为毫秒，因此较少的加速在图中呈现出较大提升，易被其他因素干扰。为解决其他因素干扰问题，重复运行实验七次，取中间数据平稳的时间作为最终时间。实验采用文献[4]中 TVM 本身运行函数作为基准，将文献[5]中的传统优化方法和本章方法进行对比，从实验结果可以看出，基于卷积特征的优化方法在优化基本的卷积运算时获得了平均 20% 的加速，证明卷积特征提取能够有效提取和表示卷积信息。

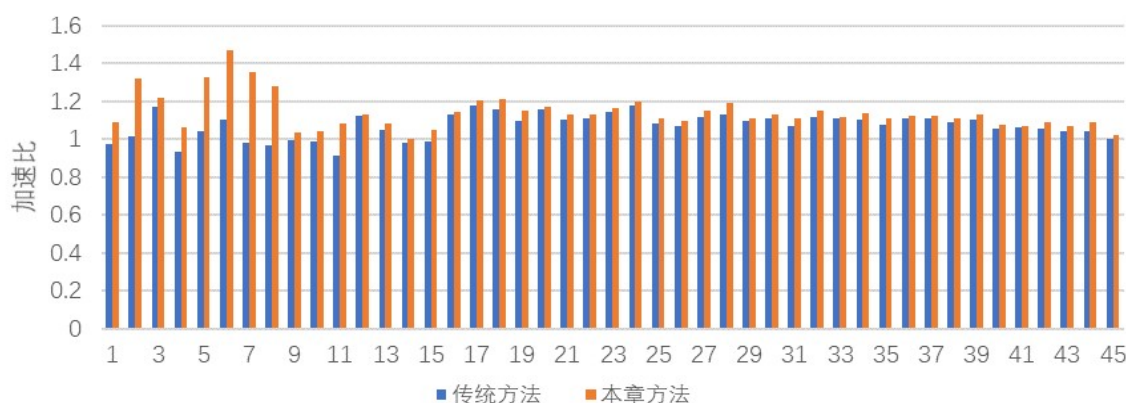


图2.11 计算特征优化

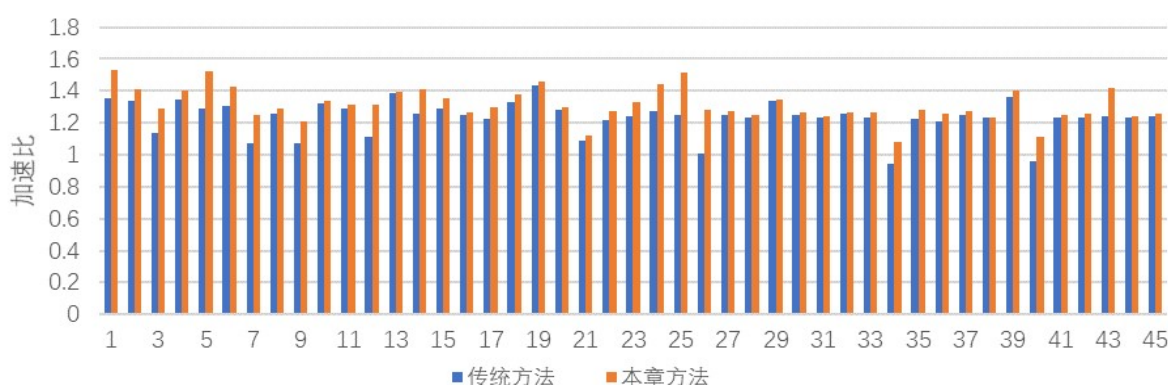


图2.12 访存特征优化

为验证 2.3.1 小节中 Winograd 算法所提取特征无法对其进行有效加速优化，本节针对该算法进行特征提取并采用本节的优化方法对算法进行优化，实验采用本节优化后的 Winograd_opt 和文献[44]作对比，实验结果如图所示。从图 2.13 中可以看出，算法随着数据规模的增大，推理所用耗时也随之增加。在数据规模较小时 Winograd 和 Winograd_opt 在推理速度上相差不明显，甚至在 6 和 7 上略有提升。但是，随着数据规模的不断增加，opt 算法在速度上明显低于 Winograd 算法，这是因为算法提取特征时，

重点需要算法本身访存模式复杂度一般，提取的计算特征和访存特征具有很强的规律性，而该算法本身访存模式复杂，使用本节优化方法无法对其进行有效的加速。

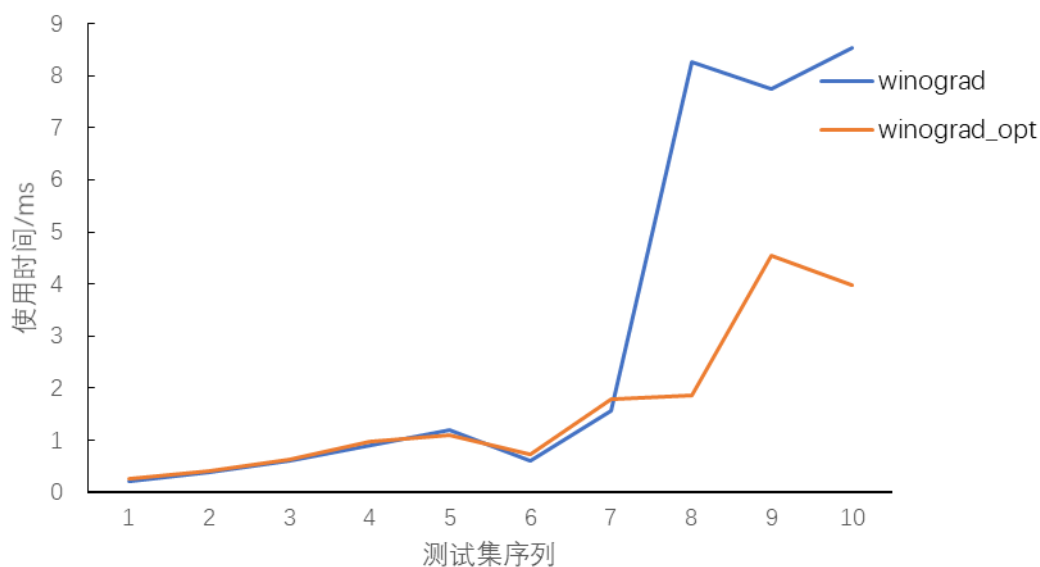


图2.13 Winograd 和 Winograd 优化对比

2.5 本章小结

为挖掘 CNN 算法的分支特性和卷积特性，本章深入研究分析的 CNN 算法的行为模式。首先，使用 TVM 获取计算图中的基础结构信息、分支结构和分支间依赖关系。其次，根据此类信息构建特征计算图来描述 CNN 算法的分支信息，获取 CNN 算法的分支特征。然后，提取了卷积中的访存模式，即访存特征和计算图特征，用于指导卷积的优化操作。实验结果表明，依据获取的分支特征制定划分方法与 TVM 传统方法对比，平均获得了 18% 的性能提升。基于卷积特征制定的卷积优化方法与传统方法对比，平均获得了 20% 的性能提升。说明 CNN 算法行为模式分析能有效提取 CNN 算法特征，挖掘特征信息，设计的特征提取算法和提取的特征用于指导第三章卷积特征优化和第四章分支特征划分。

3 基于访存模式的 MEC 卷积算法优化方法研究

为更有效利用计算资源，需要挖掘 CNN 中卷积算法的访存模式，研究符合卷积算法访存模式的优化方法，以提高硬件资源的利用率。因此，本章节结合第二章卷积特征信息提取算法，针对 MEC 卷积算法存在的访存效率低的问题进行研究，提供设计方案及优化算法。

3.1 MEC 卷积算法访存模式优化研究

访存模式是指算法或程序在访问存储器件时所体现出的规律，其规律主要有两种：空间特征和时间特征。时间特征指数据使用的频度和生命期，典型应用是最近最少替换策略，将一段时间内最长时间未被访问的数据块替换出去。空间特征指空间局部性，典型应用是数据预读取技术，将所取数据及相近的数据块一同取到缓存中。在下次计算单元需要数据时，从缓存中读取，无需从内存中取数据，从而减少访存延时^[47]。

卷积运算过程中所涉及的数据以矩阵形式进行存储，矩阵在内存中有行主序和列主序两种存储方式^[48]，如图 3.1 所示，行主序在存储矩阵时按行写入内存，列主序存储时按列写入内存。矩阵在内存中的存储方式由编程语言决定，C++ 和 Python 等神经网络使用的语言，其矩阵的存储方式为行主序。优化算法只针对矩阵存储方式为行主序的编程语言，本节使用的编程语言为 Python。



图3.1 不同设备中内存组织形式

MEC 算法卷积过程如图 3.2 所示，计算过程主要分为两步：中间矩阵转换和矩阵乘法。卷积过程中输入矩阵通常采用四维矩阵存储数据，即批次 (Batch)、通道 (Channel)、高度 (Height) 和宽度 (Weight)，分别用 N、C、H、W 表示。参与运算的矩阵有输入矩阵 (Input)、中间矩阵 (Lower)、卷积核矩阵 (Kernel) 和输出矩阵 (Output)，分别

用 I、L、K 和 O 表示。本节讨论的 MEC 算法中，输入矩阵采用二维表示，即 $N=1$ ， $C=1$ ， $H=1$ ， $W=7$ 。使用二维数据可以排除其他维度干扰，更加直观的对 MEC 算法进行理解，同时分析算法中存在的不足。

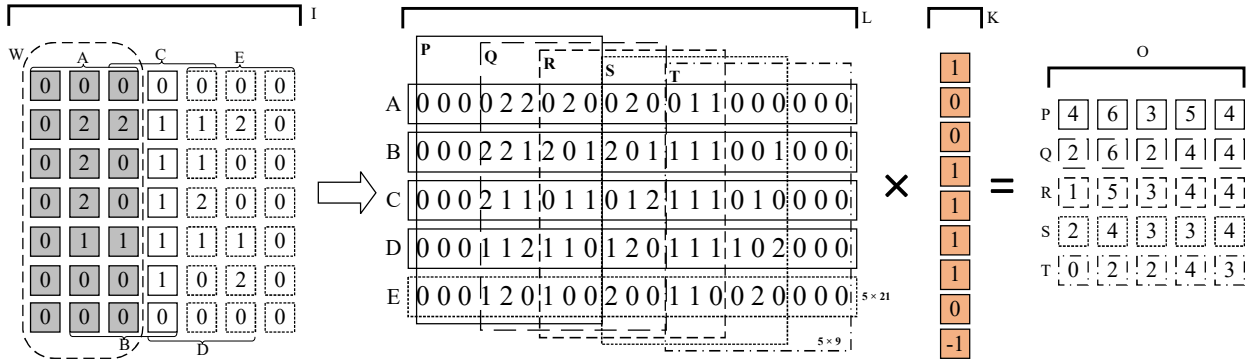


图3.2 MEC 算法计算过程

3.1.1 中间矩阵转换访存特征优化研究

中间矩阵转换作为 MEC 算法的第一步，主要负责把原始输入矩阵 I 转换成中间矩阵 L。主要计算步骤如下：首先，读取输入矩阵中 $A=I[0:7,0:3]$ 部分的每个值，并将其组织成中间矩阵 L 的第一行。然后，以步长 S（步长为滑窗每次移动的距离，默认步长为 1）作为滑动距离，滑动窗口 W 至矩阵 $B=I[0:7,1:4]$ 处，读取矩阵 B 中数据，组织成中间矩阵 L 的第二行。以此类推，完成从输入矩阵到中间矩阵的转换。

本节通过设计对中间矩阵的提取算法分析中间矩阵的计算特征和访存特征，提取用于描述中间矩阵转换的访存模式信息，提出优化方案，为中间矩阵转换优化提供支撑。结合第 2.1.2 小节对中间矩阵进行访存特征和计算特征的提取，提取过程主要分为两步，特征判断和特征提取。

设计特征判断算法主要用于特征提取方法的选择，由于不同矩阵计算时采用的计算方式不同，仅涉及到访存特征或者计算特征。需要对矩阵计算进行概要特征的提取，以判断存在的相应特征，为下一步详细特征提取提供选择，防止对无效特征的提取。判断时，使用算法分别对表 3.1 中的四项特征进行特征判断。为简化判断步骤，使用 3×3 小矩阵对作为特征判断算法的实验对象。首先对执行顺序和数据访问次数进行判断，当矩阵不是对同一个元素重复进行读取时或者未对矩阵进行读取时，表明该矩阵计算过程中存在访存特征。其次对依赖数目和依赖距离进行判断，对矩阵搜索时发现矩阵中没有重复利用的数据时，表明该矩阵计算中不存在计算特征。

特征提取主要根据特征判断的结果，对特征进行提取。如表 3.1 所示，从输入矩阵到中间矩阵转换过程中，数据之间没有相互依赖，所以转换过程中无法获取计算特征，只体现出访存特征。

表3.1 中间矩阵转换访存和计算特征表示

特征	提取结果
执行顺序	按滑动窗口 W 进行从左往右滑动，滑窗内优先列读取
数据访问次数	最大访问次数为 3 次，最少访问次数为 1 次
依赖数目	数据之间无依赖
依赖距离	数据之间无依赖

本节通过 2.3.1 节提出的访存特征优化算法，针对 MEC 算法的中间矩阵转换进行访存特征提取。具体访存特征的获取步骤如下：

- 1) 对于数据访问次数，当访问一个新数据时，对应创建一个标记 i ，称为递增量，并初始化为 1，再次访存该元素时，则将 i 加 1。继续下一数据的访问，直至访问结束，得出 $I[0:7,0]$ 与 $I[0:7,3]$ 这两列的每个数据的递增量 i 为 1，其余数据的递增量 i 为 2。
- 2) 对于执行顺序，首先，在访问矩阵之前，创建变量 j ，初始化为 0。其次，当访问一个新元素时，创建一维数组，将 j 存入数组末尾，变量 j 自增。最后，完成矩阵元素的访问，依次输出各个元素对应的数组，得到数据的访问顺序。

对输入矩阵的 64 个节点进行访存特征信息提取，提取结果如表 3.2 所示，以 $A=I[0:7,0:3]$ 和 $B=I[0:7,1:4]$ 为例。由提取的访存特征可知，MEC 算法在读取输入矩阵时，采用分段读取的方式，且每段划分都以列为主。这种划分方式不适应矩阵在内存中行优先的存储方式，每段数据间的地址不连续，同一数据的执行顺序跨度过大，导致 cache 命中率降低，访存延时长。为解决中间矩阵转换过程中每段数据不连续的问题，结合访存模式中两种访存特征，提出一种针对中间矩阵转换的优化方案。主要利用预读取技术，改变数据在内存中的组织模式，把数据组织成计算单元可以顺序使用计算的形式，以增加数据的连续性，加快数据读取顺序，具体方案设计将在 3.2.1 章节中说明。

表3.2 中间矩阵转换访存特征信息

节点	执行顺序	数据访问次数
$I[0,0]$	[1]	1
$I[0,1]$	[2,21]	2
$I[0,2]$	[3,22]	2
$I[0,3]$	[23]	1

3.1.2 矩阵相乘计算特征优化研究

MEC 算法第二步是矩阵乘法，将中间矩阵与卷积核矩阵相乘得到输出矩阵。首先，把中间矩阵分成 $\{P、Q、R、S、T\}$ 五个子矩阵，每个子矩阵的大小由 $(OW \times (KW \times KH) = 5 \times 9)$ 决定，子矩阵之间的距离由 $(SH \times KW = 3)$ 决定。其次，将卷积核矩阵按行展开成列向量。最后，把五个子矩阵分别与列向量相乘。例如，子矩阵 $P_{59} = L[0:5, 3:12]$ 与卷积核相乘时，得到输出矩阵 O 中 P 行， Q 与 K 相乘得到矩阵 O 的 Q 行，以此类推完成矩阵乘法运算，得到输出矩阵 O 。这种方法很大程度上减少了原始 $Im2col$ 卷积的内存开销。同时，根据 MEC 算法的特性，每次只计算输出矩阵中的一个值，可以充分利用并行库进行加速。

同 3.1.1 节类似，需要对矩阵相乘进行特征信息提取，不同的是，矩阵运算中包含的计算特征和访存特征两部分。访存特征的提取步骤已在 3.1.2 节中描述，本小节使用提取算法对矩阵相乘的计算特征进行提取，并加以分析，以中间矩阵 L 的 $L[A, 0]—L[A, 6]$ 为例，矩阵相乘的特征表示如表 3.3 所示，访存示例如表 3.4 所示。

表3.3 矩阵相乘访存和计算特征表示

特征	提取结果
执行顺序	按滑动窗口 P 进行从左往右滑动，滑窗内优先行读取
数据访问次数	最大访问次数为 3 次，最少访问次数为 1 次
依赖数目	输出矩阵 O 中每个元素依赖数目为 18
依赖距离	数据间最大距离为 6，最小依赖距离为 3

表3.4 矩阵计算访存特征示例

节点	执行顺序	数据访问次数
$L[A, 0]$	[1]	1
$L[A, 1]$	[2]	1
$L[A, 2]$	[3]	1
$L[A, 3]$	[4, 45]	2
$L[A, 4]$	[5, 46]	2
$L[A, 5]$	[6, 47]	2

计算特征主要体现在中间矩阵同卷积核矩阵相乘部分,由于中间矩阵为了节省存储空间,进行矩阵运算时对中间矩阵数据进行重复读取。因此,数据被重复访问,能够进行依赖数目和依赖距离的提取。本节使用 $O[P,0]$ — $O[T,0]$ 作为提取计算特征的示例,对矩阵相乘部分进行计算特征的提取,如表 3.5 所示,计算图特征提取主要对依赖数目和依赖距离两个特征进行提取。针对依赖数目,在卷积运算中,对输出矩阵中的同一个数据 **data**,由输入矩阵同卷积核进行多次乘累加操作得出。因此,对于 **data**,设置依赖数目为变量 **k**,当 **data** 参与运算时,**k** 自增,直至不再对 **data** 进行操作时,得到的 **k** 值为 **data** 所依赖的变量数。对于依赖距离,首先,设置变量 **i** 统计内层循环的循环次数。其次,每个元素设置一个临时观察量 **j**,记录第一次访问的 **i** 值,当再次访问到同一元素时,两者相减取绝对值得出依赖距离。

表3.5 矩阵计算的计算特征示例

节点	依赖数目	依赖距离
$O[P,0]$	18	45
$O[Q,0]$	18	42
$O[R,0]$	18	42
$O[S,0]$	18	42
$O[T,0]$	18	-

经过特征可以看出,矩阵相乘时,对同一数据的依赖距离较长,即两次访问的距离较长,增加了查找数据的耗时。从访存特征上看,数据的执行顺序不是按照列优先或者行优先的顺序进行读取,这导致访存时间耗时增加。并且,随着输入通道数 **IC** 和输出通道数 **OC** 的增加,卷积核矩阵大小也随之增加,导致依赖距离和执行顺序的增加,如何加快卷积核矩阵的处理速度成为不可忽视的一点。为解决中间矩阵中存在的问题,需要充分利用访存模式中的时间特征。修改计算顺序和数据在内存中的排列方式(即数据布局),以解决访问的数据地址不连续的问题,从而提高 **cache** 命中率,降低访存延时。具体方案设计及实现将在 3.3 章节进行详细描述。

3.2 MEC 卷积算法中间矩阵转换访存优化设计与实现

3.2.1 中间矩阵访存特征优化方案设计

基于 3.1 章节提取的计算和访存特征以及访存模式,设计 MEC 中间矩阵访存特征

优化方案，主要包括优化原则和详细的优化方案。中间矩阵的优化原则：1) 根据 TVM 下获取的中间矩阵访存特征信息，获得数据访问次数和访问顺序。2) 该特征能够表述中间矩阵的数据转换方式，如数据读取顺序、数据依赖距离等。

中间矩阵访存模式分析过程如图 3.3 所示，从图中可得。首先，通过 TVM 分析中间矩阵的特征，获取数据的访问次数和访问顺序。然后，根据获取的特征信息，进行特征分析。最后，结合特征信息提出针对中间矩阵的优化方案，具体优化过程将在 3.2.2 小节进行说明。

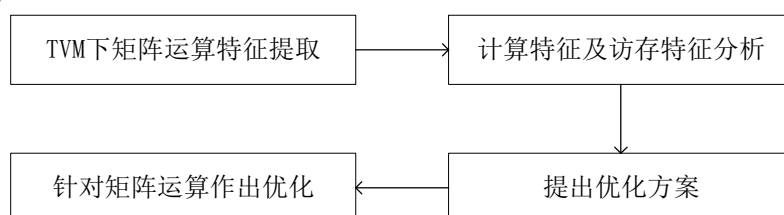


图3.3 中间矩阵分析过程

中间矩阵优化修改了中间矩阵的转换方式，主要分为输入矩阵读取和中间矩阵数据写入两部分。输入矩阵读取修改了 MEC 算法读取输入矩阵的方式，通过卷积核长度和步长确定一次性读取的长度（即滑块长度）和每次读取数据的间隔。同时，将按列读取的方式改为按行读取，按照滑块长度对输入矩阵的每一行进行滑动读取，直至读取完整个输入矩阵，完成输入矩阵的读取。中间矩阵数据写入主要修改了 MEC 算法中间矩阵的数据排列顺序，把滑块读取的每一行数据放入中间矩阵中，作为中间矩阵的一列，重复完成写入，直至中间矩阵转换完成。

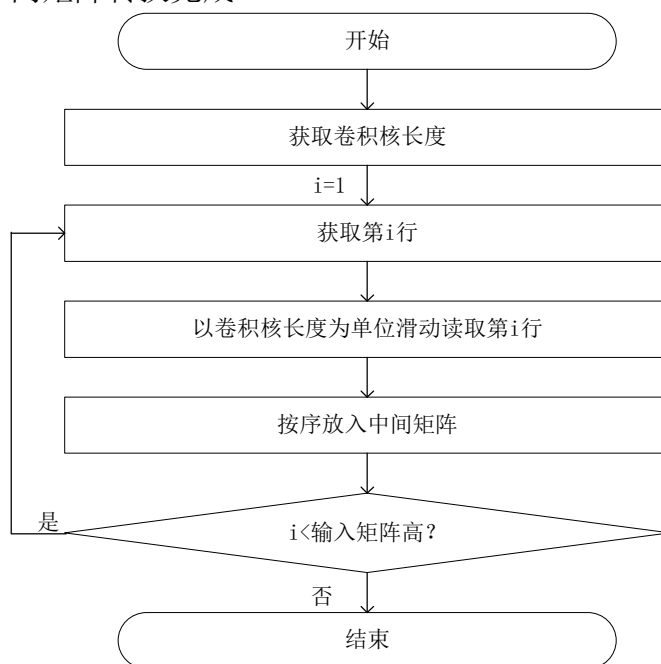


图3.4 中间矩阵优化方案

借助 TVM 获取的中间矩阵特征信息和优化方案，提出针对中间矩阵的优化方案，

详细的优化方案如图 3.4 所示, 这里以 7×7 的输入矩阵和 3×3 卷积核矩阵举例说明优化过程。首先, 获取卷积核长度 $H=3$, 将此作为输入矩阵一次性读取的数据长度。其次, 获取输入矩阵的第 i 行数据, 将 $H=3$ 作为滑块长度, 依次对第一行数据进行读取。最后, 将读取后的数据依次存储进中间矩阵, 判断当前行是否为最后一行, 若是则终止矩阵转换, 若当前行不是最后一行, 则重复步骤 2。该优化方案针对访存模式进行优化, 解决了数据间的读取距离长的问题。具体优化工作将在 3.2.2 节和 3.2.3 节中进行描述。

3.2.2 单通道下矩阵转换算法实现

单通道卷积矩阵转换部分如图 3.5 所示, 为解决中间矩阵转换过程中存在的数据访存距离长, 导致的读取速度慢的问题, 本文重新设计输入矩阵的读取方式和中间矩阵的数据读取方式。将按列读取修改为按行读取, 以保证数据读取的连续性, 同时, 修改数据存储在中矩阵中的顺序。本节将以 7×7 的输入矩阵和 3×3 的卷积核矩阵为例, 详细阐述单通道中间矩阵转换的修改过程。

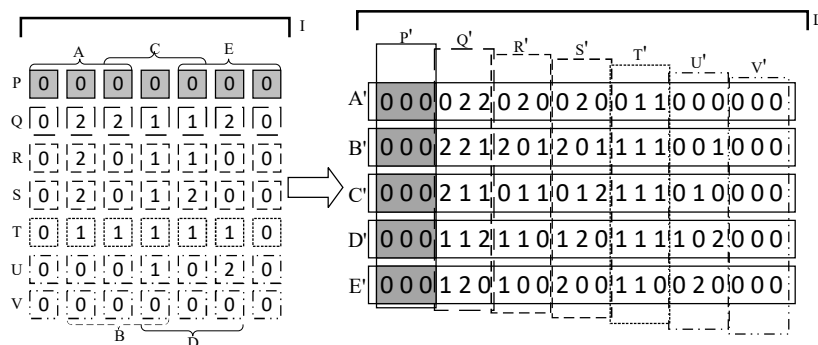


图3.5 单通道输入矩阵转换中间矩阵过程

首先, 本节对输入矩阵区域按行进行划分, 依次划分成 $\{P, Q, R, S, T, U, V\}$ 七个行向量, 如图 3.5 左部分所示。每个行向量作为单独的数据块, 便于程序分别进行处理。

然后, 分别对七个行向量进行读取。例如, 对于行向量 $P=I[1,1:7]$, 采用核矩阵宽度 $KW=3$ 作为滑块宽度。具体转换时, 读取区域 $PA=P(1:3)$, 将数据按序填充到中间矩阵 P' 区域 A' 行所示部分 ($P'A'$)。同时以步长 $SH=1$ 作为步幅向右滑动滑块, 得到 $\{PA, PB, PC, PD, PE\}$ 五个子向量, 分别填充到中间矩阵 L 中对应 $\{P'A', P'B', P'C', P'D', P'E'\}$ 部分, 完成从输入矩阵 P 到中间矩阵 P' 的转换。

最后, 向量 P 处理完成后, 对输入矩阵中剩余行向量进行处理。由于各个行向量读取顺序一致, 因此重复上述过程, 将向量 Q-V 读取并转换至中间矩阵 L 的 Q'-V', 得到中间矩阵 L。在矩阵转换过程中, 行向量的处理过程相互独立, 没有数据间依赖关系, 使用 TVM 自带的并行库对各个行向量进行并行加速。

为验证中间矩阵转换的有效性，本节获取了优化算法中间矩阵转换过程的访存特征，以图 3.1 中输入矩阵 $I[0,0]$ — $I[0,3]$ 为例，具体特征如表 3.6 所示。从表中可以得出，相比于原有中间矩阵转换的表 3.2 而言，优化算法采取的访存方式可以让 $I[0,0]$ — $I[0,3]$ 实现连续读取，验证了访存优化的有效性。

表3.6 单通道中间矩阵转换优化访存特征示例

节点	执行顺序	数据访问次数
$I[0,0]$	[1]	1
$I[0,1]$	[2,4]	2
$I[0,2]$	[3,5]	2
$I[0,3]$	[6]	1

3.2.3 多通道下中间矩阵转换算法实现

输入矩阵到中间矩阵的转换过程如图 3.6 所示，采用 $IN \times IH \times IW \times IC$ (NHWC) 作为输入矩阵的数据布局，多通道卷积增加批次 (IN) 和输入通道 (IC) 两个维度。图中含有三个批次，不同颜色代表不同通道，每个批次内采用二维方式表示三维数据 (宽、高以及输入通道)。与 3.2.2 节中间矩阵转换类似，多通道中间矩阵优化主要分为输入矩阵读取方式优化以及中间矩阵数据写入优化两部分。与单通道卷积中间矩阵转换不同的是，多通道卷积需要额外对批次和通道数进行处理。

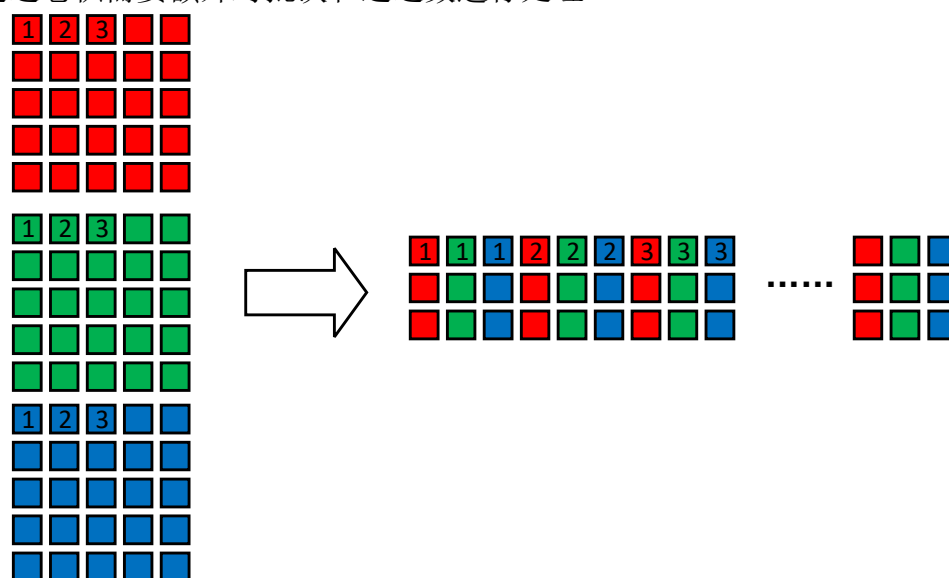


图3.6 多通道矩阵转换示例

多通道采用 NHWC 作为输入矩阵的数据布局，即四维矩阵中批次数在最外层，通道数在最内层。在读取输入矩阵时，优先读取批次数会使数据的访问距离增大。因此，

修改算法的读取顺序，采用从内向外的读取顺序进行读取，即优先访问通道数，按照通道数、宽、高和批次数的顺序对输入矩阵进行读取。本节分别对三维（宽、高和通道数）和四维（宽、高、通道数和批次）进行修改。三维矩阵相对二维矩阵增加了批次数，输入矩阵的维度从外向内排序为高、宽、通道数。由于通道数在最内层，需要对输入矩阵的读取和中间矩阵的写入这两个方面进行改进。在读取方面。如图 3.6 所示，红绿蓝三种颜色分别代表三个不同通道，对于输入矩阵的读取操作，需要对不同通道的同一位置上的数据进行读取。首先读取各个通道中的 1，再依次读取各个通道中的 2 和 3，完成一次以卷积核滑块为长度的数据读取。因为需要对不同通道上的同一位置数据进行读取，将滑块的长度设置为卷积核宽与输入通道数的乘积，按照单通道卷积里中间矩阵的计算方式依次对输入矩阵进行读取。在写入方面，申请的矩阵大小以通道数为高，因此高为 3，列宽以输出矩阵的行高和通道数的乘积为长度。进行写入操作时，将读取的数据按照通道数、宽和高的顺序进行写入，即如图 3.6 中间矩阵所示，优先将各个通道数的同一宽高位置的元素写入，之后在进行 2、3 的写入。

四维矩阵在矩阵的最外层增加了批次数，在矩阵读取方面，本节增加了对批次数的处理。由于在图 3.7 中，本文对三维矩阵以 HWC（高、宽和通道数）的顺序对读取做出了优化，且四维由多个图 3.7 中三维矩阵组成。所以对于四维矩阵，按照批次顺序对各个批次中的三维矩阵进行读取。将数据写入中间矩阵时，由于增加了批次数，需要对写入方式进行修改。写入时，首先进行通道数的写入，如图 3.7 中间矩阵所示，写入中间矩阵时，按照通道数依次进行写入。同时，读取输入矩阵的滑块采用 $L = \text{通道数} \times \text{宽} \times \text{高}$ 的长度对其进行读取，中间矩阵写入按照滑块 L 的长度写入每一行的矩阵中。

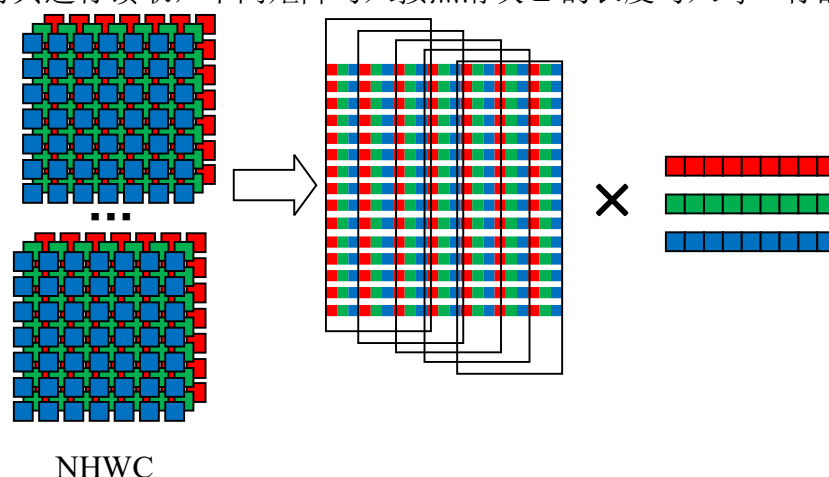


图3.7 多通道中间矩阵优化

此外，中间矩阵的宽扩充为批次数 $\times$ 高。将单通道扩充至多通道时，中间矩阵长度会随着通道数的数量增加，而高度不变。对于内存组织形式为行主序的 TVM 来说，本节提出的多通道中间矩阵优化算法，优先读取中间矩阵的可以更好的增加 cache 命中率。

3.3 MEC 卷积算法的矩阵运算优化设计与实现

3.3.1 矩阵计算特征优化方案设计

矩阵运算作为卷积算法中主要的实现方式，一直是卷积优化的主要目标，本节根据 MEC 算法矩阵运算的方式，提出基于计算特征和访存特征的设计原则与设计方案。设计原则如图 3.8 所示，首先，基于 TVM 获取矩阵运算过程中的计算特征和访存特征。提取特征时，使用 2.3 小节制定的特征提取算法，对矩阵计算进行特征提取。主要针对中间矩阵以及卷积核矩阵进行特征提取，遍历矩阵运算时生成的 TIR 原语，统计各个元素的访问次数、访问顺序等特征。其次，对于获取的特征信息，针对性的加以分析，提出优化方案。对于获取的访问次数、访问顺序等特征，设计将访问次数较多的数据集中访问，以及将需要连续访问的数据在内存中排列成顺序读取的方式来制定优化方案。最后根据提出的矩阵运算优化方案，对矩阵进行优化。



图3.8 矩阵运算的计算和访存特征设计原则

MEC 算法进行矩阵运算时，分为中间矩阵读取、卷积核矩阵读取以及计算三部分，因此，本节综合考虑这三部分，提出了矩阵运算优化设计方案。如图 3.9 所示，优化中间矩阵读取时，以卷积核长度为滑块长度，对每一行进行滑动读取。读取时，对内存中相邻数据进行连续读取，可以减少读取时间。卷积核矩阵读取时，修改卷积核在内存中的组织形式，将内存中数据顺序修改为优化算法的访问顺序，以加快卷积核的读取速度。最后计算时，利用 TVM 内置的计算函数对计算方式进行修改，分别读取输入矩阵和卷积核矩阵的每一行进行运算，得到输出矩阵的每一列。使用具体步骤如下：

1) 将卷积核矩阵由多维度转换为二维，高度等于输出通道数，宽度等于输入通道数、原矩阵宽和高这三个维度的乘积，然后获取转换后的卷积核长度 L 。

- 2) 读取中间矩阵的第 i 行, i 初始值为 1, 读取跨度为 L 。
- 3) 读取卷积核矩阵的第 j 行, j 的初始值为 1。
- 4) 将读取的数据进行相乘累加, 得到输出矩阵中的一个值。
- 5) 判断 j 是否小于卷积核矩阵高度, 若是, 则执行步骤 3。
- 6) 判断 i 是否小于中间矩阵高度, 若是, 则执行步骤 2。
- 7) 得到完整的输出矩阵。

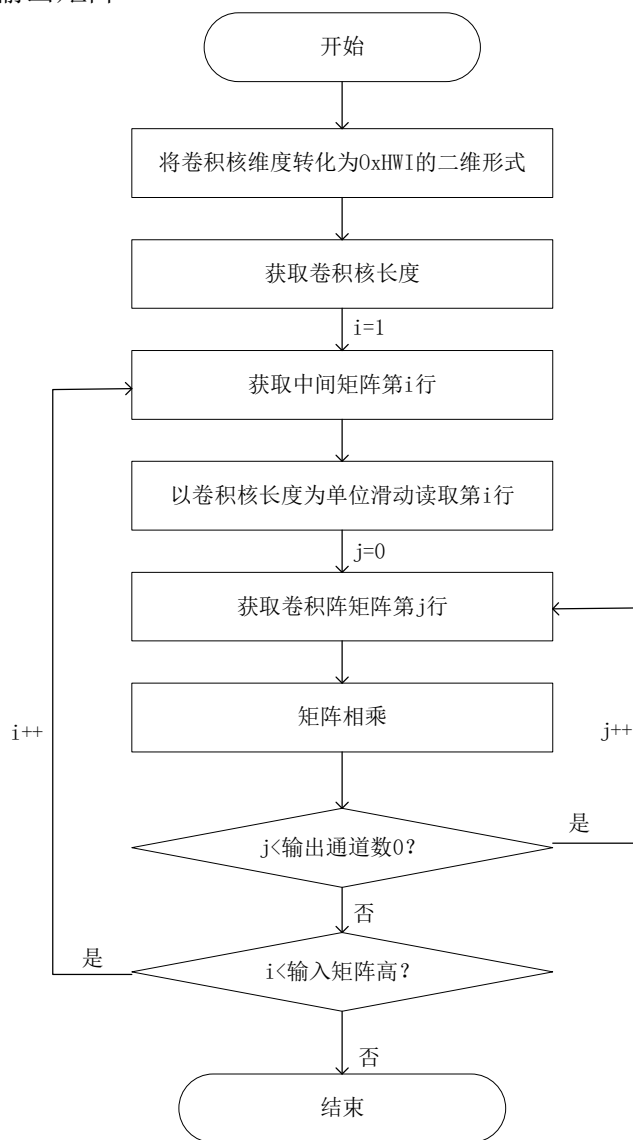


图3.9 矩阵乘法的计算特征和访存特征提取方案

3.3.2 单通道下矩阵运算方法实现

单通道矩阵运算部分如图 3.10 所示, 根据 3.2.1 节可知, 对于矩阵运算部分, 原有算法读取中间矩阵的方式比较复杂。为简化原有算法读取步骤, 加快读取速度, 本节更

改了参与矩阵运算的顺序。与修改矩阵转换过程类似，按行读取中间矩阵。将原先按矩阵的运算方式修改为按行向量进行运算，对于每一行数据，使用滑块长度对中间矩阵的第一行进行读取。滑块从左向右移动时，移动的距离为卷积核的长度。在读取完滑块内的数据时，与卷积核矩阵进行相乘，获得输出矩阵的一个元素。对中间矩阵的一行进行滑动读取计算后，获得输出矩阵的一行数据。重复读取中间矩阵的每一行并与卷积核相乘，得到完整的输出矩阵。

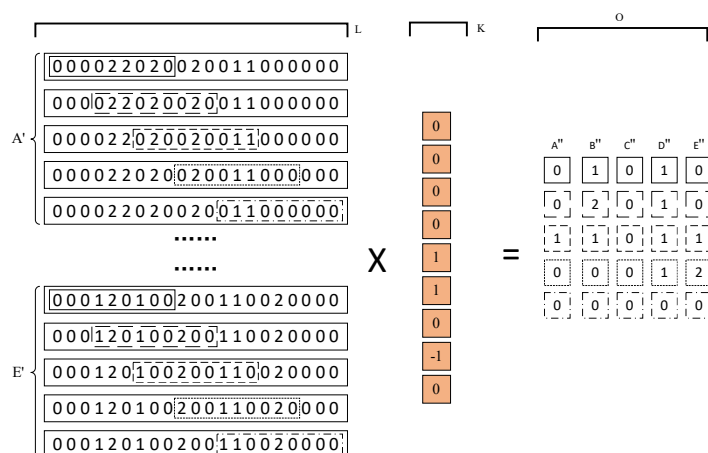


图3.10 矩阵运算计算过程

行向量 $A'=L[1,1:21]$ 是矩阵运算读取的第一部分，读取 A' 时，首先，读取向量 A' 中长度为 $KH \times KW=9$ 的数据，将读取的数据与卷积核向量 K 进行点积，得到输出矩阵 O 的值 O_{11} 。其次，取核向量 K 的长度作为滑块大小，按照步长 $KW \times IC=3$ 的距离向右滑动滑块，得到的数据分别与 K 相乘，直至滑块到达向量 A 的末尾，得到输出矩阵 O 中的 A'' 列。最后，分别对 $\{B、C、D、E\}$ 做相同处理，对应得到矩阵 O 的 $\{B''、C''、D''、E''\}$ 列。在矩阵运算过程中，参与矩阵运算的各部分相互独立，可以利用并行加速。本节对矩阵运算修改后，获取了矩阵运算优化后的计算特征，如表 3.7 所示，经过优化后，依赖距离由原来的 45 减少到 15，从而达到增加 cache 命中率的目的。

表3.7 矩阵计算的计算特征示例

节点	依赖数目	依赖距离
$O[P,0]$	18	15
$O[Q,0]$	18	12
$O[R,0]$	18	12
$O[S,0]$	18	12
$O[T,0]$	18	-

3.3.3 多通道下矩阵运算方法实现

与输入矩阵相同，参与卷积运算的卷积核矩阵同样有四个维度，分别为宽（KW）、高（KH）、输入通道数（IC）和输出通道数（OC）。通常，卷积核大小为 $KW \times KH$ ，一个输入矩阵对应一套卷积核，一套卷积核内包含 IC 个卷积核，一套卷积核可与输入矩阵进行卷积运算得到一个单通道输出矩阵。OC 套卷积核分别与输入矩阵卷积得到 OC 个输出矩阵，即多通道输出矩阵。与输入矩阵（NHWC）进行运算的卷积核矩阵通常采用 $KH \times KW \times IC \times OC$ （HWIO）作为数据布局，将卷积运算转换为矩阵乘法，利用 BLAS 库中高度优化的 `gemm` 函数进行加速。本文使用 TVM 作为优化算法的实现平台，将卷积核矩阵的数据布局修改为符合 MEC 算法访存模式的 $OC \times KH \times KW \times IC$ （OHWI）布局方式。如图 3.11 所示，依次按照输入通道数、宽和高的顺序将一套卷积核展开成行向量，分别对 OC 套卷积核进行展开，得到行高为 OC，列宽为 $IC \times KW \times KH$ 的卷积核矩阵。

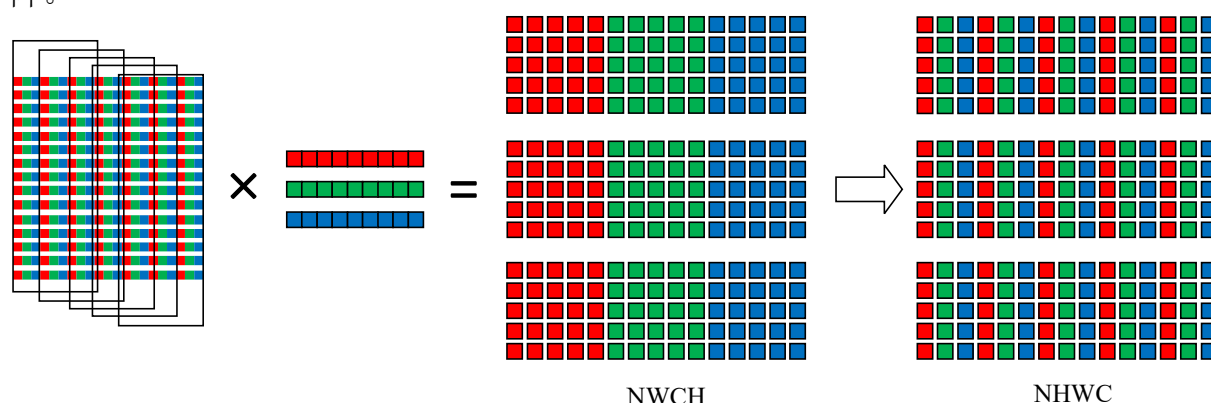


图3.11 多通道矩阵运算优化

进行卷积时，需要将中间矩阵和卷积核矩阵经过计算得到输出结果，但由于输出结果的数据布局同输入矩阵的数据布局不匹配，需要进行布局转换，在优化算法中，进行布局转换的方式为矩阵转置。矩阵转置过程如图 3.12 所示，优化算法主要对宽、高和批次三个维度进行转置，取 $N=1$ 、 $W=3$ 、 $H=3$ 、 $C=3$ ，不同颜色代表不同通道数。在转置过程中，首先，依次读取结果矩阵中各通道的第一列第一个值，排列成输出矩阵中第一行相邻的三个值，即图 3.12 中 1、2、3 号值的位置变化。其次，对各通道的第一列上的其余值进行转换，排列成输出矩阵的一行。最后，对各通道的其他列依次重复上述过程，得到完整的输出矩阵。在转置过程中可以看出，在读取结果矩阵时，顺序读取的两个值之间通常相隔的长度为 H ，由 3.2.1 节提到的数据预读取技术可知，读取数据时会把相邻数据存入 `cache` 中。但是读取结果矩阵时数据跨度大，导致 `cache` 命中率降低，转置时间增加，对优化算法的计算速度造成了一定的影响，这也是算法以后需要改进的地方。

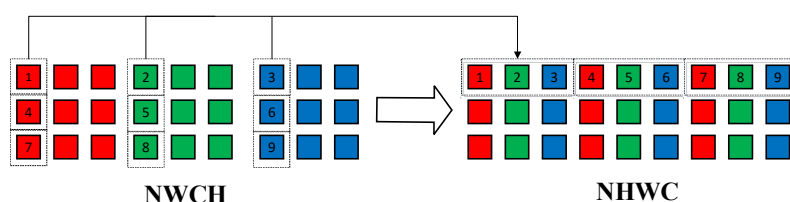


图3.12 输出矩阵转置过程

利用 TVM 内置计算函数对原有算法运算方式做出改进，具体改进步骤如下。首先，将第一个批次的中间矩阵依次与卷积核矩阵的每一行相乘，得到一个批次的三通道输出矩阵。其次，对剩余批次重复上述步骤，得到完整的结果。同时采用分批次对矩阵进行处理，可以利用 TVM 内置的并行库对卷积运算进行加速。最后，对输出结果进行整理，对运算结果进行矩阵变换。这种运算方式中，访问核矩阵的顺序为行优先，符合算法访问模式，加快读取速度。

3.4 MEC 优化算法实验结果与分析

3.4.1 实验环境与实验方案

3.4.1.1 实验环境搭建

本章使用 x86 架构 CPU，基于 TVM 编译器对 MEC 算法进行优化，通过设计多组实验验证该算法的计算性能。TVM 针对多样的硬件体系结构与模型定义，在模型定义和后端代码实现之间的转换部分进行了高度优化，使模型可以更好的部署到各个硬件平台^[49]。硬件平台 CPU 型号为 Intel Core i5-10400CPU @2.90GHz，支持 AVX2^[50] 矢量指令集，可以更好的对计算进行加速。

本文使用单个卷积层和三个多层神经网络模型对优化算法进行测试。对于单个卷积层，这里从当前流行的神经网络模型中选取了 10 个不同规模的卷积层作为优化算法的测试集，测试集规模如表 3.8 所示。分别对 MEC 算法、优化算法以及空间组合算法(spatial pack)进行测试。文献[14]中使用了空间组合算法，采用分治的思想，将输入矩阵的高和宽均匀分成若干份，分别对每一部分进行卷积运算。作为一种高效的卷积算法。空间组合算法同优化算法类似，对输入矩阵进行划分，只需在填充(padding)时增加额外的存储空间，减少了存储空间占用，同时也采用修改读取顺序的方式对卷积算法进行优化。

对于多层神经网络，采用 PyTorch 官方提供的预训练模型，包含 ResNet18、ResNet101 以及 DenseNet121 网络。分别使用 MEC 算法和优化算法作为神经网络模型的卷积层算法，对三种模型进行前向推理测试。ResNet 网络由何凯明提出，使用残差学习解决了神经网络因深度过深导致过拟合的问题。DenseNet 由刘壮提出，优点在于缓解了梯度消失问题，加强了特征传播，极大的减少了参数量^[51]。预训练模型由网络上公开的数据集

ImageNet^[52]对网络模型进行训练后得到，包含完整的模型参数^[53]，可以直接使用预训练模型进行推理测试，本文选用 ImageNet 中推理子集作为多层神经网络的测试集。

表3.8 测试集规模

测试集	输入矩阵规模 $IH \times IW \times IC$	卷积核规模 $KH \times KW \times OC(SW)$
01	$227 \times 227 \times n$	$11 \times 11 \times n, 4$
02	$231 \times 231 \times n$	$11 \times 11 \times n, 4$
03	$24 \times 24 \times n$	$5 \times 5 \times n, 1$
04	$12 \times 12 \times n$	$3 \times 3 \times n, 1$
05	$224 \times 224 \times n$	$3 \times 3 \times n, 1$
06	$112 \times 112 \times n$	$3 \times 3 \times n, 1$
07	$56 \times 56 \times n$	$3 \times 3 \times n, 1$
08	$28 \times 28 \times n$	$3 \times 3 \times n, 1$
09	$14 \times 14 \times n$	$3 \times 3 \times n, 1$
10	$7 \times 7 \times n$	$3 \times 3 \times n, 1$

3.4.1.2 实验方案设计

对于单个卷积层，采用 0-1 之间的随机浮点数作为输入矩阵和卷积核矩阵的数据。如表 3.7 所示，测试集分别采用 $n=64$ 、96、128、256 和 512 五个不同规模的通道数作为测试数据，每个测试集含有 10 个不同的测试数据，总共 50 组测试数据组成测试集，对单个卷积层进行测试。首先，获取优化算法与 MEC 算法的中间矩阵转换和中间矩阵读取的速度对比，以测试优化算法各个阶段的性能提升。其次，比较总体优化算法与 MEC 算法的性能提升、优化算法同空间组合算法的性能提升，依次验证优化算法性能提升的有效性。同时，通过获取优化算法在转置上的耗时来分析优化算法的不足。最后，使用 TVM 平台自带的优化库对 MEC 算法、优化算法和空间组合算法进行加速优化。其中，对空间组合算法中输入矩阵的划分，由 TVM 给出最佳划分方法。对三种算法进行相同批次下前向计算性能测试，分别统计在不同测试集上的计算时间，连续测试 10 次取平均值。

对于多层神经网络，本节将优化算法和 MEC 算法作为卷积层算法，以函数形式嵌入 TVM 计算流程中。同时，将针对优化算法和 MEC 算法的优化函数嵌入到 TVM 的优

化流程中。修改 TVM 的卷积选择策略，将推理所用卷积算法选择修改为优化算法和 MEC 算法。运行时，使用 TVM 加载预训练模型，从 ImageNet 测试集中取 30 张图片分别对 MEC 算法和优化算法进行推理测试，每张图片重复推理 10 次取平均值。

3.4.2 单层卷积实验结果与分析

对于 10 个测试集 50 组输入的测试，将优化算法的实验性能与 MEC 算法的性能进行对比分析，分别获取中间矩阵转换、中间矩阵读取和整个卷积过程的运行时间。同时，与空间组合算法进行对比，分析优化算法的优缺点。

3.4.2.1 小节中间矩阵读取和运算实验结果与分析

中间矩阵读取和运算时，采用运行时间作为性能对比数据，使用的公式如下。设 MEC 运行时间为 T_{mec} ，优化算法运行时间为 T_{opt} ，速度提升为 $T_{speedup}$ ，则计算公式为：

$$T_{speedup} = \frac{T_{mec} - T_{opt}}{T_{opt}} \times 100\% \quad (3.1)$$

表3.9 中间矩阵数据规模

测试集	中间矩阵规模 OW×IH KW IC	卷积核规模 OC×KW KH IC
01	55×2497*n	96×121*n
02	56×2541*n	96×121*n
03	20×120*n	256×25*n
04	10×36*n	512×9*n
05	222×672*n	64×9*n
06	110×336*n	128×9*n
07	54×168*n	64×9*n
08	26×84*n	128×9*n
09	12×42*n	256×9*n
10	5×21*n	512×9*n

如表 3.9 和图 3.13-图 3.14 所示，为了更好的对中间矩阵进行分析，本文分别获取每个测试用例中间矩阵数据规模以及两个算法的数据读取、运算的性能提升。由于数据规模不同，使用 n 代替输入通道数 IC 在表 3.9 中进行表示。由于步长的存在，把中间矩阵的规模修改成参与运算的规模，数据读取时间的获取方式为单独访问矩阵，不进行与

核函数的乘法运算。

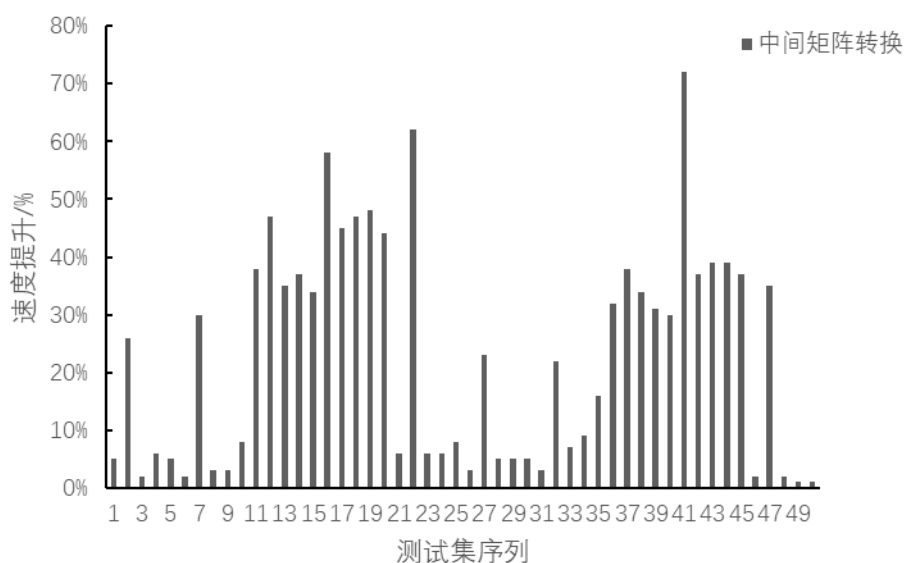


图3.13 优化算法中间矩阵转换速度提升对比

从图 3.13 中可以看出，优化算法中间矩阵转换的性能相比文献[26]提出的 MEC 算法平均提升 10%。优化算法修改了中间矩阵转换时读取输入矩阵的方式，在读取时，采用按行读取的方式代替按列读取，使得优化算法相比 MEC 算法有性能上的提升。随着通道数的增加，中间矩阵的转换速度并未随着增加，这是因为在输入矩阵读取完成后，还需要对中间矩阵进行写入操作。写入时为按列写入，通道数增加导致每列跨的数据间隔较长，增加的写入耗时。

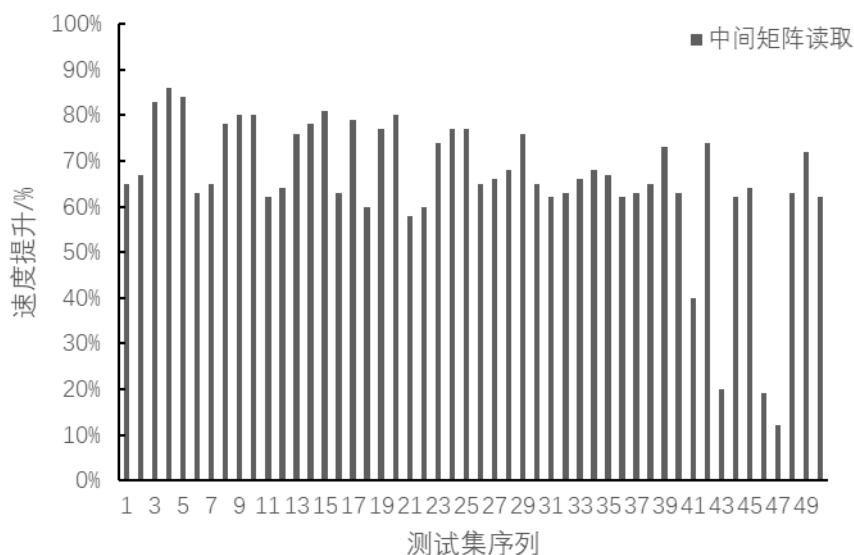


图3.14 优化算法中间矩阵读取速度提升对比

优化算法中间矩阵读取速度如图 3.14 所示，相比 MEC 算法，优化算法访问中间矩阵数据的速度明显加快，较原有算法平均提升了 60%。在读取中间矩阵时，优化算法完

全采用按行读取，且统计时只统计读取时间，减少了其他因素的干扰，使得数据更加准确。结合表 3.8 中数据规模，在处理较大规模的矩阵时，MEC 算法中一次所需处理的行数增多，根据程序访存特性可知，增加了访存时间。优化算法采用按行读取策略，充分利用访存特性，使得 cache 命中率明显提升，加快转换速度。表中 1-2 测试数据中，在中间矩阵列宽不变，行宽较小时，优化算法的访问速度提升不明显。但随着行宽的增加，优化算法的访问速度提升更加明显，最高时访问速度提升了 60%，这符合本文对优化算法的分析。

3.4.2.2 优化算法总体对比实验与分析

如图 3.15 所示，结合表 3.7 可以发现，同文献[26]相比，在输入矩阵的宽高较小且通道数较多时，优化算法都能取得 50%以上的提升。这是因为中间矩阵 L 的行高由 OW 决定，列宽大小由 $IH \times KW \times IC$ 决定。随着通道数 IC 的增加，中间矩阵的列宽也会跟着增加，通常会增加到行高 OW 的几十倍。再加上本文对卷积核矩阵数据的重新排列，使得优化算法符合程序的访存特性，增加 cache 命中率，从而降内存访问延时，提高整体的计算性能。

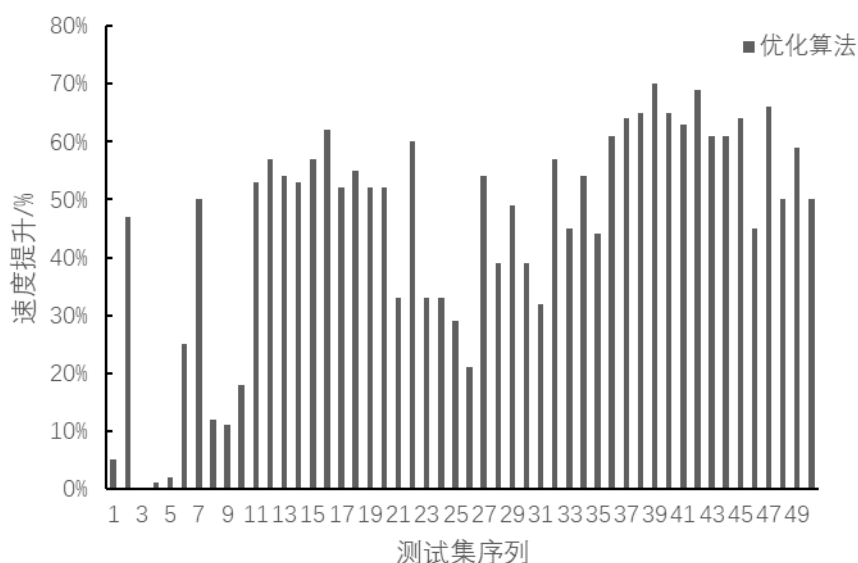


图3.15 优化算法总体卷积速度提升对比

但是，优化算法也有明显不足，如图中 1、3 和 8 所示，优化算法的速度提升不明显，甚至有些许下降。这里获取了优化算法转置所耗时间 (T_t) 同优化算法整体时间的的时间比，计算公式为：

$$T_{\text{时间占比}} = \frac{T_t}{T_{\text{opt}}} \times 100\% \quad (3.2)$$

本文在 3.3.3 小节提到，得到输出矩阵之前，需要对计算得到的结果矩阵进行转置操作，对于较大规模矩阵，需要读取的数据间隔较大，造成转置时间一定程度增加，这是速度提升不明显的主要因素。从图 3.15 和图 3.16 对比可以看出，转置时间占用较大

的测试序列速度提升不明显，转置时间占用较小的测试序列速度提升明显，呈负相关关系，证明了转置过程一定程度上影响了优化算法的运算速度。

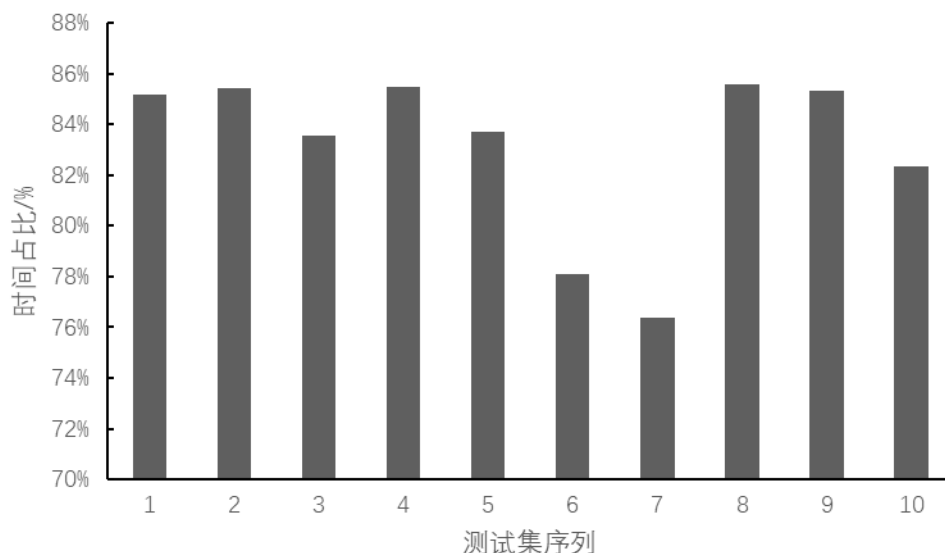


图3.16 转置时间占比

将优化算法同 MEC 算法进行对比之后，对文献[31]提出的空间组合算法和优化算法的运行时间进行了对比。其中 $T_{spatial}$ 表示空间组合算法所用时间， T_{opt} 为优化算法所用时间，计算公式如下：

$$T_{时间对比} = \frac{T_{spatial} - T_{opt}}{T_{spatial}} \times 100\% \quad (3.3)$$

从对比结果可以看出，优化算法整体上的运算速度明显优于空间组合算法，除测试序列 6 外，其余均有 30%以上的速度提升。其中测试序列 5-6、8 和 9 的速度提升都在 40%以上，结合表 3.7 可以看出，这些测试序列采用的高 H 和宽 W 都较小，而通道数 C 较大。根据第三章优化算法得知，此种情况下，中间矩阵的宽会增加，更有利于增加优化算法的 cache 命中率，使得优化算法速度提升明显。

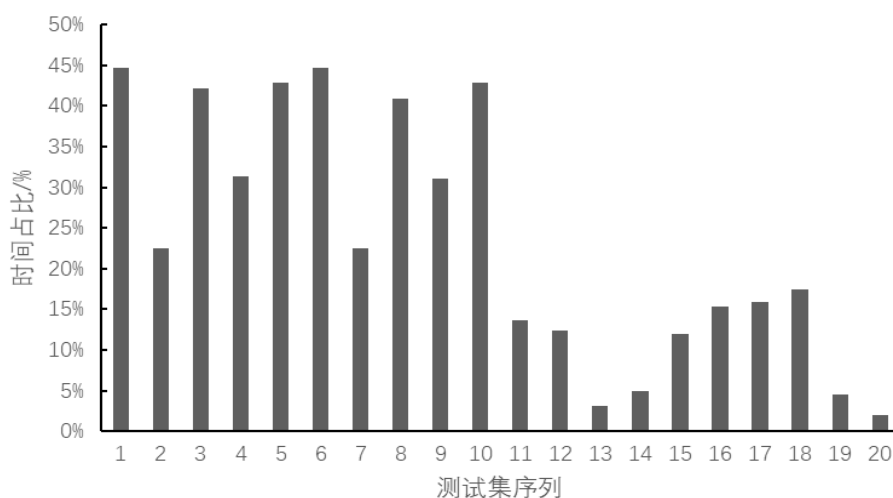


图3.17 优化算法卷积同空间组合算法速度提升对比

而在 11-16 中, 优化算法速度提升略低, 这是因为随着矩阵宽和高的增加, 空间组合算法可以发挥分治的优势, 将矩阵切分成合适的数量进行卷积。最后, 测试序列 13 和 20 出现了优化算法提升不明显的现象, 这是因为优化算法不适合处理宽高较大, 通道数较少的矩阵, 同时, 空间组合算法适合处理此种情况。因此, 从实验数据可以看出, 优化算法整体上优于空间组合算法, 但是随着通道数的不断增加, 空间组合算法的推理速度逐渐加快, 导致优化算法的推理速度相较于空间组合算法速度增加不明显。

同时, 本节获取了 10 个不同测试集在优化算法和空间组合算法的内存占用情况, 从表 3.10 可以看出, 空间组合算法由于存在数据的复制, 导致占用的内存增加。优化算法在切分矩阵时需要对矩阵的数据进行复制, 以满足矩阵运算的条件。但是在批次数和通道数较低的情况下, 需要复制的数据较少, 因而占用内存较低, 不会出现占用内存过多的情况。测试集 4 和测试集 8 采用 256 和 512 批次数, 出现的批次数较多, 导致空间组合算法填充的数据多于优化算法复制的数据, 因此出现内存占用较多的情况。

表3.10 内存占比

测试集	优化算法 (MB)	Spatial 算法 (MB)
01	0.003	9.519
02	0.007	0.015
03	0.008	2.625
04	0.048	20.347
05	0.003	0.015
06	0.011	0.210
07	8.918	78.210
08	0.004	0.679
09	0.004	8.675
10	0.012	0.453

3.4.3 多层卷积网络实验结果与分析

在 TVM 平台中实现 MEC 卷积算法和优化算法, 分别替换卷积层中的卷积算法为 MEC 和优化算法, 使用 ResNet18、ResNet101 和 DenseNet121 三种神经网络模型作为推理测试模型的实验结果如图 3.18 所示。这里设使用 MEC 算法作为卷积的神经网络运算时间为 T_{mecnn} , 使用优化算法作为卷积的神经网络运算时间为 T_{optnn} , 则速度提升 T_{sun} 计

算公式为:

$$T_{\text{sun}} = \frac{T_{\text{mecnn}} - T_{\text{optnn}}}{T_{\text{mecnn}}} \times 100\% \quad (3.4)$$

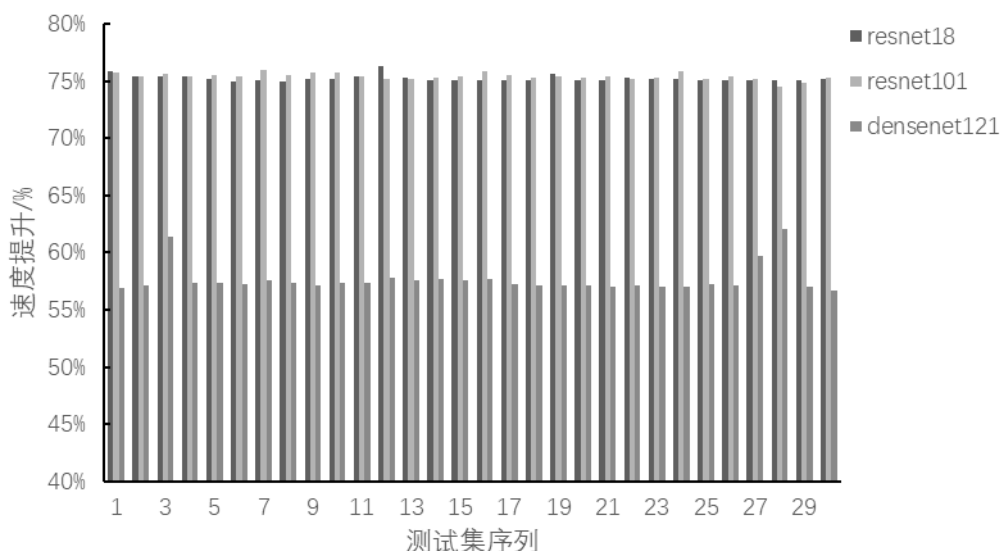


图3.18 不同网络模型速度提升

本次实验分别获取不同网络模型下两种算法的时间比,如图 3.18 所示,在 ResNet18 网络中,相比 MEC 算法,优化算法的速度提升了 57%以上。在更深层的 ResNet101 网络中,优化算法的性能提升平均在 75%以上。两者对比可以发现,随着网络层数加深,速度提升更明显,这是因为 ResNet101 中含有更多卷积层,每个卷积层都得到不同程度的加速,使得优化算法对性能提升更加明显。最后,在近几年新兴的神经网络 DenseNet 中,优化算法获得了 75%的速度提升。由此可以看出,相较于使用 MEC 作为卷积算法的神经网络模型,使用优化算法作为卷积算法的网络模型有更加良好的性能提升。

3.5 本章小结

MEC 卷积算法是处理内存占用过高的选择之一,但是,优化 MEC 算法的运算速度同样重要。因此,本文提出一种针对 MEC 的卷积优化算法。首先,对输入矩阵和中间矩阵的处理方式进行优化,将原有的访问方式修改为行优先访问,使得访问顺序符合程序访存模式。然后,本文更改了卷积核的数据布局,使用 TVM 封装的计算函数对卷积的计算方式进行优化,修改卷积计算方式为行乘以行。此外,本文还利用 TVM 平台的优化方法对算法进行优化加速,达到提高卷积计算性能的目的。实验结果表明,优化算法相较于 MEC 算法,获得不同程度的性能加速。在测试不同规模的单层卷积时获得了平均 50%的性能提升。此外,本文将优化算法和 MEC 算法集成到 TVM 平台中,使用 ResNet18、ResNet101 和 DenseNet121 三个神经网络模型作为测试模型,分别获得 57%、75%和 75%的速度提升。同时,本文将优化算法同空间组合算法进行对比,获得了最高

45%的速度提升，又进一步验证本文提出的优化算法的有效性。

4 基于分支特征的推理计算图划分方法研究

针对 TVM 中的流水线运行机制中子图划分方法不灵活的问题，本章提出了一种基于分支特征的推理计算图划分方法。该方法不同于需要手动输入节点进行划分的默认方法，基于第二章的分支特征自动将计算图划分为不同子图，并对子图进行配置，以加快 CNN 算法运行。

4.1 推理计算图划分方案设计

在 TVM 中，计算图划分为两种方式。第一种是基于后序遍历的子图划分（PartitionGraph）方法，该方法的目的是为了充分利用各种神经网络加速器，将计算图进行子图标注，再将其使用子图划分函数进行划分。该方法的优点是将不同特点的子图手动分配到加速器中，实现对特定子图的优化加速，缺点是当运行多个计算图时，计算图之间的运行方式仍为串行，无法充分利用计算资源。为此，TVM 社区中提出了另一种机制——流水线（Pipeline）机制，该方法通过手动划分，将计算图划分为不同子图，并分配到不同的设备或者不同核之间。在多个计算图运行时，可以实现多个计算图间的并行，充分利用计算资源。但是，流水线默认的划分方法需要使用者观察计算图以手动进行划分，导致使用难度增大，且划分效果不理想，达不到相应的加速效果。因此，本章基于流水线机制，结合第二章分支特征，提出了一种自动的计算图划分方法。如图 4.1 所示，该方法首先利用获取的分支信息对计算图进行自动划分，其次对不同的子图进行信息配置，最后将子图放置到设备运行。

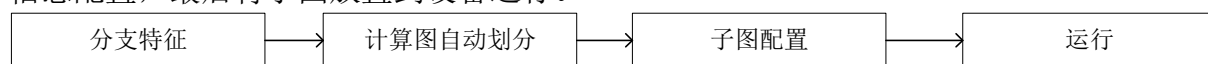


图4.1 基于分支特征的计算图划分总体过程

4.1.1 计算图分支自动划分方案

基于分支特征信息，计算图分支自动划分主要包括分支查找与划分、子图信息记录和重写两部分。分支查找与划分主要利用 2.3 节设计的分支信息搜索算法获取分支信息，遍历计算图，将查找到的分支进行划分。子图信息记录和重写主要对划分后的子图进行重写和依赖统计，便于后续子图信息配置。

计算图分支查找与划分如图 4.2 所示，主要使用分支特征提取算法针对各个神经网络进行分支信息的提取，获取所需的分支信息。利用分支信息对计算图进行前向遍历，查找符合分支信息的算子，将包含算子的分支存储进数组。

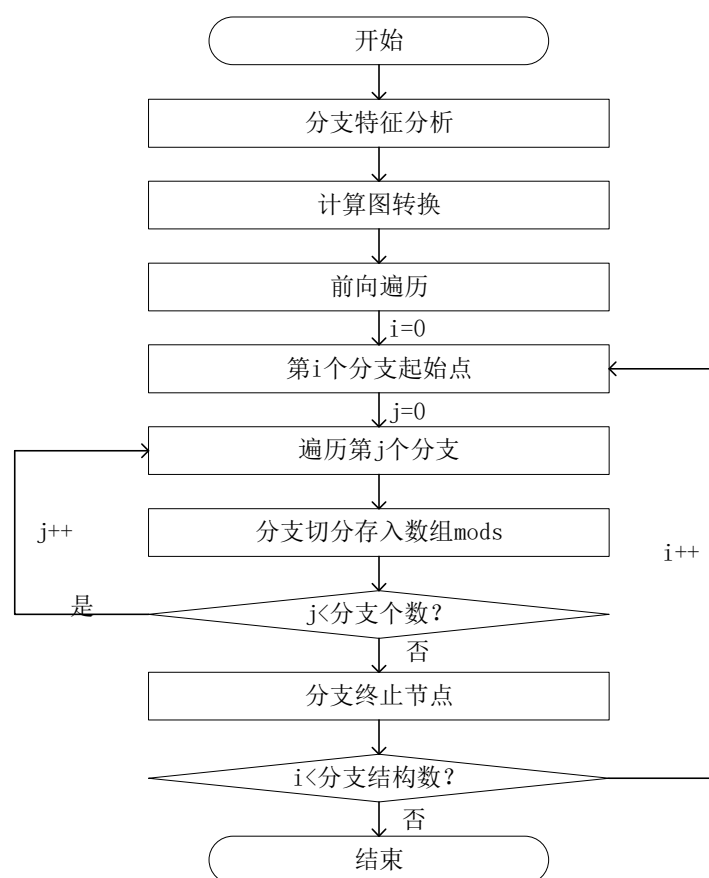


图4.2 分支查找与划分

数组作为存放信息的主要结构，包含分支子图的信息，以及各类输入输出信息。详细的步骤如下，第一，使用分支信息搜索算法对计算图进行分支信息的获取，得到分支开始和结束节点。第二，使用 TVM 中的 `ToANormal` 函数，将计算图转换为包含 `Let` 节点的形式，便于从输入开始进行前向遍历至计算图末尾。第三，根据分支信息查找分支开始节点，当搜索到时，按顺序对各个分支进行遍历，并将各个分支作为独立的函数（子图）切分出来，存入 `mods` 数组中。最后，当搜索到结束节点时，继续进行前向遍历，若遍历至计算图末尾则停止遍历，否则继续搜索下一个分支开始节点。详细实现将在 4.2.1 节进行阐述。

子图记录及重写是对子图划分后的进一步处理，便于后续子图信息配置。如图 4.3 所示将原有计算图切分为各个子图后，子图间存在数据依赖，主要包括各个子图的输入输出，以及对应算子的名称。由于在切分子图时，无法同时对各个子图进行输入输出记录和算子名称的修改，需要重写定义 TVM 中特有的遍历函数对子图进行遍历记录。主要步骤如下，首先，对 `mods` 数组中的每个计算图进行遍历。其次，遍历时，先遍历计算图的参数，判断是否为输入算子，若是，则将算子进行重写，并将其记录到输入算子的数组 `t` 中。最后，遍历计算图，获取分支的开始和结束节点，将获取的信息存入数组

中。子图重写解决了子图间输入输出对应算子名称不匹配的问题，便于后续子图信息的自动配置。

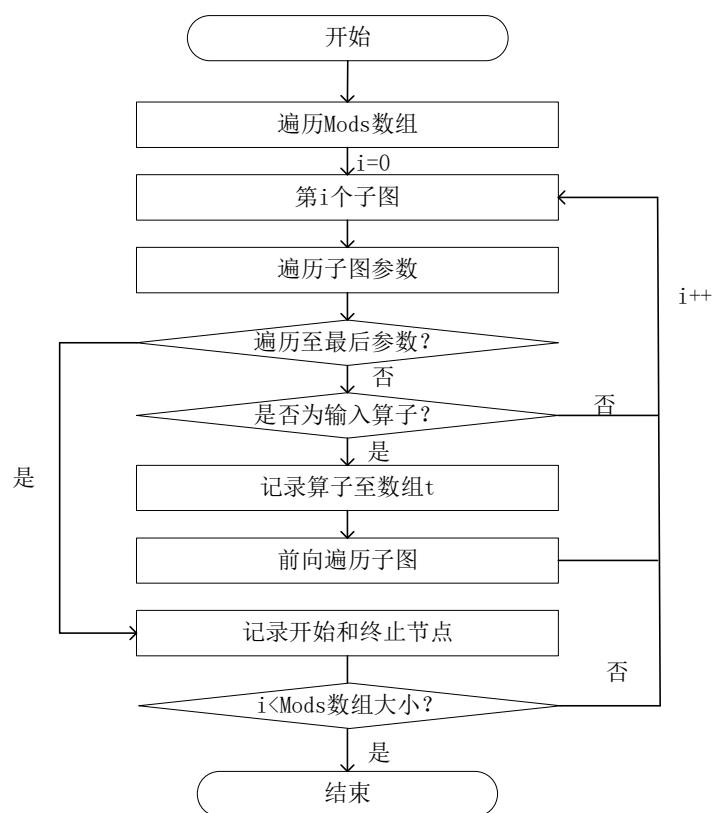


图4.3 子图记录及重写

4.1.2 子图信息自动配置方案

根据计算图划分方案完成子图划分后，还需对计算图进行依赖关系配置以明确各个子图间的依赖关系，此外，还需对各个子图进行不同参数的配置。由于原先子图配置过程为手动，在子图数量较多时配置过程繁琐，因此，本节设计并实现了子图自动配算法。子图信息配置的原则有如下几点。1) 流水线提供的计算图依赖信息配置需要手动配置，过程繁琐，应根据不同子图的配置自动完成依赖信息的配置。2) 对于子图划分前所使用的各项参数，需根据不同子图的特点进行对应拆分和配置。

如图 4.4 所示，依赖关系配置涉及子图间的输入输出配置，包括首个子图输入输出配置、中间子图输入输出配置及末尾子图输入输出配置三部分，三部分均由基于子图依赖信息设计的自动配置算法进行自动配置。自动配置时，首个子图作为原计算图中的开始部分，承担着对输入图片进行处理的作用，因此需要单独配置，自动读取模型输入同首个子图的输入建立连接。中间子图配置包括输入节点的子图来源和输出节点的子图去向，使用数组内的依赖信息对各个子图的输入输出进行对应配置。读取各个子图中输入

输出的关系，将不同子图间的输入输出进行自动识别和绑定。末尾子图作为原有子图的输出部分，需要特殊的定义，将该输出作为整个计算图的输出，将末尾子图的输出同模型的输出建立连接，用于神经网络模型的推理结果输出。

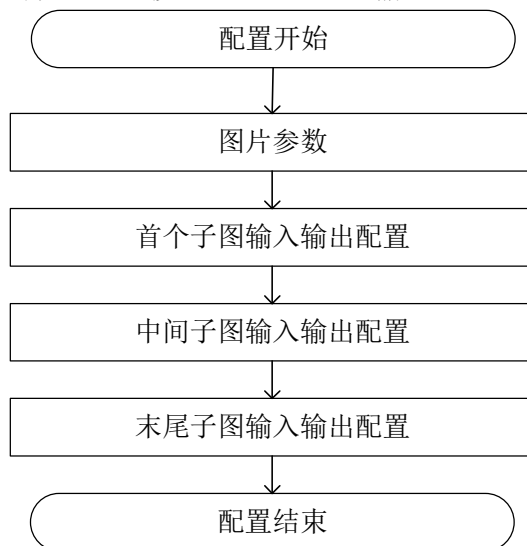


图4.4 子图依赖关系配置

子图参数自动配置包括各个子图的输入参数自动配置，以及设备参数自动设置。如图 4.5 所示，输入参数设置主要将拆分前计算图参数按照拆分后的各个子图进行对应输入参数绑定，设备参数设置主要包括使用的不同设备，后端和核心数。首先，对于输入参数匹配，原有模型中包含了所有算子的参数数据。在划分成子图后，各类算子也被划分到子图中，但是相应的参数未被划分。因此，制定了参数自动匹配算法，将各个子图的参数在原有计算图参数中进行查找，查找完成后对照各个子图进行参数分配。子图依赖配置将在 4.3.1 节进行详细阐述，子图参数配置将在 4.3.2 小节中进行详细阐述。

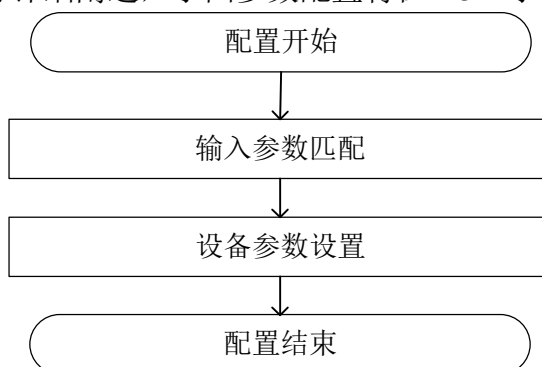


图4.5 子图参数配置

4.2 面向推理计算图的划分方法设计

4.2.1 计算图分支自动划分方法设计

本节分别对分支特征提取、计算图形式和自动划分进行设计，实现计算图的自动划分。

4.2.1.1 分支特征提取

在原有流水线机制上，采用手动方法获取分支特征，并将节点标号写入切割数组中，交由流水线切分函数进行节点识别和切割。使得切分效率比较低，且过程相对繁琐。手动获取分支的效率主要与神经网络本身的结构复杂性和分支的个数决定。如表 4.1 所示当网络层数较浅时，如 Squeezenet 等复杂度较低的网络，在进行划分时，需要了解的节点数较少，可以较快的对算法进行划分。但是，当面对 Inception 网络时，网络层数较深，含有几百个节点，需要开发人员对整个结构进行熟悉，然后依照内部不同结构的特性进行划分。此外，分支个数也是降低手动划分效率的因素之一。例如，当分支中出现嵌套分支时，开发者需要仔细阅读嵌套部分代码，使得划分后的子图具备合理性，有出现划分后子图无法运行的概率，本文提出的自动划分方法可以对这两个问题进行有效解决，从而减少了开发人员的配置时间，同时对算法的速度有部分提升。

表4.1 手动划分效率影响因素

影响因素	浅层网络（squeezenet）	深层网络（inception）
结构复杂性	结构简单，易于划分	结构复杂，需要对网络较为熟悉
分支数	分支个数少且分支结构简单	分支个数多且存在嵌套分支

为对计算图进行有效划分，本节设计了分支特征提取算法，为计算图划分提供支撑。根据 2.2 节提出的分支特征信息提取算法，针对不同计算图获取对应分支信息和节点唯一标识。如表 4.2 所示，计算图利用许多重复操作来完成计算，若使用节点名称来查找某个特定算子，会出现重复，不便于某个特异性节点的查找。因此，在进行分支特征提取时，除记录分支类型、节点名称及参数等信息外，还需记录特异性标识 Relay.Id，该 ID 在整个计算图中唯一标识节点，能够使用该标识查找需要的节点。

表4.2 分支特征示例

类型	节点	特异性标识
分支开始	MaxPool	Relay.Id(0x562abf6fcfc0)
分支结束	Concatenate	Relay.Id(0x562ac06fb410)

因此，编写分支提取算法进行分支特征提取。首先，通过识别算子的参数中算子类

型的参数数量判断是否为分支结束节点，进行节点的名称和 Id 的记录。之后，对分支进行后序遍历，使用数组将各个分支进行保存，查找公共节点，此公共节点即为分支开始节点。将开始节点的信息进行记录，存储到数组中。

4.2.1.2 计算图转换

计算图作为描述神经网络流程的方式，在 TVM 中默认有两种表达方式，一种为 GraphNormal，另一种为 ANormal。GraphNormal 作为在 TVM 中常用的计算图形式，运行时更加便捷高效。在进行节点查找时，GraphNormal 只能使用从后往前的后序遍历模式，进行查找时能够快速的找到特定节点。但是，进行子图划分时，从前往后的顺序划分和创建子图更加高效，后序遍历在修改子图时步骤繁琐，不易实现。因此，本章节使用 ANormal 计算图形式对计算图进行前向遍历和划分。并且，该形式将每个节点进行 Let 绑定，使得每个节点都有特定的编号，不易造成传统计算图的歧义。本节将计算图转换为 ANormal 模式，用以加快查找和子图创建，此外，还使用了推理优化等措施对计算图进行基本优化。

计算图转换主要是对算子进行绑定，便于后续对计算图进行从前向后遍历。如表 4.3 所示，为计算图转换示例。在对计算图进行转换过程中，根据 ANormal 计算图中 Body、Value 和 Var 属性，将算子绑定过程分为三类。第一类是对于 Var，读取原有计算图中算子的名称进行绑定。第二类是对于 Value，读取算子包含的参数和类型，打包并赋值给 Value。第三类是对于 Body，Body 包含该算子节点的后继 Let 节点信息，因此，在算子赋值过程中，需要保存后继节点信息，赋值给 Body 属性。

表4.3 计算图转换示例

转换前	转换后
<code>%1 = nn.bias_add(%input, %classifier.l.bias);</code>	<code>let %x1: Tensor[(1, 1000, 13, 13), float32] = nn.bias_add(%x, %classifier.l.bias) /* ty=Tensor[(1, 1000, 13, 13), float32] */;</code>
<code>%2= nn.relu(%3);</code>	<code>let %x2: Tensor[(1, 1000, 13, 13), float32] = nn.relu(%x1) /* ty=Tensor[(1, 1000, 13, 13), float32] */;</code>

4.2.1.3 计算图自动划分

在转换计算图的表示形式后，需要对计算图进行节点的自动查找与划分。输入到子图自动划分函数的参数有三个，分别为转换后的计算图 Fun，子图及配置信息 Pipeline_mods 和节点标识 Another_vid。其中，Pipeline_mods 为二维数组，用于存放各个阶段生成的子图及其配置信息，Another_vid 用于存储当前遍历节点的唯一标识。

自动划分作为子图划分设计的最后一步，主要分为两部分，第一部分为分支查找，使用查找算法对计算图进行递归查找，使用获取到的分支信息，对分支节点进行遍历查找。第二部分为分支划分及信息存储，将符合条件的节点依次放入子图中，进行子图的处理和构建。

在进行计算图遍历时，采用递归方式对图进行遍历，递归传递的参数为函数和节点两种。因此，需要对两种情况分别进行判断。首先，需要判断输入的计算图是否为函数。若是，则将计算图进行进一步拆解，拆解成参数 **Var**（计算图参数），函数体 **Node**（计算图节点）等结构。并进入下次循环，否则表明正在进行节点遍历。其次，检查输入参数 **Node** 是否为 **Let** 节点，若是，则记录当前的值 **Value**，进入下一步，否则表明遍历至计算图末尾。

进行节点查找时，主要对 **Let** 节点各个部分进行遍历。如表 4.4 所示，**Let** 节点包含三部分，分别为 **Body**、**Var** 和 **Value**。**var** 表示当前节点参数列表，含有特异性标识 **Relay.vid**。**Body** 表示当前节点的后继节点信息，根据 **Body** 的前后关联可实现计算图的从前向后遍历。**value** 表示 **var** 对应的值，一般为 **CallNode** 形式或 **Relay.Var** 形式，前者含有当前节点的各种信息。

表4.4 Let 节点示例

Var	Value	Body
Var(x, ty=TensorType([1, 64, 111, 111], float32))	CallNode(Op(nn.relu), [Var(x, ty=TensorType([1, 64, 111, 111], float32))])	LetNode(Var(x, ty=TensorType([1, 64, 55, 55], float32)), CallNode(Op(nn.max_pool2d)

在 **Let** 节点的三部分中，**Var** 和 **Value** 保存当前节点的名称和值。因此，首先对 **Var** 和 **Value** 进行判断。判断当前值 **value** 是否为 **Call** 节点，若是则进一步判断当前节点的 **Body** 是否为 **Let** 形式以及 **Body.value**（即当前节点的后继节点 **Value** 值）。之后，检查当前节点的 **Vid** 是否为特征信息中分支开始节点的 **Vid** 值，若是，则进一步处理，否则，将节点打包成 **Let** 形式，进入下一次递归查找。进一步处理时，获取当前 **Let** 节点的 **Vid** 值，与当前节点 **value** 中 **CallNode** 参数的 **vid** 进行对比，若不相等，则表明检测到分支，对分支开始节点收集。

在 **TVM** 中，**LetNode** 形式的计算图主要以前序遍历形式存储计算图，除一个分支前驱和最后一个分支后继外，分支间互为前驱和后继。在 **CallNode** 形式中，节点使用后序遍历的形式存储，各个分支均有共同的前驱和后继，即分支的开始和结束节点。利用 **Let** 节点和 **Call** 节点的差异性，能够判断除第一个分支外的其他分支，并进行子图节点

收集。查找到分支时,进行分支结构中第一个分支的划分。在其他分支查找时,获取分支开始节点的 Vid,将其作为搜索条件对第一个分支进行查找,查找完成后进行子图搜集。一个分支结构划分完成后,将划分后的子图合集转换为 GraphNormal 形式,按序存入 Pipeline_mods 数组中。最后,重复递归过程,完成对计算图的遍历查找。

4.2.2 子图构建方法设计

根据 4.1.1 节中子图记录及重写的方案设计指导下,设计子图构建方法,主要包括三部分,子图组建、子图信息记录和子图重写。子图组建是将 4.2.1 节获取的 Pipeline_mods 数组中各个子图节点集合进行遍历,根据节点间的依赖关系构建计算图。子图信息记录是对各个子图间的依赖节点进行记录,并写入 Pipeline_mods 中,便于后续子图配置。子图重写是把计算图中各个参数和算子进行重写,避免运行时出现歧义。本节将在后续详细描述各个过程。

4.2.2.1 子图构建

在进行子图查找后, Pipeline_mods 中含有多个子图集合,每个子图集合中含有的节点相互依赖关系为单向,即每个节点只有一个前驱和一个后继,无双向依赖分支存在。因此,可以采用如图 4.6 所示的子图组建流程来构建子图。

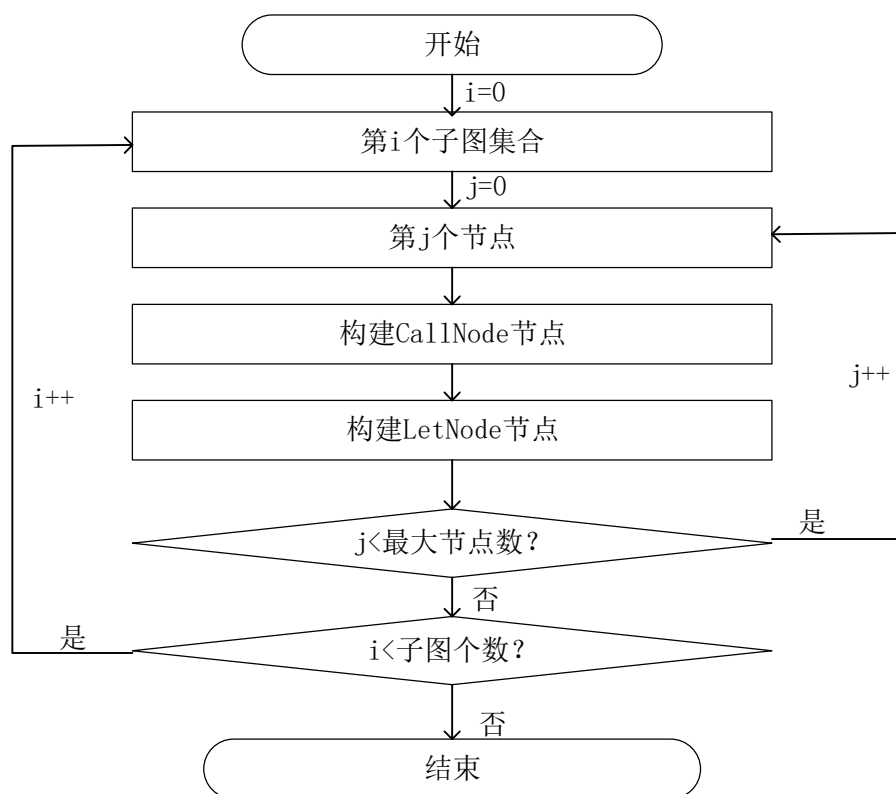


图4.6 子图组建流程图

进行子图构建时,首先,依次提取数组中的各个子图集合,子图集合中包含遍历时存储的各个节点信息,这些节点信息包含节点名称、节点参数和节点类型等信息。将这些信息对应填充到 CallNode 结构体中对应参数,完成 CallNode 节点的建立。其次,为了更高效的对计算图进行遍历,采用 LetNode 形式组建计算图,LetNode 节点主要包含 Var、Value 和 Body 三部分。其中 value 值对应 CallNode 节点,因此,将当前节点的 CallNode 信息存储到 Value 中。Var 使用节点名称对其赋值,Body 包含节点的后继节点,因此,从数组中取出当前已构造好的 LetNode 节点,将其赋值给 Body,完成 LetNode 节点的构建。最后,对每个节点进行 LetNode 节点建立及其依赖关系配置,得到完整的子图,并存入 Pipeline_mods 数组中。

4.2.2.2 子图信息记录

在完成子图组建后,需对各个子图进行信息记录,主要记录各个分支间的依赖关系。如图 4.7 所示,主要分为三部分,首先,继承 ExprVisitor 类并重写其内部遍历函数,该类的功能是进行计算图遍历。重写的函数主要为 Visit_let 函数,该函数功能为遍历 Let 节点,本节使用该函数执行输出节点 Output 的记录。由于 Let 节点绑定的节点类型为 Call 和 Var 两种,前者包含计算节点,后者为参数变量,为此,需要添加判断条件判断是否为 Call 节点。计算图从前向后遍历时,会不断进入 Visit_let 函数,刷新 Output 中的变量值,直至遍历完成,得到输出节点 Output 的最终值。其次,处理输入节点,在子图中输入节点以参数形式存在,因此,需要重写 Visit_var 函数。在组建子图时,输入节点统一命名为 x,所以,检索输入参数中名称为 x 的参数,将其添加进数组 Input 中。最后,将记录的输入节点 Input 和输出节点 Output 对应各个子图存储进 Pipeline_mods 数组中。

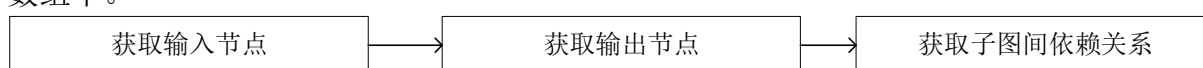


图4.7 子图间信息记录

4.2.2.3 子图重写

针对 4.2.2.2 小节信息记录时存在的输入参数重名问题,为避免运行时出现歧义,需要将输入算子的参数进行遍历重写操作。继承并重写 ExprMutator 类,此类的作用是遍历并重写计算图,本节改写了 Visit_var 函数。由于在子图划分过程中,输入参数自动修改为 x,当遇到分支结束节点时,需要对多个分支的输出进行输入运算。输入时,多个分支的输入参数名称均为 x,为避免歧义,在遍历变量 var 时,设置判断条件,判断当前节点名称是否为 x。若是,则使用 Relay.var 重新配置参数的名称和其他信息来重写此算子。为防止出现多个输入参数重名的情况,对不同输入参数采用 x_加 i 的形式进行命名,其中 i 为自增量,便于对子图中的重名参数进行命名排序。

4.3 子图划分后信息配置方法设计

4.3.1 子图间连接策略设计

对于子图间连接方法，采用流水线执行器（Pipeline_executor）中的 PipelineConfig 类进行操作。该类主要用来进行流水线信息配置，其包含的有向无环图（Directed Acyclic Graph, DAG）用以描述流水线中各个模块（子图）参数的依赖关系。在此类中，使用 Connect 函数进行配置，每个子图称为模块，各个模块组成一个流水线，子图连接也成为模块绑定。其主要分为三步，流水线输入连接至模块输入，模块输出连接到流水线输出，模块输出连接到模块输入。

进行模块绑定时，主要使用 4.2 节中包含输入信息、输出信息和子图的 Pipeline_mods 数组，其示例如表 4.5 所示。从中可以看出，模块 1 作为开始节点，无输入节点配置，需要作为流水线模块进行输入节点配置。模块 2 和模块 3 作为分支模块接收模块 0 的输入，模块 4 作为汇合模块，接收分支模块 2 和分支模块 3 的输出作为输入。由此可得，该数组详细记录了各个模块间的依赖关系，可以根据此数组对模块间依赖进行配置。

表4.5 Pipeline_Mods 示例

名称	输出节点	输入节点	含义
模块 1	0	[]	分支开始
模块 2	1	[0]	分支 1
模块 3	2	[0]	分支 2
模块 4	3	[1,2]	分支汇合

依赖配置如图 4.8 所示，根据数组中存储的各个子图前后依赖关系进行自动配置。首先，作为流水线的整体输入，需要进行单独配置，配置时，将流水线输入绑定至首个模块的输入。同时，对输入参数的名称进行修改，使其符合流水线输入名称。其次，对 Pipeline_mods 进行两次嵌套循环，外层循环负责循环各模块输出节点，内层循环负责循环各模块输入节点。

在内层循环中，判断模块 i 的输出节点是否存在于模块 j 的输入节点中，若是，则记录各自名称，使用 Connect 函数将模块 i 的输出绑定至模块 j 的输入。该部分主要读取各个子图中的输入输出操作，若经过查找对比发现该子图的输出出现在另外子图的输入中，表明该子图的输出作为另外子图的输入，因此，调用内置连接函数对子图间依赖进行连接，其他子图依赖过程同此类型。最后，单独配置流水线输出，设置搜索条件，判断子图的输出节点不存在当前任何一个子图的输入参数中，则表明该输出为流水线输

出。因此将流水线输出绑定至末尾模块的输出，完成子图的依赖关系配置。

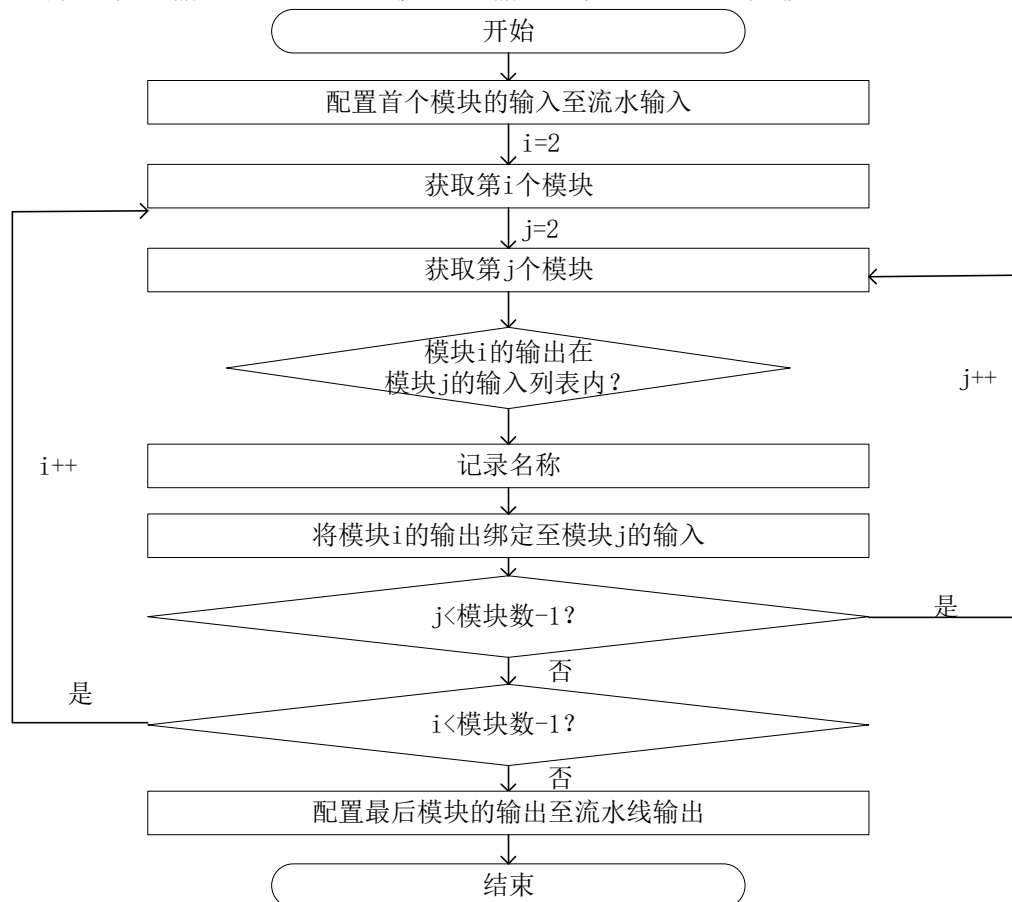


图4.8 子图依赖配置流程图

4.3.2 子图设备及参数分配策略设计

根据分支划分算法将计算图划分为各个模块后，输入参数及其他参数作为独立部分，并未参与计算图的划分，因此本小节拟对参数进行配置。参数配置步骤如下，首先，获取原有计算图的参数字典 **Params**，字典中存有参数名称和参数值的键值对，原有计算图中的各个参数名称均为唯一。其次，依次遍历每个子图模块的参数名，若该子图中含有 **Params** 中的参数，则将参数值添加进该子图的参数字典中。由于 4.2 节对划分后的计算图进行参数名称的重写，因此各子图中的参数名称都是唯一的，不会出现歧义。最后，将各个子图的输入参数依次写入进配置中。

完成输入参数配置后，需要进行其他参数的配置，主要包括子图运行目标，运行设备，所用核心数等。在配置时，需根据不同子图的特点，绑定到不同的设备后端及使用的核心数量。如表 4.6 所示，本节使用的后端工具 **Target** 为 **LLVM**，**TVM** 可根据 **Target** 自动生成 **LLVM** 代码，并部署到不同设备上运行。在选择后端设备 **Dev** 时，可以根据不同子图的特性使用后端设备，如 **CPU** 和 **GPU**。此外，使用线程接口，可以修改参与

运算的线程数，利用根据线程数的变化来查看子图划分性能与线程数间的关系。最后，将所有子图 and 对应配置存储进 PipelineConfig 中，使用构造函数对子图进行运行时构建，将不同子图（模块）间使用参数绑定和输入输出绑定对计算图进行配置并根据依赖关系判定执行顺序。

表4.6 其他参数配置

Target	Dev	Threads
Llvm	Tvm.cpu(0)	48,96
Cuda	Tvm.gpu(0)	1024

4.4 推理计算图划分方法的实验结果与分析

4.4.1 实验环境与实验方案

推理计算图划分方法实验选用的软硬件环境如下所示，使用 Intel 公司推出的 CPU 芯片 Intel Xeon Gold 6248R @3GHz 作为硬件处理器，所使用的软件环境如表 4.7 所示。

表4.7 软件环境

系统环境	工具
Ubuntu 18.04 操作系统	神经网络开发框架 PyTorch
Python3.8.6 开发环境	神经网络编译器 TVM
Visual Studio 开发环境	编译工具 LLVM

实验所用测试数据为 ImageNet 测试集中提取的三十张图片作为神经网络的输入数据，图片采用三通道图片，大小为 224×224 ，识别内容为动物图像。采用的神经网络模型为 SqueezeNet、Googlenet、Inception_v3 三个典型的分支神经网络，SqueezeNet 网络是一种轻量级的卷积神经网络，该网络在没有大幅度降低模型精度的前提下，最大程度的提高运算速度，主要用于手机等移动端。GoogLeNet 和 Inception_v3 由何凯明提出，都是在网络中添加多个分支，每个分支采用不同的卷积核，增加了并行结构的同时提高了预测的准确度。实验对象为文献[54]提出的 TVM 传统运行机制与使用本章划分方法的流水线机制，使用多个神经网络在 TVM 和流水线中进行 30 次图片推理。首先，为验证推理计算图的划分正确性，获取划分前计算图和划分后计算图是否符合划分要求。其次，获取子图的依赖信息以验证子图配置的正确性。第三，为验证不同架构下推理速度的影响，分别在 CPU 和 GPU 下进行实验验证。在 GPU 下，使用 CUDA 对计算图进行

划分和性能比较。在 CPU 下,使用不同核心数对计算图进行划分推理,以分析不同核心数下对计算图划分的性能影响,使用核心数分别为 48 和 96。将在 4.4.2 章节进行具体的计算图的划分验证与性能比较。

4.4.2 推理计算图划分结果与性能比较分析

4.4.2.1 推理计算图划分结果验证

如图 4.9 所示,本节获取了 SqueezeNet 划分前的部分计算图,Let 形式表示计算图运行时的实际顺序,因此采用 Let 形式进行展示,通过理解图 4.9 中计算图信息,可以看出。%x7 与 %x10 节点同时使用 %x6 节点作为输入, %x9 和 %x12 节点在 %x13 节点处进行合并,因此可以得出 %x0-%x6 为分支开始部分, %x7-%x10 为第一个分支, %x11-%x13 为第二个分支。 %x13-%x17 作为最后的分支合并部分。两个分支无相互依赖关系,因此可以使用分支划分算法对其进行划分。

```
let %x = nn.conv2d(%input0, %features.0.weight, strides=[2, 2], padding=[0, 0, 0, 0], chann
let %x1 = nn.bias_add(%x, %features.0.bias);
let %x2 = nn.relu(%x1);
let %x3 = nn.max_pool2d(%x2, pool_size=[3, 3], strides=[2, 2], padding=[0, 0, 0, 0], ceil_m
let %x4 = nn.conv2d(%x3, %features.3.squeeze.weight, padding=[0, 0, 0, 0], channels=16, ker
let %x5 = nn.bias_add(%x4, %features.3.squeeze.bias);
let %x6 = nn.relu(%x5);
let %x7 = nn.conv2d(%x6, %features.3.expand1x1.weight, padding=[0, 0, 0, 0], channels=64, k
let %x8 = nn.bias_add(%x7, %features.3.expand1x1.bias);
let %x9 = nn.relu(%x8);
let %x10 = nn.conv2d(%x6, %features.3.expand3x3.weight, padding=[1, 1, 1, 1], channels=64,
let %x11 = nn.bias_add(%x10, %features.3.expand3x3.bias);
let %x12 = nn.relu(%x11);
let %x13 = (%x9, %x12);
let %x14 = concatenate(%x13, axis=1);
let %x15 = nn.conv2d(%x14, %features.4.squeeze.weight, padding=[0, 0, 0, 0], channels=16, k
let %x16 = nn.bias_add(%x15, %features.4.squeeze.bias);
let %x17 = nn.relu(%x16);
```

图4.9 SqueezeNet 部分计算图信息

经过分支划分算法对 SqueezeNet 进行划分后,结果如图 4.10 所示,将计算图划分为三个函数,分别代表三个子图。各个子图由分支划分算法对计算图进行前向遍历,以相邻节点的前驱后继作为判断条件进行划分。分支开始作为独立部分划分成第一个函数,该函数以分支开始节点 `nn.relu(%5)` 作为尾节点。对开始节点所在函数划分完成后,将图 4.9 中第一个分支划分成第二个函数,通过遍历图 4.9 中节点 %x9 包含的后继 %x10 与 %x10 的前驱 %x6 不一致,表明遇到分支开始节点。之后从 %x 开始查找 %x6,找到后进行切分,切分结果如图 4.10 中第二个函数所示,可以看出,该函数包含了第一个分支的三个节点及其参数。第三个函数中包含了第二个分支以及分支结束节点,这符合分支划分算法的策略,从图 4.9 中 %x10 开始向后进行前向遍历,未查找到前后节点的前驱后继不连续的情况,因此单独划为一个函数。

```

fn (%input0: Tensor[(1, 3, 224, 224), float32], %features.0.weight: Tensor[(64, 3, 3, 3), float32], %features.3.squeeze.weight: Tensor[(16, 64, 1, 1), float32], %features.3.squeeze.bias: Tensor[(16), float32]) {
  %0 = nn.conv2d(%input0, %features.0.weight, strides=[2, 2], padding=[0, 0, 0, 0], channels=64, kernel_size=[1, 1], float32) */;
  %1 = nn.bias_add(%0, %features.0.bias) /* ty=Tensor[(1, 64, 111, 111), float32] */;
  %2 = nn.relu(%1) /* ty=Tensor[(1, 64, 111, 111), float32] */;
  %3 = nn.max_pool2d(%2, pool_size=[3, 3], strides=[2, 2], padding=[0, 0, 0, 0], ceil_mode=True) /* ty=Tensor[(1, 64, 55, 55), float32] */;
  %4 = nn.conv2d(%3, %features.3.squeeze.weight, padding=[0, 0, 0, 0], channels=16, kernel_size=[1, 1]) /* ty=Tensor[(1, 16, 55, 55), float32] */;
  %5 = nn.bias_add(%4, %features.3.squeeze.bias) /* ty=Tensor[(1, 16, 55, 55), float32] */;
  nn.relu(%5) /* ty=Tensor[(1, 16, 55, 55), float32] */
}
fn (%x: Tensor[(1, 16, 55, 55), float32], %features.3.expand1x1.weight: Tensor[(64, 16, 1, 1), float32], %features.3.expand1x1.bias: Tensor[(64), float32]) {
  %0 = nn.conv2d(%x, %features.3.expand1x1.weight, padding=[0, 0, 0, 0], channels=64, kernel_size=[1, 1]) /* ty=Tensor[(1, 64, 55, 55), float32] */;
  %1 = nn.bias_add(%0, %features.3.expand1x1.bias) /* ty=Tensor[(1, 64, 55, 55), float32] */;
  nn.relu(%1) /* ty=Tensor[(1, 64, 55, 55), float32] */
}
fn (%x: Tensor[(1, 64, 55, 55), float32], %x1: Tensor[(1, 16, 55, 55), float32], %features.3.expand3x3.weight: Tensor[(64, 16, 3, 3), float32], %features.3.expand3x3.bias: Tensor[(64), float32], %features.4.squeeze.weight: Tensor[(16, 128, 1, 1), float32], %features.4.squeeze.bias: Tensor[(16), float32]) {
  %0 = nn.conv2d(%x1, %features.3.expand3x3.weight, padding=[1, 1, 1, 1], channels=64, kernel_size=[3, 3]) /* ty=Tensor[(1, 64, 55, 55), float32] */;
  %1 = nn.bias_add(%0, %features.3.expand3x3.bias) /* ty=Tensor[(1, 64, 55, 55), float32] */;
  %2 = nn.relu(%1) /* ty=Tensor[(1, 64, 55, 55), float32] */;
  %3 = (%x, %2);
  %4 = concatenate(%3, axis=1) /* ty=Tensor[(1, 128, 55, 55), float32] */;
}

```

图4.10 SqueezeNet 计算图划分结果

在进行子图划分后，进行子图重写，重写主要通过重命名各个子图间互为输入输出的参数名称，从图 4.10 中可以看出。由于划分后各个依赖节点作为参数输入到函数中，将各个参数统一命名为%x，因此，对函数的各个参数进行遍历，查找到%x 时进行重命名，按照参数个数进行 x_加 i 的形式进行命名，i 逐渐递增。如图 4.11 所示，对于第三个函数，需要两个输入节点作为参数，因此重命名为%x_0 和%x_1。

```

fn (%x_0: Tensor[(1, 16, 55, 55), float32], %features.3.expand1x1.weight: Tensor[(64, 16, 1, 1), float32], %features.3.expand1x1.bias: Tensor[(64), float32]) {
  %0 = nn.conv2d(%x_0, %features.3.expand1x1.weight, padding=[0, 0, 0, 0], channels=64, kernel_size=[1, 1]);
  %1 = nn.bias_add(%0, %features.3.expand1x1.bias);
  nn.relu(%1)
}
fn (%x_0: Tensor[(1, 64, 55, 55), float32], %x_1: Tensor[(1, 16, 55, 55), float32], %features.3.expand3x3.weight: Tensor[(64, 16, 3, 3), float32], %features.3.expand3x3.bias: Tensor[(64), float32], %features.4.squeeze.weight: Tensor[(16, 128, 1, 1), float32], %features.4.squeeze.bias: Tensor[(16), float32]) {
  %0 = nn.conv2d(%x_1, %features.3.expand3x3.weight, padding=[1, 1, 1, 1], channels=64, kernel_size=[3, 3]);
  %1 = nn.bias_add(%0, %features.3.expand3x3.bias);
  %2 = nn.relu(%1);
  %3 = (%x_0, %2);
  %4 = concatenate(%3, axis=1);
}

```

图4.11 子图重写结果

4.4.2.2 推理计算图自动配置结果验证

计算图自动配置作为计算图划分的第二步，读取数组中存储的依赖关系对子图进行依赖配置。配置时，使用编写的自动配置算法对图 4.12 中 (a) 部分所示数组进行搜索遍历。遍历数组中每个子数组中的第一部分和第二部分，分别代表输出和输入，原有数组中存储形式为特异性标识 Relay.id，为便于展示，修改为数组。数组 00 作为首个子图，无其他子图作为输入，显示为空。数组 02 作为含有分支结束节点的子图，包含数组 00 和 01 子图中的输出作为输入。对数组进行遍历时，查找各个子数组中输出（即第一部分）是否存在于其他子数组的输入中（即第二部分）。查找到子数组 00 的输出存在于子

数组 01 和 02 的输入中，因此将 00 数组的子图输出绑定至 01 数组和 02 数组子图的输入，即图（b）中第一、二行配置。进行第二个子数组 01 中输出的查找，发现 02 数组的输入中存在 01 的输出，则将 01 数组子图的输出绑定至 02 子图的输入，即图（b）中最后一行配置。

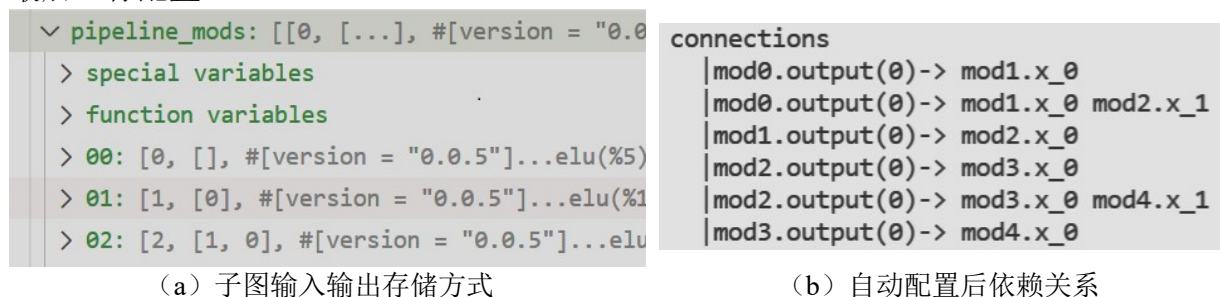


图4.12 子图依赖关系

4.4.2.3 推理计算图自动配置性能分析

为验证基于流水线的计算图划分性能，设 TVM 本身运行机制所需时间为 T_{tvm} ，流水线运行机制所用时间为 T_{pipe} ，则运行速度提升 $T_{speedup}$ 为：

$$T_{speedup} = \frac{T_{tvm} - T_{pipe}}{T_{tvm}} \times 100\% \quad (4.1)$$

使用基于流水线的分支划分方法和文献[54]提出的方法进行速度对比，如图 4.13 和图 4.14 所示。从图中可以看出，分支划分方法明显优于 TVM 传统运行方法。在 48 和 96 个核心数下，和 TVM 传统方法对比，分支划分方法的速度在 48 个核心数情况下提升稳定，平均提升 17%，其中 SqueezeNet 和 Inception 网络速度提升均在 20%以上。影响计算图划分的两个关键因素是设备空闲时间和数据移动时间，SqueezeNet 网络相较于其他两个分支网络，含有的节点数量较少，且分支数量较少，使得多个网络运行时数据移动量较少，且分支内节点数量较少，设备空闲时间短，因此有较好的性能表现。

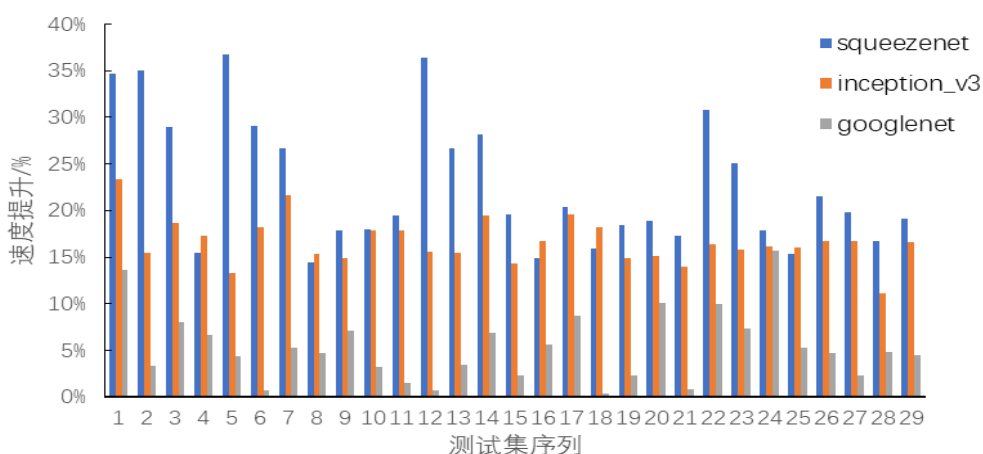


图4.13 48 核心数下性能对比

Inception 网络作为三个网络中网络规模最大的一个，特点是含有较多的节点，分支较少且每个分支中含有较多节点。在多个网络进行流水线运行时，数据移动时间和设备

空闲时间相应增加，但由于数据移动在设备内部进行，所有性能仍有较好提升，性能提升平均在 15%。GoogleNet 网络性能不稳定原因是因为其网络结构中，各个分支内部的节点个数较少，但是分支数量过多，频繁的分支划分会导致性能降低。经过观测 48 核心数下 CPU 使用率发现，每个核心数利用率均在 80% 以上，能够很好的利用设备资源进行加速。在 96 个核心数性能比较下，分支划分性能不稳定，性能平均在 10%。经实验观察发现，进行 96 核心数推理时，设备无法满载运算，且需要跨 CPU 进行，使得数据间交互占据较多时间，成为速度提升的不稳定因素，这也是下一步需要改进的方向。

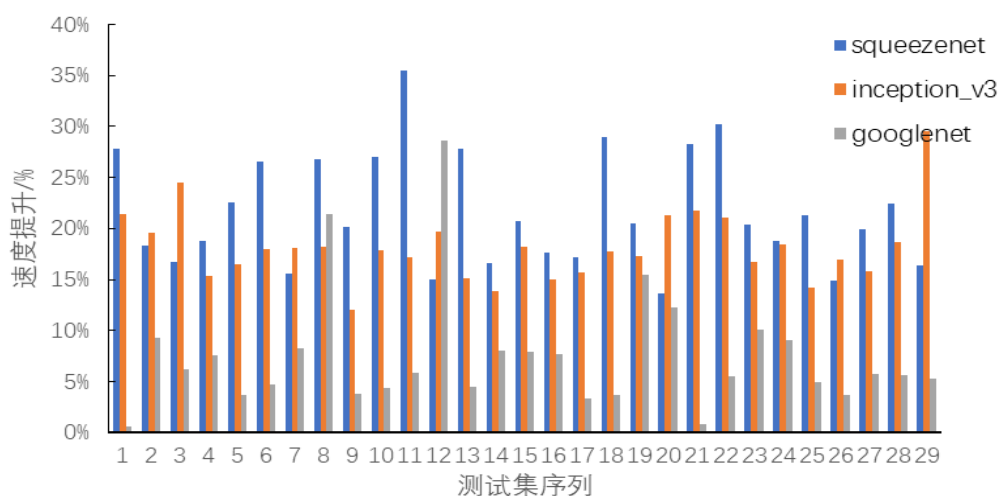


图4.14 96 核心数下性能对比

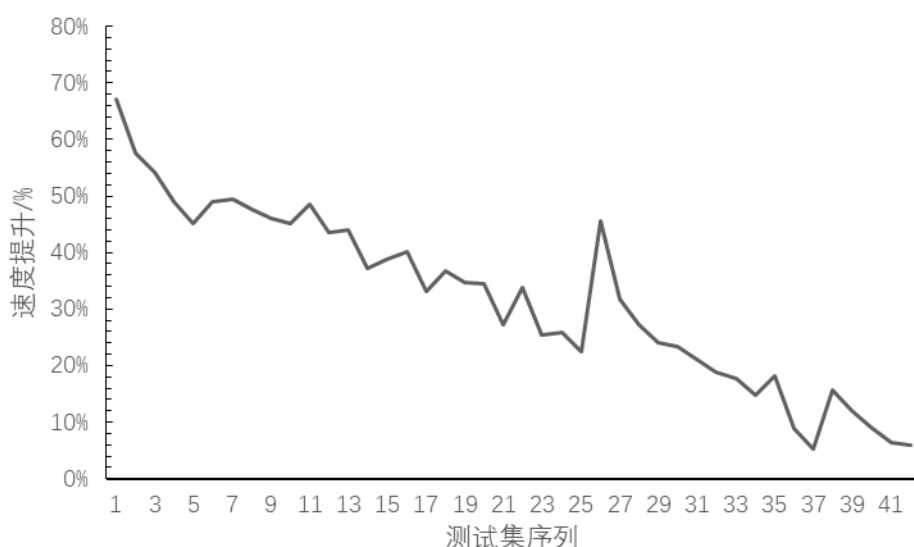


图4.15 Googlenet 网络不同核数下的速度对比

本节对 48 核心数以内的 Googlenet 网络进行不同核数下的对比，对比如图 4.15 所示。可以看出，在核心数较少时，基于分支划分的流水线方法提升较为明显，但是随着核数的增加，分支划分的提升速度不明显。这是因为 TVM 内置的加速函数可以对网络进行更好的并行加速，而流水线模式尚处在初步开发阶段，尚未支持 TVM 内置的优化

方法。同时，本节的优化方法尚未对各个子图的核数分配进行算法设计，无法做到自动将各个子图合理的安排到不同核心数下，这也是本节算法需要改进之处。

本节同时在 GPU 设备下对计算图分支划分进行性能对比，对比结果如图 4.16 所示。与 CPU 下测试规模及测试数据相同，采用三种网络分别进行三十张图片的推理。由结果可以看出，SqueezeNet 网络在大部分情况下，速度提升在 78%，多次运行时，算法相比于 TVM 在 GPU 上串行运行时设备空闲时间较少。另外，占卷积神经网络主要运算量的卷积运算更符合 GPU 的计算方式，可以在设备空闲时间内更快的进行运算，使得算法运行时间有较大提升。Inception 网络在运行时，随着神经网络数据规模的增大，需要增加并行性以利用 GPU 的计算特性，因此，TVM 本身在 GPU 上对计算图使用了子融合等优化手段，将多个算子融合为一个算子，并在算子内部进行并行操作，使得 TVM 原有机制可以加快运算，Inception 网络在 GPU 上的流水线划分推理速度也随之降低。

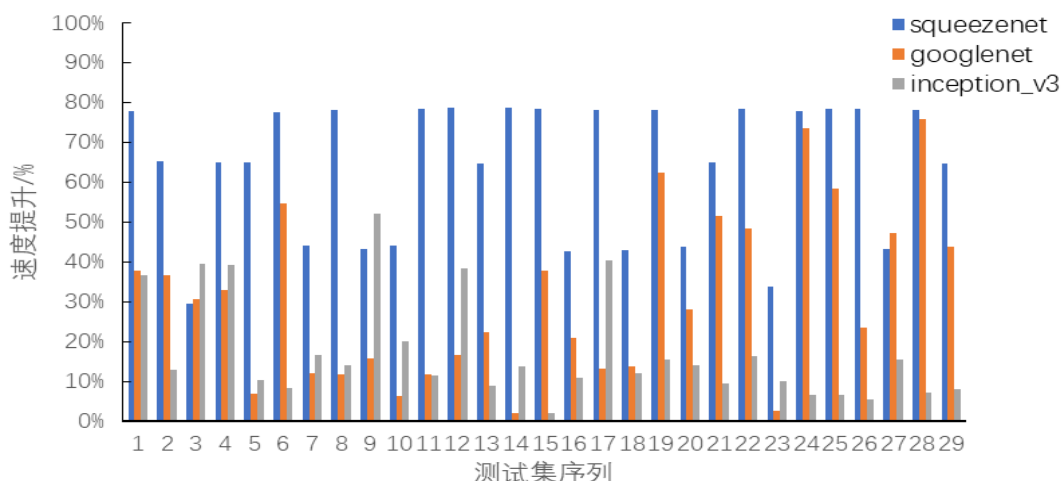


图4.16 GPU 下计算图划分性能对比

本章方法依据分支特征信息和设备进行划分，并对划分后的结果进行测试。实验结果表明，该划分方法可以有效提升分支网络的推理速度。因此，本章方法能够提升 CNN 算法的推理速度。

4.5 本章小结

由于流水线划分过程中过于依赖人工手动划分，因此，本章基于第二章分支特征信息，提出了计算图划分方法。该方法基于分支信息特征，获取分支开和结束节点，指导算法进行分支查找与划分，同时进行各个子图间的重写和参数配置，用于构建完整的计算图流水线。实验结果表明，在 48 和 96 个核心数下，使用基于分支特征的计算图划分方法，相比于 TVM 本身的推理方法，速度提升分别了 20%和 15%，表明分支划分算法能对计算图进行加速。

5 推理计算图划分的并行调度设计实现

针对 TVM 中 CNN 算法运行时使用串行方式运行 CNN 算法的模式, 未考虑节点间并行问题, 为加速 CNN 算法的运算速度。本章基于第四章计算图划分方法, 提出一种流水线并行调度方法。该方法通过构建带有并行标记的计算图, 完成对并行节点的标记。同时, 在运行时文件中增加子图并行方法, 通过修改运行时队列中出队入队操作以及线程池实现子图的并行调度。

5.1 计算图并行化设计方案

在 TVM 中, 基于计算图划分的流水线机制仅支持多个计算图的流水线并行机制, 但对于 Inception 等分支网络, 并未考虑单个计算图的分支并行。研究表明, 分支并行能够提升计算图的性能。本章基于流水线机制, 将计算图进行并行标记和划分运行, 增加计算图的并行性。基于推理计算图划分的并行调度设计方案主要分两部分, 第一部分为并行节点标记设计。在 TVM 中, CNN 算法转换为计算图后, 对应优化方法 Pass 会对计算图做出多种优化。其中包括对计算图的分块表示, 将计算图的不同部分打包成不同函数, 指导程序在运行时对不同函数进行统一处理。并行节点的标记同这个过程类似, 需要对分支节点进行打包, 并标记为并行函数。标记完成后交由第二部分设计的运行时 (Runtime) 进行并行调度运行, 在并行调度时, 需要修改算法运行时代码, 增加对并行标记的识别, 创建基于并行的运行时算法, 设计对应的线程池, 增加对不同线程间的数据交互设计, 完成并行调度的实现。

5.1.1 并行节点标记方案设计

并行标记方法通过构建支配树优化查找效率, 使用并行标记方法实现并行计算图的构建。并行节点标记方案主要分为后序支配树建立和并行计算图构建两部分, 其中, 后序支配树时为了便于分支节点标记所重新建立的, 该支配树基于最近公共祖先 (Least Common Ancestor, LCA)^[60]。后序支配树建立主要步骤如图 5.1 所示, 并在 5.2.1 节中详细阐述。首先是有向无环图的建立, 主要通过遍历计算图对有向无环图进行建立, 使用遍历各个算子的关键依赖信息, 如前驱、后继节点。虽然计算图中包含算子间的依赖关系, 但是其包含的节点信息过多, 需要对节点信息进行筛选, 以获取数据流向。获取数据流向后, 对每个算子添加数据流向的标记, 以构建有向无环图。同时, 保存节点的后序遍历顺序, 便于获取每个节点后序依赖节点。

后序支配树建立作为第二步，主要通过对各个遍历函数的重写进行后序支配树的建立。使用当前节点和存储节点拓扑顺序的拓扑图进行后序支配树的构建，拓扑图通过数据流图中各个节点的遍历获取，同时利用数据流图对当前节点进行重写更新。在后序支配树中，本节重写了用于搜索最近公共祖先的算法，使其能够对并行节点进行查找，便于分支节点的标记。

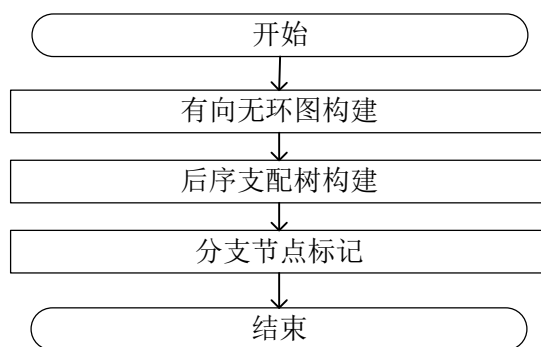


图5.1 后序支配树构建方案

并行标记方案的第二部分为并行计算图构建，构建过程主要分为三步。如图 5.2 所示，首先，检查分支节点是否为需要的节点，主要判断主干分支和其他分支。为了便于将节点进行打包，创建数组并初始化，用于存储分支。判断分支时，使用分支结束节点中存在多个算子参数的形式对分支分别进行查找。对于分支开始节点，使用重写的最近公共祖先查找各个分支的开始节，查找完成后存入数组中。第二步，对数组中的分支节点进行提取，并将节点进行打包，生成组（Group），在添加对应参数生成函数，使用分支内部算子作为函数体，算子的输入参数作为函数的整体输入参数，函数名为带有并行标记的名称。

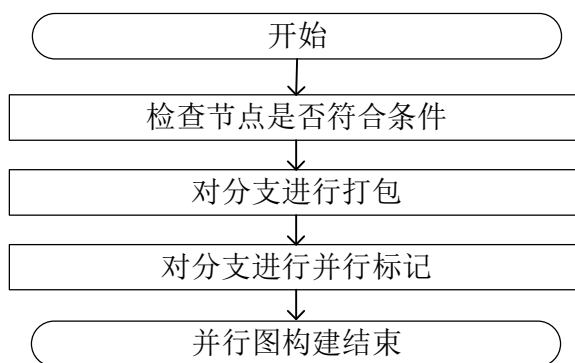


图5.2 并行计算图构建方案

在并行节点标记时，应遵循如下几个原则：第一，在进行节点标记时，应确保主干和分支明确，例如，在三个分支中，需确保其中一个分支作为计算图的主干分支。第二，若存在嵌套分支的情形，当嵌套分支所含算子较少时，应忽略嵌套分支情况，当作正常分支处理。

5.1.2 运行时并行调度方案设计

为加快神经网络的训练和推理速度，学界和工业界提出了许多并行方案，最常见的有三类：数据并行^[55]、模型并行^[56]和流水线并行^[57]。如图 5.3 所示，流水线并行是指将网络切分为多个阶段，并分发到不同的计算设备上，各个计算设备之间以接力的方式完成神经网络运行^{[58],[59]}。

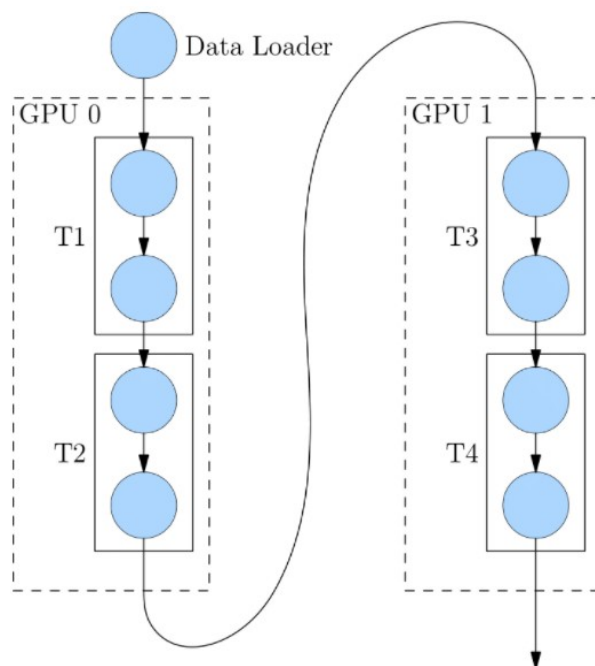


图5.3 流水线并行示意图

本节基于流水线思想，提出针对单个网络的流水线并行方案。其总体方案设计如图 5.4 所示，图中采用两个设备 CPU1+CPU2，三个含有共同开始节点的子图 S1、S2 和 S3，左右两图分别表示 t_1 时刻和 t_2 时刻不同子图在两设备上的运行情况。在 t_1 时刻装载并运行子图 S1，待子图 S1 运行完成后，交由 CPU2 上运行子图 S2。同时，由于 CPU1 处于空闲状态，加载并运行子图 S3，实现分支子图的流水线并行。根据分支并行总体方案，分别设计并行线程池、数据交互以及并行实现三部分具体方案。

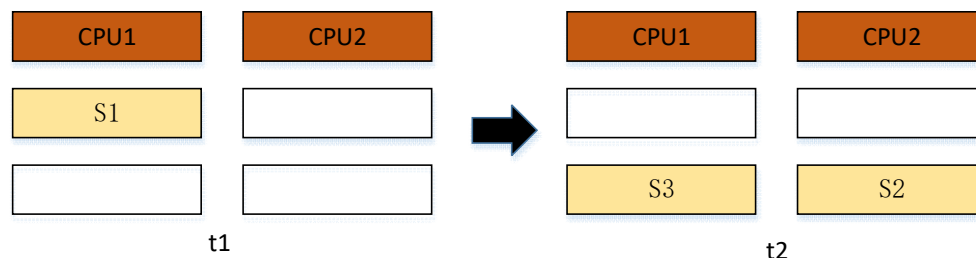


图5.4 分支网络流水线并行调度方案

5.1.2.1 并行线程池设计

并行线程池设计主要针对运行时设计，在流水线机制基础上对线程池进行改进。并

行线程池主要包括任务实体和子图的运行队列的设计，数据交互包括运行时队列的交互，并行实现主要是继承并重写流水线运行时的各类函数。三部分相互调用，完成计算图的分支并行。对分支并行而言，线程池设计是其中非常重要的一个环节，承担着对线程创建销毁、任务运行顺序以及线程管理的重要作用。为实现分支并行线程池，对流水线机制中原有的线程池进行修改和完善，线程池设计如图 5.5 所示。首先，创建一个任务实体，存储并行任务及任务序号，便于后续在线程中对任务进行操作。其次，对于线程队列，修改队列中的入队和出队操作，入队时将需要并行的分支进行入队，出队时判断当前设备是否完成其他任务，同时进行出队操作，以适应分支并行情况。最后，重写线程池，对线程池类的初始化函数进行初始数据的增加，对类中析构函数进行数据的重置操作，以实现线程的创建和销毁。

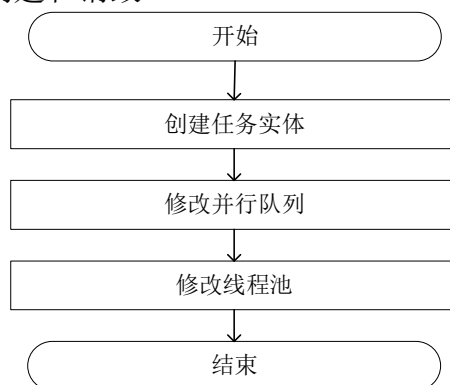


图5.5 并行线程池设计

5.1.2.2 并行数据交互设计

数据交互作为整个计算图运行期间最为重要的环节之一，需要增加方法对数据的管理和交互进行处理，确保数据的正确流向。如图 5.6 所示，首先，使用配置工具类自动配置各个分支任务同分支开始和分支结束任务间的依赖关系，将各个任务间的数据流向进行配置。其次，依次获取每个分支的运行结果，并将其存储在队列中。最后，当分支任务运行完成后，依次从队列中读取各个分支任务的运算结果，作为分支合并任务的输入，进行运算。

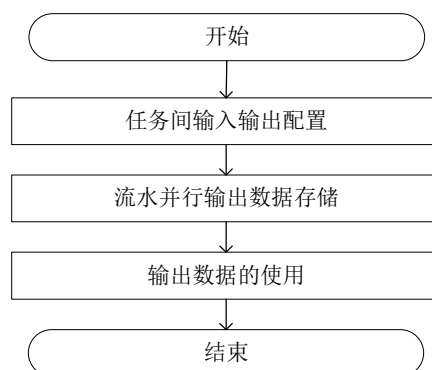


图5.6 数据交互设计

5.2 计算图并行节点标记方法研究

5.2.1 针对并行节点的后序支配树构建

有向无环图作为数据流图的一种，可以清晰的表示数据流向，TVM 中使用此方式描述计算图，但是其结构无法高效满足并行计算图的搜索和生成。因此，本节构建了一个简单的有向无环图作为后序支配树的分析作支撑，称为前向数据标记（IndexedForwardGraph, IFG）。如表 5.1 所示为 IFG 类中的各项属性，主要记录了节点的各种顺序。

表5.1 前项数据标记参数

类型	名称	含义
结构体	Node	存储节点信息，包含对应边集指针、节点序号、节点的输出
结构体	Edge	包含节点指针和该节点对应模式
哈希表	Node_map	节点到图的映射
数组	Post_dfs_order	节点的后序遍历顺序

IFG 中为了保证从数据流图中获取正确的信息，类中编写了包含对节点数据的更新，节点的增加以及重写各个遍历函数，主要重写三个内部函数。首先是更新操作，更新操作针对当前计算图节点 Node，首先获取该节点的 Key，即唯一标识，在 Node_map 中进行查找，若未在 Node_map 中搜索到，则将 Node 的类型由计算图节点转换为 IFG 节点，并在哈希表中存入该节点。其次，节点增加操作，步骤同节点更新操作类似，在进行 Node_map 添加后，将对应的序号等信息存入 Node 中，最后将 Node 添加 Post_dfs_order 数组中。最后，重写遍历函数，主要对 CallNode 遍历函数进行重写，判断 CallNode 的各个参数是否为算子，并将其进行更新操作。重写完成后，调用 Create 函数对计算图进行遍历记录。

表5.2 后序支配树类参数

类型	名称	含义
结构体	Node	存储 IFG 节点、父节点、深度
数组	Nodes	IFG 节点数组

完成数据流图的构建后，需要进行后序支配树的建立，后序支配树的各项参数如表

5.2 所示，主要包含两部分，一是当前节点 **Node**，用于存储节点的前驱和深度，当前节点的信息大部分来源于 IFG 中的数据。二是包含当前所有节点的数组 **Nodes**。

在后序支配树中，如表 5.3 所示，采用了离线算法查找最近公共祖先 LCA，在分支查找中即查找分支的公共开始节点。通过该算法，可以查找出各个分支的开始节点，再结合第二章提取的分支特征，可以对后续计算图进行并行节点标记。其伪代码如下，遍历时从根节点开始进行遍历，首先遍历根节点 **u** 的所有子节点 **v**，并标记这些子节点 **v** 被访问过，直到 **v** 中没有子节点。之后使用 **Combine** 并查集函数将 **v** 合并到 **u** 上，寻找当前与点 **u** 有询问关系的点 **v**，若 **v** 已被访问过，则确认 **u** 和 **v** 的最近公共祖先为 **v**。

表5.3 LCA 伪代码

Function: Tarjan

In: 根节点 **u**

Out: 最近公共祖先

```

1. Tarjan(u)
2. {   for each(u,v)
3.     {   Tarjan(v); //查找 v 的子节点
4.         join(u,v); //把 v 并到 u 上
5.         visit[v]=1; //访问标记
6.     for each(u,v) //遍历与 u 有关的节点 v
7.     {   if(visit[v])
8.         ans=find(v); } } //若果 v 被访问过，则查找 v

```

5.2.2 带有并行节点标记的计算图生成

完成后序遍历树的建立后，开始对并行计算图进行构建，构建过程主要分为三步，如图 5.7 所示，第一步检查分支节点和主干节点，只对分支节点进行操作。第二步，使用前项数据标记图和后序支配树对计算图进行分组。最后，对每组进行并行标记。

第一部分为分支节点的查找及存储，创建 **groups_** 数组，并对其进行初始化，初始化时。**Group_** 数组用于存放符合条件的数组，根据 IFG 中的后序广度优先遍历（Depth First Search, DFS）搜索的节点数量作为 **groups_** 数组的大小。同时，将 IFG 图中每个节点的属性作为一组写入 **groups_** 中，便于数组进行查找。

第二部分为计算图分组，将符合条件的计算图分成一组。需要对 **groups_** 数组进行循环遍历，由于存储数组时采用后序广度优先遍历，从小到大遍历数组即以后序 DFS 的顺序进行节点的操作。遍历时，使用 IFG 图和后序支配树两者的属性进行分支的查找匹配，主要是用 IFG 中的 **Post_dfs_order** 数组和后序支配树中的 **Post_dom_tree** 结构体，其中，可以根据后序支配树的节点位置查找该节点的前驱，进而查找共同祖先。在进行

节点的逐一遍历时,若遇到分支结束节点,则将参数类型为算子的参数放入后序支配树中的 LCA 函数进行最近公共祖先的查找,即分支开始节点。在查找到后,使用 CheckPath 函数对节点前驱进行查找,判断是否为主干节点。若否,则记录各个分支内的开始节点和结束节点,交由子图打包函数 CommitParallelGroup 进行打包。

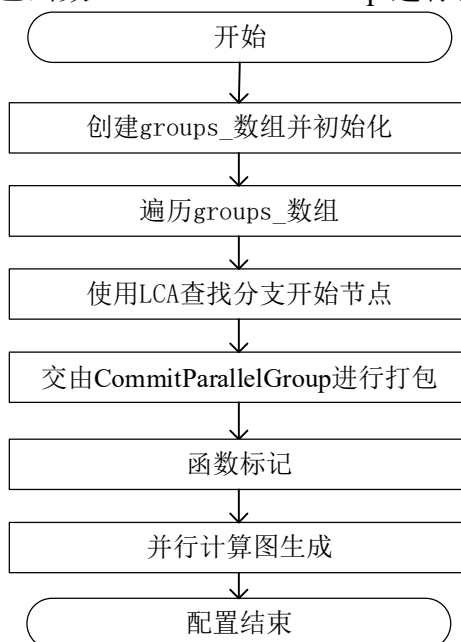


图5.7 并行计算图生成

第三部分为计算图并行标记的添加,将符合并行条件的分支进行打包。打包时采用 CommitParallelGroup 函数以递归的形式对节点进行打包,将后继节点与前驱结点合并为集合,集合采用函数形式,将函数名命名为 Parallel。同时,处理各个节点的输入参数,将节点参数统一绑定至函数的输入参数中并重命名各个参数。

5.3 运行时并行调度算法设计

5.3.1 并行线程池设计

根据 5.1 小节中并行线程池设计方案可知,线程池设计主要分为任务实体、线程队列和线程池设计三部分。任务实体作为整个线程池设计中的基础结构,承担着后续子图任务的执行和交互,具体包含的参数如表 5.4 所示,主要包含两部分,任务载入类对象和任务标识 ID。

任务载入对象 Launcher 是 ParallelLauncher 类生成的一个对象,该类定义了并行任务的基本操作,如表 5.5 所示,包含等待任务 WaitForJobs 和标识任务结束 SignalJobFinish 两部分。WaitForJobs 用于判断当前时刻是否有任务处于运行状态,若有,则返回运行的任务数量,否则返回-1, SignalJobFinish 用于标识当前任务为结束状态。Task_id 作为任

任务的执行唯一标识，用于标识任务执行顺序，以及任务运行出错时标记出错节点。

表5.4 任务实体参数

类型	名称	含义
ParallelLauncher	Launcher	任务载入对象
Int32_t	Task_id	任务标识

完成任务实体的定义后，需要对任务队列进行修改定义。任务队列包括出队、入队和任务终止三个操作，本节主要修改入队和出队操作。入队操作是将处在等待状态的任务压入队列尾部。入队时识别分支任务，检查该任务是否已经存在队列中，若否，则压入队尾。出队操是将符合运行条件的任务弹出队列运行，在出队时，检查当前是否有任务执行，若无任务执行，则进行出队操作，同时运行任务。在出队入队时，为保证多线程时进队出队有序，采用获取-释放（acquire-release）的内存顺序对两个线程进行排序。即对于 A、B 两个线程操作同一个对象 x 时，需要线程 A 先写入 x（释放操作），再进行线程 B 的读取操作（获取操作）。

表5.5 ParallelLauncher 类内函数

名称	含义
WaitForJobs	等待任务结束
SignalJobFinish	标记任务结束

当线程队列建立完成后，基于此队列创建线程池，如表 5.6 所示，主要分三部分。第一部分为线程池初始化操作，首先，将线程池进行初始化，即根据子图数进行队列的扩充操作，队内每个元素为指针，便于后续对具体属性的添加修改。其次，创建线程组，对队内任务进行弹出操作，弹出时，调用 TVM 本身并行函数执行弹出时所包含的任务。最后，根据执行情况标识任务完成情况。第二部分为线程析构函数，即进行队列的停止和线程的重置操作。首先，停止队列时，设置退出信号量为 TRUE，表明队列不在运行状态，之后标识所有队内任务为不执行状态，其次，重置线程指针。第三部分为线程池函数重置，同析构函数类似，同样需要停止队列和线程重置。不同的是，需要将队列进行清零，即删除所有尚未完成的任务，之后重新执行初始化操作，以初始化数组和线程组。

表5.6 线程池操作

名称	含义
ThreadPool	线程池初始化函数
~ThreadPool	线程池析构函数
Reset	线程池重置函数

5.3.2 并行节点数据交互设计

完成分支任务并行操作后,需要获取各个分支任务的输出数据作为分支合并操作的输入。首先使用配置数据交互的工具类对分支任务进行输入输出的交互操作,前端配置过程同 4.3 章节类似,对各个分支进行输入输出配置。后端读取计算图生成的 JSON 格式数据,JSON 中包含的主要数据如表 5.7 所示,主要有任务序号、依赖名称、参数名称和设备等参数,各个参数在当前运行时均保持唯一性。具体操作步骤如下,首先,创建用于存储各类参数的同名变量。其次,使用 while 循环对 JSON 进行遍历,依次使用 if 语句判断当前遍历的键值是否为需要的值,若是,则读取对应值并存储到同名变量中。最后,依次判断各个变量值是否合理,即名称是否为空和序号是否大于 0。将对应任务读写完毕后,执行任务得到输出。

表5.7 JSON 包含数据

名称	含义
Mod_idx	任务序号
Mod_interface_name	任务名称
Lib_name	依赖名称
Params_name	参数名称
Dev	设备

一个任务完成后,需要对数据输出做出处理,为减少数据读写的次数,设计输出队列用于存储每个分支任务的输出,具体处理过程如下。首先,根据任务 ID 判断该任务是否存在于输出队列,若存在,则利用 ID 获取该数据,同时弹出队列。其次,将该数据压入运行队列的队尾,等待数据被正在执行任务的使用。为确保数据被接收,防止出现因为队列达到入队上限导致无法入队的问题,使用 while 循环的方式对该数据进行入队操作,直至队列中有空位时入队。

5.3.3 节点并行化实现

完成线程队列、线程池和数据交互设计后，需要设计任务的运行流程。首先，进行流水线初始化操作，读取 JSON 中的模块数目，即分支任务数量，同时，为每个模块创建一个运行时。第二，创建一个用于存储各模块输出的数组，用于存储 5.3.2 小节中任务输出的值。第三，利用 5.3.1 小节中队列初始化操作对线程队列和线程池进行初始化。第四，执行运行时前，需要对参与运算的参数和输入进行绑定匹配，便于后续运算。第五，依次执行各个运行时，每当一个运行时执行完毕后，进行数据交互处理，将得到的数据存储到数据队列。

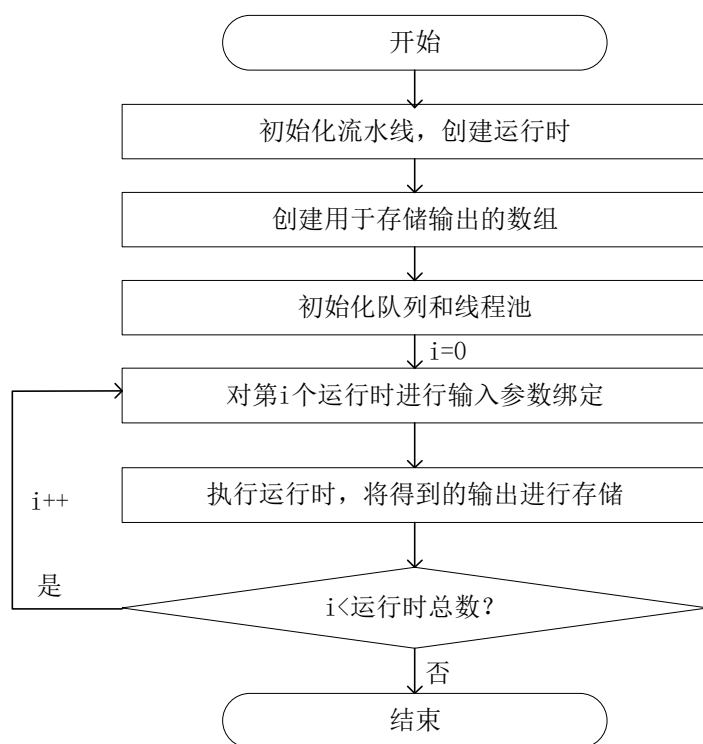


图5.8 节点并行设计实现

节点并行运行流程如图 5.8 所示，为便于理解，本节使用含有两个 A、B 分支的计算图 Graph 作为输入，两个设备 X、Y 作为运行设备，详细描述运行流程。首先，计算图 Graph 经过并行节点标识处理后，生成流水线 JSON 图。后端对 JSON 图进行读取，获取两个模块 A 和 B，分别根据 A、B 创建运行时，并且创建一个大小为 2 的数组，用于存储运行输出，然后，对每个子图进行输入参数绑定。其次，创建队列，并根据当前设备数 2 创建线程，分别对 A、B 进行操作。A 从队列中弹出，交由线程 1 运行，这时线程 2 空闲，则继续让 B 出队，交由线程 2 运行 B。其中，执行时使用 TVM 内置的运行函数执行，可以自动将代码在对应设备商生成代码并执行，返回结果。最后，运行结

束，获取两个子图的运行结果，存储到数据队列，等待计算时使用这些数据。

5.4 并行调度实验结果与性能分析

5.4.1 实验环境与实验方案

本章选用的软硬件环境如 4.4.1 节中所述，神经网络编译器采用 TVM0.9，后端编译库采用 LLVM10.0，操作环境采用 Ubuntu18.04，软件编译处理器使用戴尔服务器 Intel Xeon Gold 6248R @3GHz，96 核心。英伟达 GPU 芯片 Nvida Quadro P2200，显存容量 5GB，显存位宽 160bit，最大功耗 75W。本文中 CNN 算法特征提取、计算图划分、计算图并行实现均在服务器上进行。

本章实验方案将从不同类型分支网络算法的并行执行时间比较分支并行方法的性能，性能比较即对比运行时间快慢。本节使用多个分支网络的模块进行实验验证，网络模型采用 4.4.1 节中常用的分支网络模型，如 Inception 和 GoogleNet。实验对象为 TVM 的 GraphExecutor 串行运行机制以及文献[36]提出的 Greedy 并行机制。为便于测试分支并行的性能，将网络中各个不相同的分支结构单独提取作为一次测试模型，部分测试数据如表 5.8 所示。测试集采用 ImageNet，对每个分支网络分别测试七次速度并取平均值。本章分别在 GPU 平台和 CPU 平台分别进行分支并行的推理对比。此外，为了模拟多设备，本节在 CPU 上的实验采用锁定核心数来实现，在传统运行模式下默认使用 14 个核心，7 个核心作为一个单独设备。

表5.8 分支网络数据集

序号	分支个数	分支最大节点数
1	2	4
2	3	6
3	3	8
4	4	4
5	4	5
6	4	10
7	4	12

5.4.2 计算图并行调度性能比较及分析

5.4.2.1 计算图并行图标记结果验证

并行图标记分为两部分，分支标记和并行标记。使用分支标记方法的进行标记的结果如图 5.9 所示，图中所示为 Inception_v3 网络中多个分支所标记的注释。从分支标记的结果可以看出，算法对计算图进行了遍历，构建支配树。使用后序支配树中经过修改的 LCA 算法，从分支结束节点出发，从后向前寻找最近共同祖先。从图中可以看出，搜索到共同祖先时，记录公共祖先，并打上需要划分的标记，如图中%469 所示，算法不仅对节点进行标记，还对节点中的参数进行标记。同时，算法会进行回溯，逐步从后序遍历中返回前一步，因此，方便对分支内剩余节点的标记。当回溯到起始查找位置时，添加分支结束的标记，如图中%480-%483 所示。标记添加完成后，利用 TVM 内置函数对分支进行切割，将切分的节点进行存储。

```
%469 = annotation.compiler_begin(%389, compiler="parallel");
%470 = nn.avg_pool2d(%469, pool_size=[3, 3], padding=[1, 1, 1, 1], count_include_pad=True);
%471 = annotation.compiler_begin(%Mixed_7c.branch_pool.conv.weight, compiler="parallel");
%472 = nn.conv2d(%470, %471, padding=[0, 0, 0, 0], channels=192, kernel_size=[1, 1]);
%473 = annotation.compiler_begin(%Mixed_7c.branch_pool.bn.weight, compiler="parallel");
%474 = annotation.compiler_begin(%Mixed_7c.branch_pool.bn.bias, compiler="parallel");
%475 = annotation.compiler_begin(%Mixed_7c.branch_pool.bn.running_mean, compiler="parallel");
%476 = annotation.compiler_begin(%Mixed_7c.branch_pool.bn.running_var, compiler="parallel");
%477 = nn.batch_norm(%472, %473, %474, %475, %476, epsilon=0.001f);
%478 = %477.0;
%479 = nn.relu(%478);
%480 = annotation.compiler_end(%399, compiler="parallel");
%481 = annotation.compiler_end(%429, compiler="parallel");
%482 = annotation.compiler_end(%468, compiler="parallel");
%483 = annotation.compiler_end(%479, compiler="parallel");
%484 = (%480, %481, %482, %483);
%485 = concatenate(%484, axis=1);
```

图5.9 分支节点标记

```
%393 = @tvmgen_default_parallel_main_43(%389, %Mixed_7c.branch_pool.conv.weight, %Mixed_7c.branch_pool.bn.bias, %Mixed_7c.branch_pool.bn.running_mean, %Mixed_7c.branch_pool.bn.running_var, float32) */;
%394 = (%390, %391, %392, %393);
%395 = concatenate(%394, axis=1) /* ty=Tensor[(1, 2048, 5, 5), float32] */;
%396 = nn.adaptive_avg_pool2d(%395, output_size=[1, 1]) /* ty=Tensor[(1, 2048, 1, 1), float32] */;
%397 = nn.dropout(%396) /* ty=(Tensor[(1, 2048, 1, 1), float32], Tensor[(1, 2048, 1, 1), float32]) */;
%398 = %397.0;
%399 = reshape(%398, newshape=[0, -1, 1, 1]) /* ty=Tensor[(1, 2048, 1, 1), float32] */;
%400 = squeeze(%399, axis=[2, 3]) /* ty=Tensor[(1, 2048), float32] */;
%401 = nn.dense(%400, %fc.weight, units=None) /* ty=Tensor[(1, 1000), float32] */;
nn.bias_add(%401, %fc.bias, axis=-1) /* ty=Tensor[(1, 1000), float32] */
}

def @tvmgen_default_parallel_main_0(%parallel_0_i0: Tensor[(1, 2048, 5, 5), float32], %parallel_0_i1: Tensor[(1, 2048, 5, 5), float32], %parallel_0_i2: Tensor[(320), float32], %parallel_0_i3: Tensor[(320), float32], %parallel_0_i4: Tensor[(320), float32], %parallel_0_i5: Tensor[(320), float32], Inline=1, Compiler="parallel", Primitive=1) -> Tensor[(1, 320, 5, 5), float32] {
    %402 = nn.conv2d(%parallel_0_i0, %parallel_0_i1, padding=[0, 0, 0, 0], channels=320, kernel_size=[1, 1], float32) */;
    %403 = nn.batch_norm(%402, %parallel_0_i2, %parallel_0_i3, %parallel_0_i4, %parallel_0_i5, float32, Tensor[(320), float32], Tensor[(320), float32]) */;
    %404 = %403.0;
    nn.relu(%404) /* ty=Tensor[(1, 320, 5, 5), float32] */
}
```

图5.10 并行图标记结果

使用 IRmodule 存储分支的各个节点，同时组建成函数并用 def 定义，生成后，对函数名以及参数进行标记，如图 5.10 的第二条横线所示，添加 Parallel 并行标记，重命名的格式为 Parallel+main+x，x 为所划函数的顺序标记，表明该函数可进行并行操作。同时，对函数的所有参数进行重命名，打上并行标记。最后，交由 CommitParallelGroup 进行打包，生成完整的并行子图，便于后端运行时对子图进行识别处理。并行子图生成后，调用方式如图第一条横线所示，采用函数形式对其进行调用。函数内部包含使用分支标记后划分后的节点集合，每一个函数代表一个分支。调用时，直接使用函数名进行使用，并对参数写入对应值，以此来调用并行计算图，执行推理运算。

5.4.2.2 计算图并行实验结果及分析

使用基于流水线的分支并行方法和 TVM 的 GraphExecutor 运行机制的速度对比如图 5.11 所示。从图中可以看出，分支划分方法优于 TVM 传统运行方法。和 TVM 的 GraphExecutor 方法对比，分支划分方法的速度平均提升在 5%，其中测试集序列 1-4，11-13 性能提升较差的原因为数据的计算量较少，使得数据交互占比较大，导致性能降低。后期使用 Inception 网络靠后的节点，划分后每个分支内部节点数量较多，且各个节点需要计算的图形较大，利用分支并行时可以对其进行较为明显的加速，性能平均在 10%。经实验观察发现，进行 14 核心数推理时，设备无法进行满载运算，且需要跨 CPU 进行，使得数据间的交互占据较多时间，成为速度提升的不稳定因素，这也是下一步需要改进的方向。

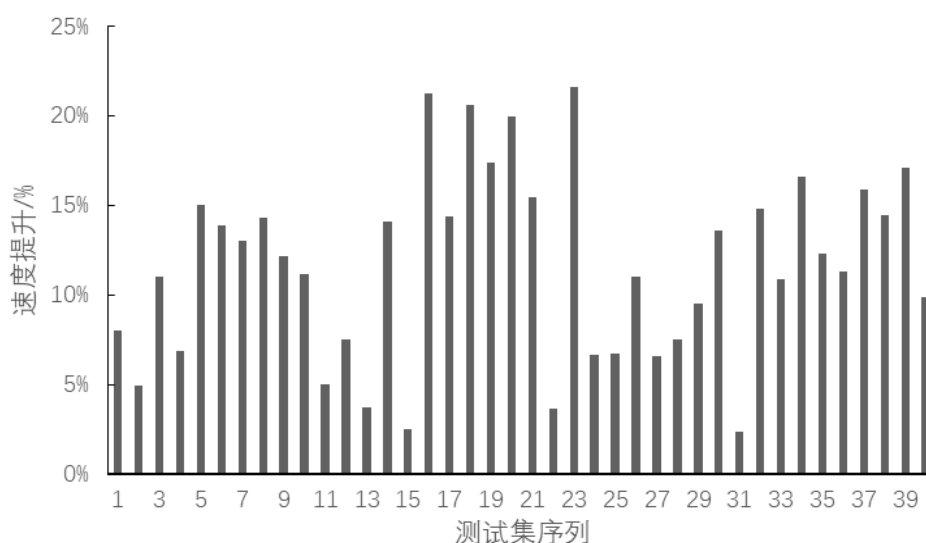


图5.11 CPU 下并行速度提升

在 GPU 下进行推理运算时，需要对各个子图进行 GPU 设备的参数配置，并行推理

结果如图 5.12 所示。从图中可以看出，相对于 CPU 下的速度提升较低的现象，GPU 下的速度提升明显高于 CPU，平均在 15%，最低速度提升也在 5%。这是由于 GPU 本身结构可以有较大的吞吐量，适合用于执行并行操作，容纳多个分支并行运算。此外，并行算法基于流水线机制改进，在进行计算时可以利用其进行加速。在速度提升数据中，大部分情况下速度提升较为明显，少部分数据如 13、16 和 23 等测试序列出现速度提升不明显的现象，这是由于各个测试集对应的算法输入规模不一致所导致。对于 GPU，采用相同小规模数据时无法充分利用 GPU 特性，因此提升效果不明显，需要进一步进行改进。

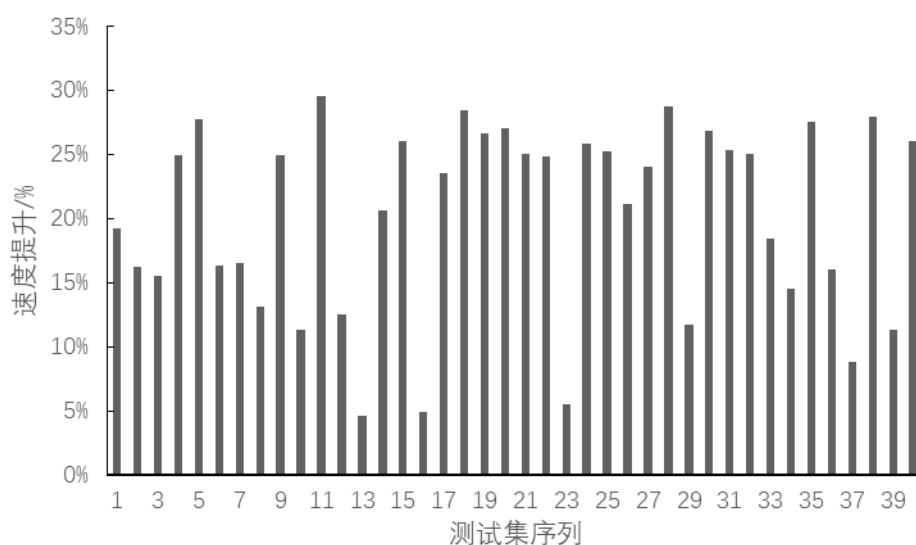


图5.12 GPU 下并行速度提升

最后，本章使用文献[40]中提到的 Greedy 调度方法与本章提出的算法进行对比，对比如图 5.13 所示。Greedy 算法采用算子间并行的方法对神经网络进行加速，本节方法采用基于流水线机制的算子间并行以及 TVM 的算子内部优化策略。

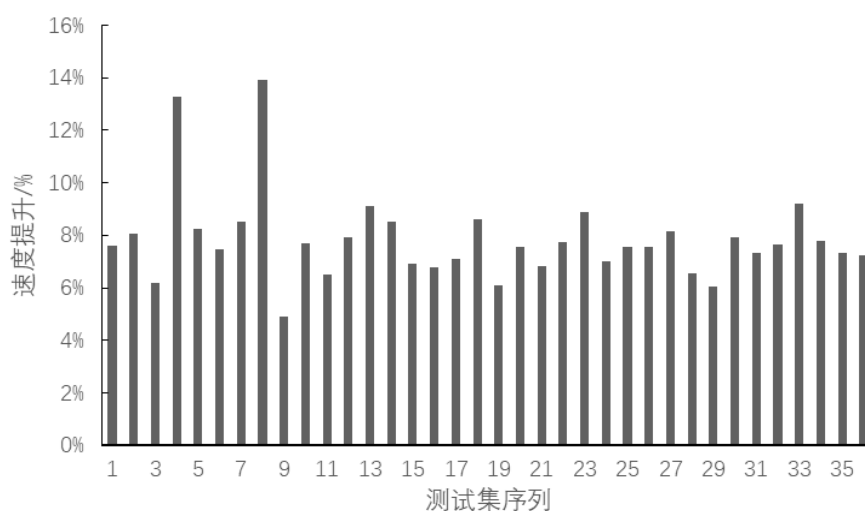


图5.13 并行调度对比性能提升

实验采用表 5.8 所示数据集规模，分别在 GPU 上进行实验，通过 TVM 后端内置的优化库以及 CuDNN 加速库分别对 Greedy 和本节算法进行优化加速。从图 5.13 中可以看出，本节算法相比文献[40]中的 Greedy 算法，平均速度提升在 5%左右，有较好的加速效果。但是，图中测试序列 9、19 和 29 等测试序列速度提升较慢，主要因为这几个测试序列的分支个数较多。本节算法没有同文献[40]中优化后的 Greedy 算法一样，设置以动态规划为基础的评估函数，导致任务出现等待的问题。但是，借助本节对线程池和队列的优化，以及流水线机制中对流水线并行的优化，算法仍有一定的优化效果。

本章方法依据分支特征信息进行分支划分及分支并行设计，并对并行后的结果进行测试。实验结果表明，该并行调度方法可以有效提升分支网络的推理速度。因此，本章方法能够提升 CNN 算法的推理速度。

5.5 本章小结

针对 TVM 中不能对分支网络进行分支并行的问题，设计了一种分支并行调度方法，使用独自设计的数据流图，包括有向无环图和后序支配树，对节点进行搜索匹配。对于匹配后的节点，将其打包成函数并作并行标记，生成并行计算图，交由运行时处理，运行时针对并行节点进行并行处理，包括并行间线程池设计，数据交互及运行。实验结果显示，相比于 TVM 传统的串行方法，分支并行方法在计算设备上能升了 10%和 20%的推理速度，相比于 Greedy 算法，本章提出的算法平均提升在 5%，经过实验结果表明，本章提出的并行调度算法可以有效对计算图进行推理加速。

6 总结与展望

6.1 总结

神经网络模型越来越呈现出多样性的特征,现有的处理框架对不同网络模型有独特的支持,难以实现统一。而 TVM 因为其设计理念的统一性,将所有神经网络模型转换为计算图进行优化和部署。针对目前 TVM 中在某些方面存在优化不明显的问题,提出了一种基于 TVM 的卷积优化和推理计算图并行调度设计。并以典型 CNN 算法为例,对提出的优化方法在 TVM0.8 编译器上实现。主要工作总结如下:

(1) 为挖掘 CNN 算法的分支特性和卷积特性,本文深入研究分析的 CNN 算法的行为模式。首先,使用 TVM 获取计算图中的基础结构信息、分支结构和分支间依赖关系。然后,根据此类信息构建特征计算图来描述 CNN 算法的分支信息,获取 CNN 算法的分支特征。其次,提取了卷积中的访存模式,即访存特征和计算图特征,用于指导卷积的优化操作。实验结果表明,经过特征优化的 CNN 算法能达到传统优化方法加速比的 20%。CNN 算法行为模式分析能有效提取 CNN 算法特征,挖掘特征信息。

(2) 提出一种针对 MEC 的卷积优化算法。首先,对输入矩阵和中间矩阵的处理方式进行优化,将原有的访问方式修改为行优先访问,使得访问顺序符合程序访存模式。然后,本文更改了卷积核的数据布局,使用 TVM 封装的计算函数对卷积的计算方式进行优化,修改卷积计算方式为行乘以行。此外,本文还利用 TVM 平台的优化方法对算法进行优化加速,达到提高卷积计算性能的目的。实验结果表明,优化算法相较于 MEC 算法,获得不同程度的性能加速。在测试不同规模的单层卷积时获得了平均 50%的性能提升,此外,本文将优化算法和 MEC 算法集成到 TVM 平台中,使用 ResNet18、ResNet101 和 DenseNet121 三个神经网络模型作为测试模型,分别获得 57%、75%和 75%的速度提升。同时,本文将优化算法同空间组合算法进行对比,获得了最高 45%的速度提升,又进一步验证本文提出的优化算法的有效性。

(3) 针对流水线中现有的计算图划分策略依赖于人工经验,不利于推广应用问题,提出了一种基于分支特征的计算图划分方法。该方法使用基于计算图的分支特征,对计算图进行分支特征的查找,并存入数组中保存信息。根据数组中的信息对子图进行子图构建、依赖配置、子图重写等操作。通过这些来指导分支划分,实验结果表明,使用基于分支特征的计算图划分方法, CNN 算法速度提升了 15%。

(4) 针对 TVM 中不能对分支网络进行分支并行的问题,设计了一种分支并行调度

方法,使用独自设计的数据流图,包括有向无环图和后序支配树,对节点进行搜索匹配。对于匹配后的节点,将其打包成函数并作并行标记,生成并行计算图,交由运行时处理,运行时针对并行节点进行并行处理,包括并行线程池设计,数据交互及运行。实验结果表明,使用基于流水线的分支并行算法,CNN 的推理速度在 CPU 下平均提升了 10%,GPU 下提升了 20%。同时,相比于 Greedy 算法,本章提出的算法平均提升了 5%,验证了分支并行算法的有效性。

6.2 展望

本文面向 TVM 设计并实现的卷积优化和推理图并行调度设计,有效提升的 CNN 算法的效能,加快的推理速度。但仍存在以下问题:

(1) CNN 算法行为模式分析中,已完成分析卷积算法结构清晰的计算和访存特征提取,但是类似于 Winograd 等复杂的算法,难以获取算法内部结构和依赖关系。因此,如何有效提取特征信息,将是实现算法优化的一个研究重点。

(2) 基于访存模式的 MEC 优化算法中,已完成根据访存模式实现算法的优化。但是,当算法结构复杂时,现有的访存模式优化方法难以使用到其他算法上。因此,如何构建一个更加通用的访存优化方法,将是提升算法效率的一个研究重点。

(3) 基于分支特征的计算图划分方法中,已完成对计算图的划分及性能提升,但是该方法还需手动获取分支特征,如何研究算法,使分支划分可以根据评估函数自动完成分支划分,将是流水线模式的一个研究重点。

参考文献

- [1] Tsai M F, Tseng H J. Enhancing the identification accuracy of deep learning object detection using natural language processing[J]. The Journal of Supercomputing, 2021: 2021, 77(7):1-16.
- [2] Kasabov N K . Spiking neural networks for deep learning and knowledge representation: Editorial[J]. Neural Networks, 2019, 119:341-342.
- [3] David, Kanter. Google TPU Boosts Machine Learning[J]. Microprocessor report, 2017, 31(5):18-21.
- [4] Chen T, Moreau T, Jiang Z, et al. TVM: An automated end-to-end optimizing compiler for deep learning[C].//13th {USENIX} Symposium on Operating Systems Design and Implementation. 2018, PP(99): 578-594.
- [5] Li M, Liu Y, Liu X, et al. The Deep Learning Compiler: A Comprehensive Survey[J]. IEEE Transactions on Parallel and Distributed Systems, 2021, 32(3):708-727.
- [6] Jiang H, Zhou Z, Ren Z, et al. CTOS: Compiler Testing for Optimization Sequences of LLVM[J]. IEEE Transactions on Software Engineering, 2021, PP(99):1-8.
- [7] Wang C, Pastore F, Goknil A, et al. Automatic Generation of Acceptance Test Cases from Use Case Specifications: an NLP-based Approach[J]. IEEE Transactions on Software Engineering, 2020, PP(99):1-9.
- [8] Lawaniya H. Introduction to Neural Network & Single Layer Neural Network featuring Computer Vision[J]. Neural Networks, 2020. PP(99):1-6.
- [9] Da'U A, Salim N, Rabiul I, et al. Recommendation System Exploiting Aspect-based Opinion Mining with Deep Learning Method[J]. Information Sciences, 2019, 512(1): 1279-1292.
- [10]Gawehn E, Hiss J A, Schneider G. Deep Learning in Drug Discovery[J]. Molecular Informatics, 2016, 35(1):1-8.
- [11]Chen H, Engkvist O, Wang Y, et al. The rise of deep learning in drug discovery[J]. Drug discovery today, 23(6): 1241-1250.
- [12]Xu J, Liu H, Wu D, et al. Generating Universal Adversarial Perturbation with ResNet[J]. Information Sciences, 2020, 537(1):302-312.
- [13]Jin X, Wu L, Li X, et al. ILGNet: inception modules with connected local and global

- features for efficient image aesthetic quality classification using domain adaptation[J]. IET Computer Vision, 2019, 13(2):206-212.
- [14] Ji J, Zhong B, Ma K K. Image Interpolation Using Multi-Scale Attention-Aware Inception Network[J]. IEEE Transactions on Image Processing. 2020. 29(1):9413-9428.
- [15] Patterson D. 50 Years of computer architecture: From the mainframe CPU to the domain-specific tpu and the open RISC-V instruction set[C].//IEEE International Solis-State Circuits Conference-(ISSCC). IEEE.2018, PP(99):27-31.
- [16] 朱虎明,李佩,焦李成,杨淑媛,侯彪.深度神经网络并行化研究综述[J].计算机学报, 2018, 41(08):1861-1881.
- [17] Abadi M, Barham P, Chen J, et al. Tensorflow: A system for large-scale machine learning[C].//In 12th {USENIX} symposium on operating systems design and implementation ({OSDI} 16) GA, USA, 2016, PP(99):265–283.
- [18] Jia Y, Shelhamer E, Donahue J, et al. Caffe: Convolutional architecture for fast feature embedding[C].//In Proceedings of the 22nd ACM international conference on Multimedia. ACM, ACM, Orlando, FL, USA, 2014, PP(99):675–678.
- [19] Ahn, Byung Hoon, et al. Ordering chaos: Memory-aware scheduling of irregularly wired neural networks for edge devices[C].//Proceedings of Machine Learning and Systems .2020, 2:44-57.
- [20] 张顺,龚怡宏,王进军.深度卷积神经网络的发展及其在计算机视觉领域的应用[J].计算机学报, 2019, 42(03):453-482.
- [21] Guo Q, Xie X, Li Y , et al. Audee: automated testing for deep learning frameworks[C].// ASE '20: 35th IEEE/ACM International Conference on Automated Software Engineering. ACM, 2020.PP(99): 486-498.
- [22] Qararyah F, Wahib M, Dikbayr D, et al. A Computational-Graph Partitioning Method for Training Memory-Constrained DNNs[J]. Parallel Computing. 2020.104(102792):1-14.
- [23] Meloni P, Capotondi A, Deriu G, et al. NEURAghe: Exploiting CPU-FPGA Synergies for Efficient and Flexible CNN Inference Acceleration on Zynq SoCs[J]. Acm Transactions on Reconfigurable Technology & Systems, 2018, 11(3):18.1-18.24.
- [24] Manasi S D, Snigdha F S, Sapatnekar S S. NeuPart: Using Analytical Models to Drive Energy-Efficient Partitioning of CNN Computations on Cloud-Connected Mobile Clients[J]. IEEE Transactions on Very Large Scale Integration (VLSI) Systems, 2020, 28(8): 1844-1857.

- [25]Deng Z, Peng X, Li Z, et al. Mutual Component Convolutional Neural Networks for Heterogeneous Face Recognition[J]. IEEE Transactions on Image Processing, 2019, 28(6):1-10.
- [26]Cho M, Brand D. MEC: memory-efficient convolution for deep neural network[C].// International Conference on Machine Learning. 2017, PP(99): 815-824.
- [27]Xing Y, Weng J, Wang Y, et al. An In-depth Comparison of Compilers for Deep Neural Networks on Hardware[C].// IEEE International Conference on Embedded Software and Systems (ICCESS). IEEE, 2019, PP(99):1-8
- [28]Urbann O, Camphausen S, Moos A. AC Code Generator for Fast Inference and Simple Deployment of Convolutional Neural Networks on Resource Constrained Systems[C].// In 2020 IEEE International IOT, Electronics and Mechatronics Conference (IEMTRONICS) IEEE.2020,PP(99):1-7.
- [29]Boemer F, Lao Y, Cammarota, R, nGraph-HE: a graph compiler for deep learning on homomorphically encrypted data[C].// In Proceedings of the 16th ACM International Conference on Computing Frontiers. 2018. PP(99): 3-13.
- [30]Boemer F, Costache A, Cammarota R. nGraph-HE2: A high-throughput framework for neural network inference on encrypted data[C].//In Proceedings of the 7th ACM Workshop on Encrypted Computing & Applied Homomorphic Cryptography.2019.PP(99): 45-56.
- [31]Lanmin Z, Tianqi C. Optimizing Deep Learning Workloads on ARM GPU with TVM[C].//In Proceedings of the 1st on Reproducible Quality-Efficient Systems Tournament on Co-designing Pareto-efficient Deep Learning (ReQuEST '18). Association for Computing Machinery, New York, NY, USA, 2018,3(1):1-5.
- [32]Dukhan M. Indirect deconvolution algorithm[C].// 2020 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW). IEEE. 2020,PP(99):922-926.
- [33]Zhao T, Hu Q, He X, et al. ECBC: Efficient Convolution via Blocked Columnizing[J]. IEEE Transactions on Neural Networks and Learning Systems, 2021, PP(99):1-13.
- [34]Yang H, Fritzsche M, Bartz C, et al. Bmxnet: An open-source binary neural network implementation based on mxnet[C].//Proceedings of the 25th ACM international conference on Multimedia. 2017,PP(99): 1209-1212.
- [35]Jiang Y, Zhao T, He X, et al. BitStream: An efficient framework for inference of binary neural networks on CPUs[J]. Pattern Recognition Letters, 2019, 125(JUL):303-309.

-
- [36]W Liu, H Liu, X Liao. NGraph: Parallel Graph Processing in Hybrid Memory Systems[J]. IEEE Access. 2019, 7(1):103517-103529.
- [37]Chen Z, Yu C, Morris T. Bring Your Own Codegen to Deep Learning Compiler. arXiv preprint arXiv:2105.11059:1-14.
- [38]T Koehler, M Steuwer. Towards a Domain-Extensible Compiler: Optimizing an Image Processing Pipeline on Mobile CPUs[C]// 2021 IEEE/ACM International Symposium on Code Generation and Optimization (CGO), 2021, PP(99): 27-38.
- [39]Xiang Y, Kim H. Pipelined Data-Parallel CPU/GPU Scheduling for Multi-DNN Real-Time Inference[C]// 2019 IEEE Real-Time Systems Symposium (RTSS). IEEE, 2019.PP(99):1-15.
- [40]Ding Y, Zhu L, Jia Z. et al. Ios: Inter-operator scheduler for cnn acceleration[C]// Proceedings of Machine Learning and Systems. 2021,3(1):167-180.
- [41]GIBSON P, CANO J, TURNER J, et al. Optimizing grouped convolutions on edge devices[C]// In 2020 IEEE 31st International Conference on Application-specific Systems, Architectures and Processors (ASAP). IEEE. 2020,PP(99):189-196.
- [42]RAJABATHER K, ABIMANNAN S, et al. Restoration of Digital Design Using Row and Column Major Parsing a Technique[C]//2020 IEEE International Conference on Electronics, Computing and Communication Technologies (CONECCT). IEEE, 2020,PP(99): 1-6.
- [43]YANG J, ZOU B, QIU H, et al. MLFNet-Point Cloud Semantic Segmentation Convolution Network Based on Multi-Scale Feature Fusion[J]. IEEE Access, 2021, 9(1): 44950-44962.
- [44]Lu, Liqiang, and Yun Liang. SpWA: An efficient sparse winograd convolutional neural networks accelerator on FPGAs[C]//Proceedings of the 55th Annual Design Automation Conference. 2018.PP(99):1-6.
- [45]Jia Liancheng, LiangYun, LiXiuhong, et al. Enabling Efficient Fast Convolution Algorithms on GPUs via MegaKernels[J]. IEEE Transactions on Computers, 2020.7(69):986-997.
- [46]SHARMA S, BAWA V S, KUMAR V. A Novel Two-stage Residual Learning Based Convolutional Neural Network for Image Super Resolution[J]. Fundamenta Informaticae, 2019, 168(2): 335-351.
- [47]Xiong W, Katzenbeisser S, Szefer J. Leaking information through cache LRU states in

- commercial processors and secure caches[J]. IEEE Transactions on Computers, 2021, 70(4): 511-523.
- [48]方玉玲, 陈庆奎. 基于矩阵转换的卷积计算优化方法[J]. 计算机工程, 2019, 045(007):217-221.
- [49]Chen T, Chen Y, Yu M, et al. NNBlocks: a Blockly framework for AI computing[J]. The Journal of Supercomputing, 2021, 77(8):8622-8652
- [50]FAZ-HERNÁNDEZ A, LOPEZ J, DAHAB R. High-performance implementation of elliptic curve cryptography using vector instructions[J]. ACM Transactions on Mathematical Software (TOMS), 2019, 45(3): 1-35.
- [51]HUANG G, LIU Z, VAN L, et al. Densely connected convolutional networks[C].// Proceedings of the IEEE conference on computer vision and pattern recognition. 2017,PP(99): 4700-4708.
- [52]LU J, FANG C, XU M, et al. Evaluations on Deep Neural Networks Training Using Posit Number System[J]. IEEE Transactions on Computers. 2020, 70(2): 174-187.
- [53]QIU X, SUN T, XU Y, et al. Pre-trained models for natural language processing: A survey[J]. Science China Technological Sciences, 2020, 63(10):1872-1897.
- [54]Roesch J, Lyubomirsky S, Chen T, et al. Relay: A new ir for machine learning frameworks[C].// In Proceedings of the 2nd ACM SIGPLAN international workshop on machine learning and programming languages. 2019,PP(99):58-68.
- [55]Ramroach S, Joshi A. Accelerating Data-Parallel Neural Network Training with Weighted-Averaging Reparameterisation[J]. Parallel Processing Letters, 2021, 31(2):2150009.
- [56]Yu Y, Zhang M. Control Chart Recognition Based on the Parallel Model of CNN and LSTM with GA Optimization[J]. Expert Systems with Applications, 2021,185(1): 115689.
- [57]Li J Y, Zhan Z, Liu R, et al. Generation-Level Parallelism for Evolutionary Computation: A Pipeline-Based Parallel Particle Swarm Optimization[J]. IEEE Transactions on Cybernetics, 2020, 51(10): 4848-4859.
- [58]Llopart A, Ravn O, Andersen N A , et al. Generalized framework for the parallel semantic segmentation of multiple objects and posterior manipulation[C]// 2018 IEEE International Conference on Robotics and Biomimetics (ROBIO). IEEE, 2018,PP(99): 561-568.
- [59]Szczotka A B, Shakir D I, Clarkson M J, et al. Zero-shot super-resolution with a

- physically-motivated downsampling kernel for endomicroscopy[J]. IEEE Transactions on Medical Imaging, 2021. 40(7):1863-1874.
- [60]Lai Y, Fan F, Wu Q, et al. LCANet: Learnable Connected Attention Network for Human Identification Using Dental Images[J]. IEEE Transactions on Medical Imaging.2022 40(3): 905-915.

附录二 流水线算法和 Greedy 算法流程

1. 流水线并行实现流程

目前很多边缘计算使用 SOC 作为推理设备，流水线机制提出可以让开发者使用多个设备进行运算，同时进行并行操作。现有的解决方案是在不同的设备上进行处理，例如图像 1 在 CPU 进行处理、图像 2 在 GPU 上以及图像 3 在 FPGA 上进行处理。但是，不同设备存在不同延迟，像 CPU 这种运算速度较慢的设备比 FPGA 的运算速度慢几倍。当使用整个计算图作为调度单元时，就会涉及延迟问题，如何更好的减少延迟，以加快处理速度、获得更好的吞吐量成为一个需要解决的问题。

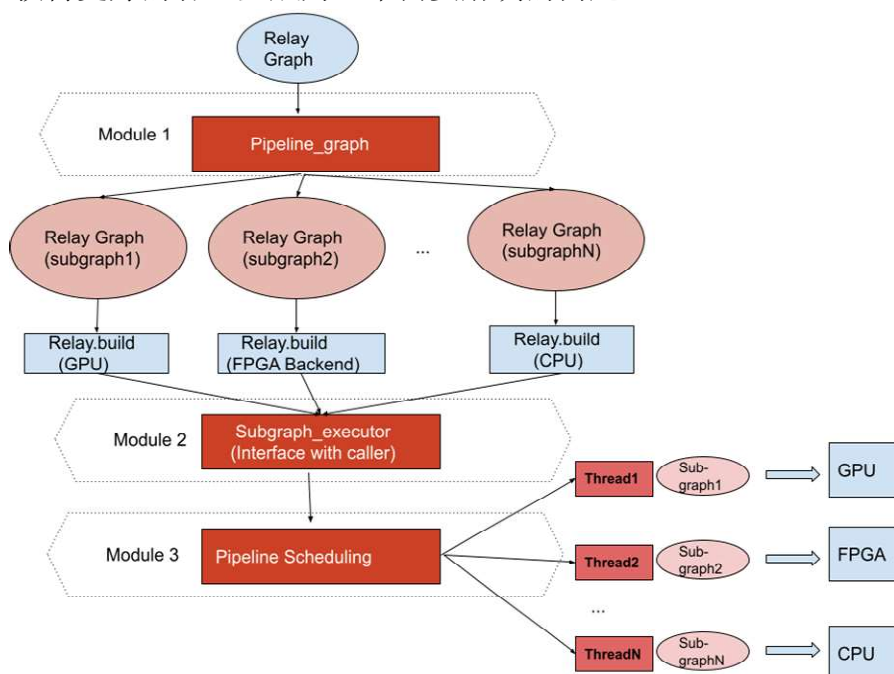


图6.1 流水线机制流程图

因此，需要将计算图划分为更小的调度单元，即切分成子图，子图间构成流水线执行，以更有效地使用异构硬件资源，达到更好的性能。流水线机制可以使用更多后端进行并行操作，此外，有了 RPC（远程控制）机制，流水线的后端可以实现云端互联。该框架首先将中继图分解为多个子图或多个组，然后进行子图和组的调度，主要包括 3 个部分。

第一部分为子图划分。子图划分使用包含 LetNode 节点的计算图，对其进行手动划分，划分示意图如下。

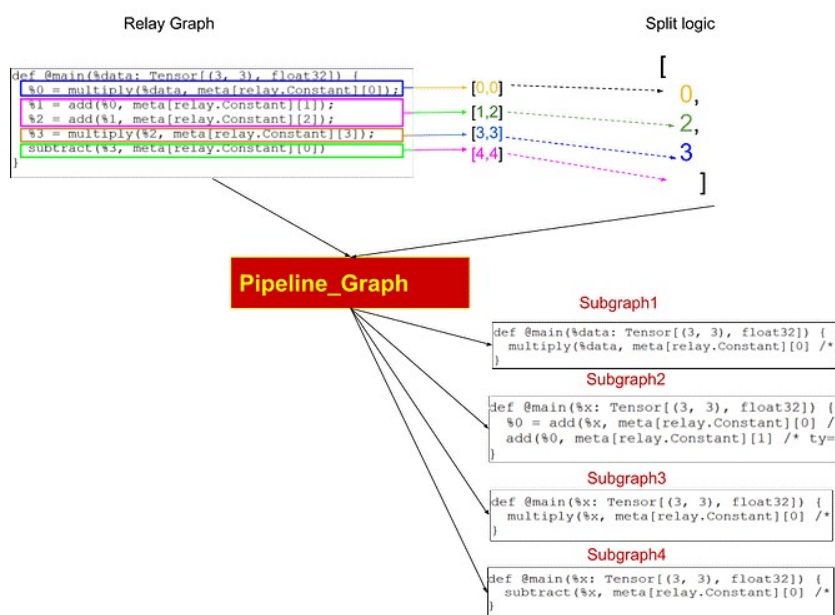


图6.2 子图划分示意

图 6.2 中为自定义的计算图以及需要手动输入的划分位置，子图划分通过识别各个节点在计算图中的默认标号来判断是否需要划分。划分时对计算图采取从上往下的顺序进行划分，即子图 1-子图 2-子图 3-子图 4 的顺序。划分完成后需要手动对各个子图的依赖关系进行逐一配置，主要配置子图间的输入输出关系以及后端设备的选定。

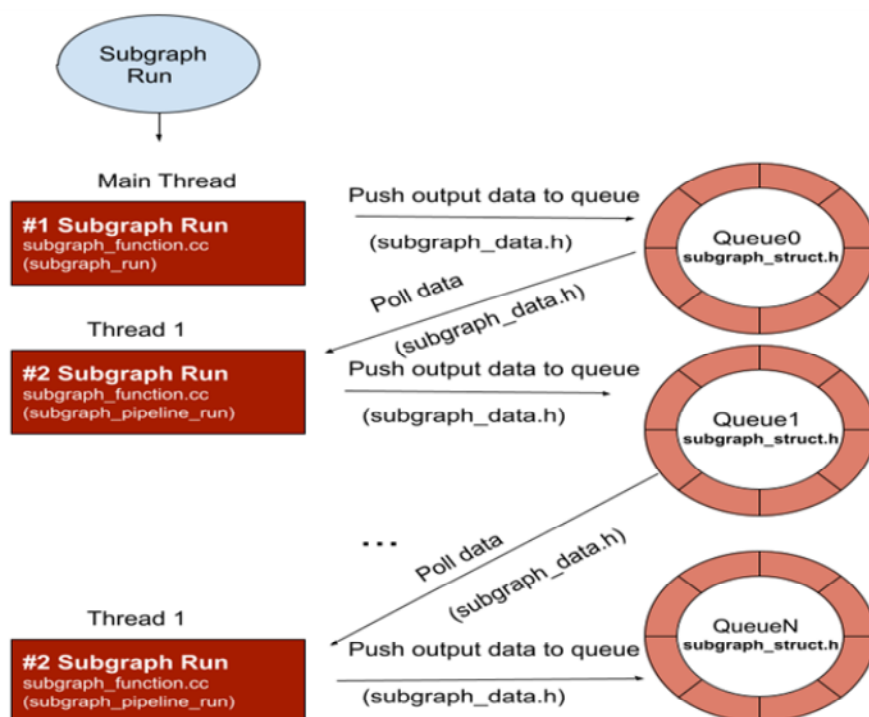


图6.3 子图运行流程

第二部分为图运行时，在将计算图划分为一组子图后，用户可以使用任意后端来构

建这些子图，之后子图运行时将在流水线模块中执行并调度这些子图进行推理。子图运行时可以分为两个模块，第一是接口部分，负责响应与调用函数进行交互，主要用来设置数据、参数和模块的初始化配置以及调度信息。第二部分为调度部分，用于响应子图的管道在不同子图间的数据传输。

第三部分为接口，接口部分分为三个函数：前端子图执行文件，后端子图执行头文件和后端子图执行文件。前端子图执行文件主要用来接收数据和参数，用于子图运行时的创建。它通过 TVM 中专用的接口唤起后端部分的子图执行文件，然后向后端传输数据，设置输入等操作，同时也可以唤起后端以获得输出结果。前端子图执行文件除了以上功能外，还为各个子图使用流水线专用的子图执行器对每个子图进行构建，子图执行器的内部实现交由后端子图执行文件进行执行。后端子图执行文件负责处理 Caller 和低级调度原语之间的交互，使用 `SubGraphRuntime` 进行底层调度逻辑的处理，如流水线模式的开始和结束等操作。同时提供数据变量的存储操作。

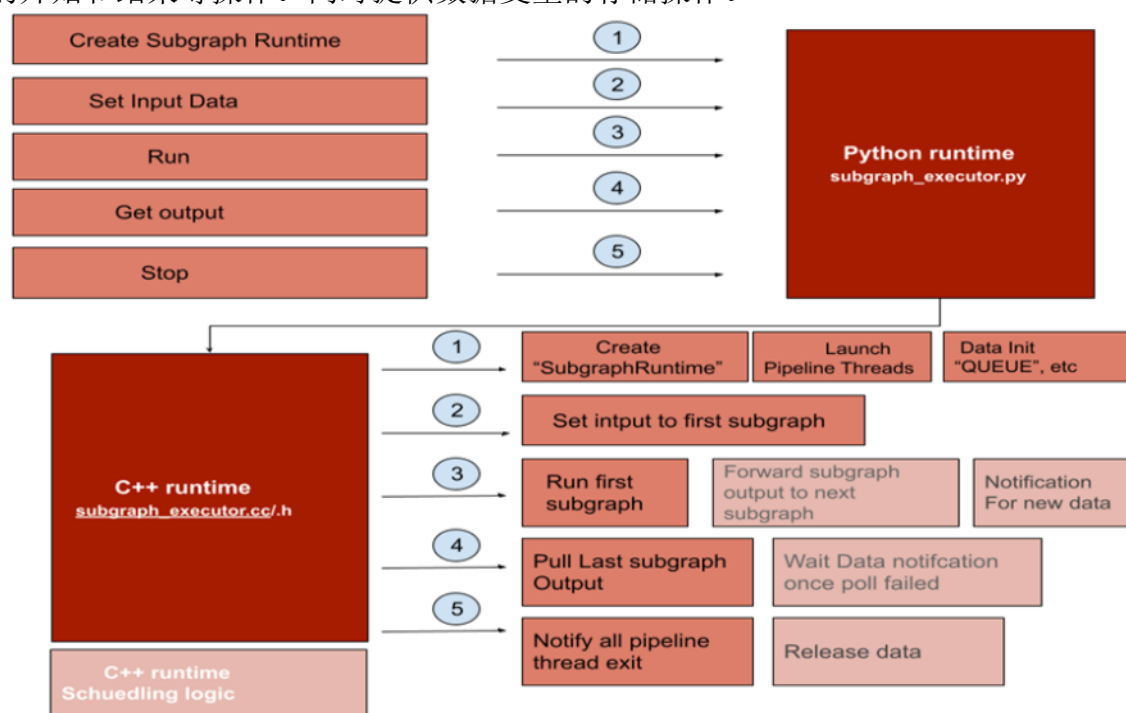


图6.4 流水线详细接口

2. Greedy 算法具体流程

传统 Greedy 算法在运行时主要通过将可并行的分支进行直接并行，减少了调度时间，并且取得一定的优化效果。但是，传统算法当需要同时并行的算子过多时，会出现资源争用的问题，出现延迟情况，如图 6.5 所示。因此，文献[40]提出了 Greedy 的优化

算法, 该方法通过评估函数, 得出同一阶段可并行的最大节点数量, 用来减少延迟。然后由多个阶段组成一个块, 对一个块统一进行调度, 再由多个块组成计算图, 完成图的调度。

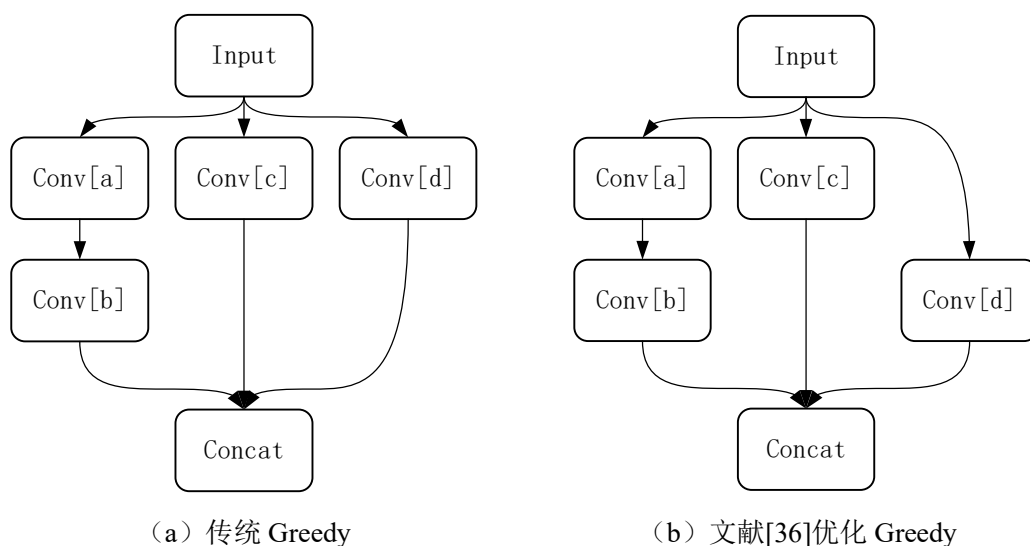


图6.5 Greedy 算法优化前后过程示例

如下图 6.6 所示, 为优化 Greedy 算法的调度流程。将计算图表示为点集合 V , V 中含有当前图的所有算子 $a-g$ 。在划分时, 优先从图的结束节点开始, 将符合条件的集合纳入 S' , 在生成 S' 后, 需要对内部的节点进行进一步判断, 当满足结束节点都存在于 S' 中时, 停止对集合的查找。当集合 S' 查找完成后, 对剩余点集合重复进行上述操作, 以此将计算图 V 划分为 S'_1 、 S'_2 、 S'_3 。

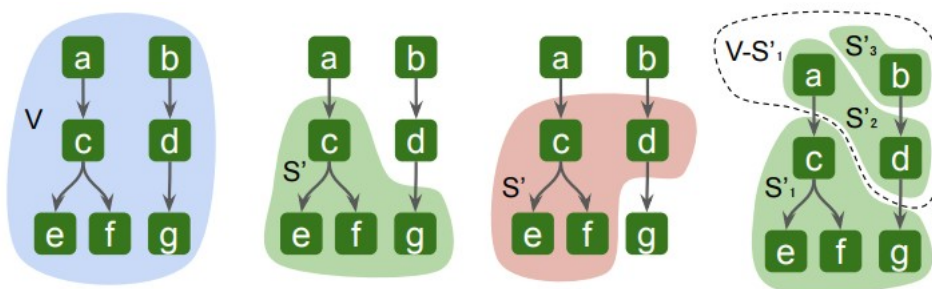


图6.6 优化 Greedy 调度过程示例

优化后的 Greedy 算法主要分为三部分, 如下表 6.1 伪代码所示, 主要包括算子间调度 `OperatorScheduler`、核心调度 `Scheduler` 和阶段生成 `GenerateStage` 三个函数。

算子间调度模块 (L3-L11) 将计算图作为输入, 并返回优化后的调度 Q 。这个模块首先调用 `Scheduler` 模块, 在调用此模块之后, 全局变量 `cost[S]` 存储 S 的调度延迟, `choice[S]` 存储对应调度的最后一个阶段。当 `choice` 数组全部得到时, 可以通过 (L6-L10) 来进行最有调度的选择。

核心调度模块 (L12-L21) 采用递归实现动态规划算法, 以一组调度 S 作为输入,

返回所有调度中最小延迟的调度。在算法中,使用阶段生成模块获取更好的并行方法和对应延迟,再将 $S-S'$ 作为子问题进行递归。为避免出现重复计算,将之前递归的结果保存到 $\text{cost}[S]$ 中,并作为函数返回值。

表6.1 优化 Greedy 伪代码

Function: Greedy

In: 计算图 $G=(V,E)$, 调度策略 P

Out: 最优调度策略

```

1.   $\text{cost}[S] = \infty$ 
2.   $\text{choice}[S] = \emptyset$ 
3.  OpeatorScheduler(G):
4.      Scheduler(V)
5.       $Q = []$ 
6.       $S = V$ 
7.      While  $S \neq \emptyset$  do
8.           $S'T = \text{choice}[S]$ 
9.          Insert ( $S', T$ ) into Q
10.          $S = S - S'$ 
11.     Return Q
12. Scheduler(S):
13.     If  $\text{cost}[S] \neq \infty$  then
14.         Return  $\text{cost}[S]$ 
15.     For all P do
16.          $Ls', Ts' = \text{GenerateStage}(S')$ 
17.          $Ls = \text{Scheduler}(S - S') + Ls'$ 
18.         If  $Ls < \text{cost}[S]$  then
19.              $\text{Cost}[S] = Ls$ 
20.              $\text{Choice}[S] = (S', Ts')$ 
21.         Return  $\text{cost}[S]$ 
22. GenerateStage( $S'$ ):
23.     Partition  $S'$  to  $S'_1, S'_2, \dots, S'_k$ .
24.      $L_{\text{concurrent}} = \text{LatencyOfParallel}\{S'_i\}$  \parallel 并行执行的延迟
25.     If operators in  $S'$  then \parallel 如果  $S'$  内的算子可以被融合
26.          $L_{\text{merge}} = \text{latency of merged operator}$  \parallel 融合后算子的延迟
27.     Else
28.          $L_{\text{merge}} = \infty$ 
29.     If  $L_{\text{concurrent}} < L_{\text{merge}}$  then
30.         Return  $L_{\text{concurrent}}$ , "concurrent execution"
31.     Else
32.         Return  $L_{\text{merge}}$ , "operator merge"

```

阶段生成模块（L22-L32）负责对输入的 S' 进行判断，从并行和融合两者之间选择更好的策略。该函数分别获取并行执行和算子融合后的调度延迟，然后对两者进行判断，返回延迟小的策略。



硕士学位论文
Thesis for Master's Degree